Minnesota State University, Mankato

CIS4/544 REPORT

Sprint 6

Team 1:

David Onyango	davidomondi.onyango@mnsu.edu
Eunsoo Park	eunsoo.park@mnsu.edu
Kiara Boyer	kiara.boyer@mnsu.edu
Pramudya Wicaksono	pramudya.wicaksono@mnsu.edu
Rei Onishi	rei.onishi@mnsu.edu

TABLE OF CONTENTS

Introduction..... 4
Objectives..... 4
Deliverables..... 5
1. Implementation of DDBS Strategy: Linking all servers from MSSQL3.....5
1.1. Verify that the whole strategy is implemented by linking these servers. Support with screenshots to clearly identify that you have linked all servers according to our DDBS strategy. All queries should be initiated from MSSQL3.....5
2. Screenshot proof that views are in MSSQL2.....5
3. Verify (screenshot proof) that you are connecting to Oracle1, Oracle2, MSSQL1, and MSSQL2 from MSSQL3.....6
4. Qlist1 implementation using DDBS setup using Linked Services.....6
4.1. For every schema, list Schema, # tables, # records.....6
4.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"..... 9
4.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name..... 12
4.4. For every schema, analyze PK, FK,and IDX..... 16
4.4.1. a: list tables without PK..... 16
4.4.2. b: list tables with PK but without FK.....17
4.4.3. c: list tables with PK but without IDX.....19
4.5. For every database/schema,..... 21
4.5.1. a: rank them according to number of NN (Not Null) have created.....21
4.5.2. b: For every table list fields with NN.....22
4.6. For every database, list tables without PK..... 25
4.7. For every database, list tables without IDX.....27
4.8. For every database, list tables with PK but without FK.....29
4.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX..... 31
4.10. For every database/schema, list tables with Check Constraints..... 32
5. Qlist2 implementation using DDBS setup using Linked Services..... 34
5.1. List employee (number, last name, first name, and the decision made about their performance..... 34
5.2. List all male employees (number, last name, first name, position, decision) that are related to sales and are not fired.....35
5.3. List all employees that are related to sales and are given raise.....35
5.4. List all employees that their performance for sales attribute category is excellent..... 36
5.5. List all employees (number, last name, first name) that have had more than one order. 37
5.6. List all employees (number, last name, first name, order number) that have had more than three products in their order..... 38
5.7. List the top 10 employees (number, last name, first name, rank, total gross sale) in term

of their total sales (gross sale)..... 38
5.8. List the bottom 10 employees (number, last name, first name, rank, number of orders) in term of their total orders..... 39
5.9. List the top 25% of employees (number, last name, first name, rank, total gross sale) in term of their total sales (gross sale).....40
5.10. In the decision table, how many employees are in each decision?..... 40
5.11. How many employees are in each of the Number_Orders, Number_Products, and sales fuzzy categories?.....41
5.12. Based on each state, how many employees are in each of the decision categories?.....42
5.13. Based on employee position, how many employees are in each of the fuzzy decision categories?.....42
5.14. Based on each branch, how many employees are in each of the fuzzy decision categories?.....43
6. Qlist3 - MLS2: Player Performance Analysis..... 44
6.1. Ranking player stat based on all game (Home and away) within player team. This should be done for every year as well as all years.....44
6.2. Ranking player stat based on all home game within player team. This should be done for every year as well as all years..... 45
6.3. Ranking player stat based on all away game within player team. This should be done for every year as well as all years..... 47
6.4. Analysis of player performance by comparing home game and away games..... 49
6.5. Ranking player yearly performance based on nationality.....50
6.6. Ranking player yearly performance based on salary.....52
6.7. Ranking player performance based on their age..... 52
6.8. Develop a report to show performance of top 10 players of each team based on their salary for every year. Performance, for every player, in terms of starts, Min, GLS, AST, GLS_AST, G_PK, PK, PKATT, CRDY, and with their rank in their team as well rank in their assigned conference and all conferences.....53
6.9. Develop a report to show performance of drafted players for every year. Performance, for every player, in terms of starts, Min, GLS, AST, GLS_AST, G_PK, PK, PKATT, CRDY, and with their rank in their team as well rank in their assigned conference..... 55
6.10. Develop a report to show performance of drafted players for every year. Performance, for every player, in terms of starts, Min, GLS, AST, GLS_AST, G_PK, PK, PKATT, CRDY, and with their rank in their team as well rank in their assigned conference..... 57
6.11. Develop a report to show performance of foreign players for every year. Performance.... 57
6.12. Develop a report to show performance of drafted players for every year. Performance, for every player, in terms of starts, Min, GLS, AST, GLS_AST, G_PK, PK, PKATT, CRDY, and with their rank in their team as well rank in their assigned conference..... 59
7. Qlist3 - MLS2: Team Performance Analysis.....62
7.1. Ranking team stat based on all game (Home and away) within team conference. This should be done for every year as well as all years..... 62
7.2. Ranking team stat based on all home game within team conference. This should be done for every year as well as all years.....63
7.3. Ranking team stat based on all away game within team conference. This should be done for every year as well as all years.....65

7.4. Analysis of team performance by comparing home game and away games..... 68
7.5. Ranking team yearly performance based on percentage of participating nationalities within team conference as well as all conferences..... 70
7.6. Ranking player yearly performance based on total salary paid within team conference as well as all conferences..... 72
7.7. Ranking player performance based on range of ages within team conference as well as all conferences. Develop a report to show performance of top 10 teams how every team based on total wage budge for every year. Performance in terms of wins. Losses, draw, total point (win=3, draw=1. Loss=0) with their rank in their assigned conference..... 73
8. Qlist3 - MLS2 Referee Analysis..... 75
8.1. Develop a report to show performance of referee for every year. Performance, for every referee, in terms of home team, away team, home team win, home team loss, away team win, away team loss, PK given, CRDY, CRDR and with their rank in all games they refereed. Rank should be based on PK, CRDY, and CRDR..... 75
Scrum Report..... 79
Analysis..... 83
Appendix..... 83

Introduction

During Sprint 6, which took place from April 9th to May 3rd, 2025, the Distributed Database System (DDBS) technique was employed to re-implement important elements of previous sprints. The primary objective was to reorganize our earlier work, precisely from Sprints 2, 4, and 5, among several Docker-hosted servers, such as Oracle1, Oracle2, MSSQL1-3, and MongoDB, in a distributed design.

In order to guarantee that all queries were started from MSSQL3, connected services were used during this sprint. Data marts were centralized as views in MSSQL2. For QList1, Pivot User Interface (Pivot_UI), and Power BI User Interface (PBI_UI), we redesigned SQL implementations while ensuring consistency with previous results. The accuracy and improvements under the new DDBS configuration were also confirmed by comparing all results against Sprint 2 and Sprint 5 outputs and validating them with screenshots.

We improved data accessibility, modularity, and system stability by modifying our data analytics system to operate in a distributed setting. This opened the door for effective and scalable business analysis.

Objectives

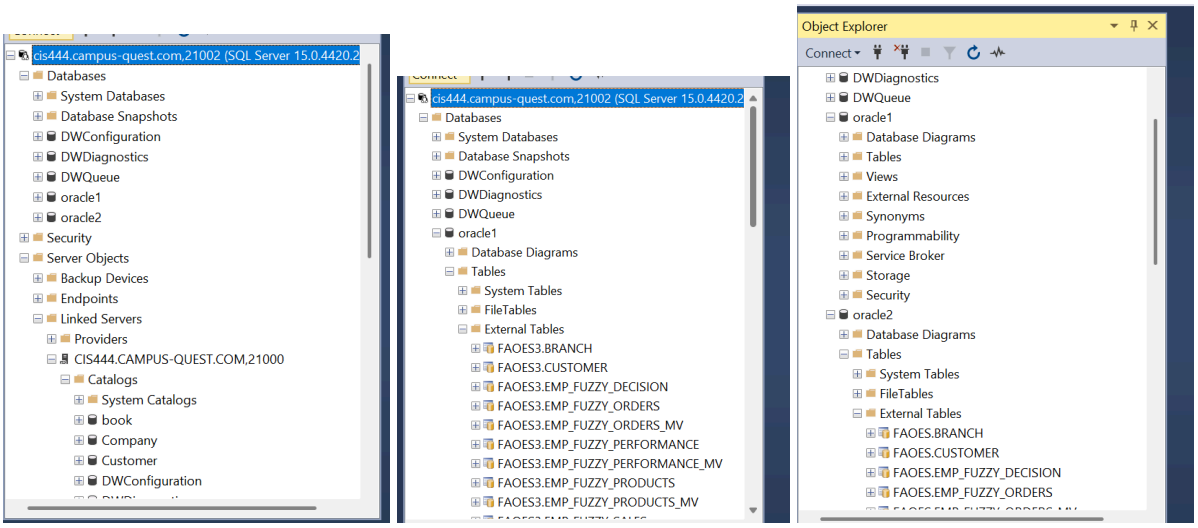
The objective of Sprint 6 is to re-implement key components from previous sprints using the Distributed Database System (DDBS) approach. The focus was on reorganizing the work from Sprints 2, 4, and 5 across multiple Docker-hosted servers, including Oracle1, Oracle2, MSSQL1-3, and MongoDB, to establish a distributed design. To ensure all queries originated from MSSQL3, connected services were leveraged, and data marts were centralized as views in MSSQL2.

We also redesigned the SQL implementations for QList1, Pivot User Interface (Pivot_UI), and Power BI User Interface (PBI_UI) while maintaining consistency with prior results. The improvements and accuracy of the new DDBS setup were validated by comparing outputs with those from Sprint 2 and Sprint 5 and cross-referencing them with screenshots. This reconfiguration enhanced data accessibility, modularity, and system stability, laying the groundwork for more effective and scalable business analysis.

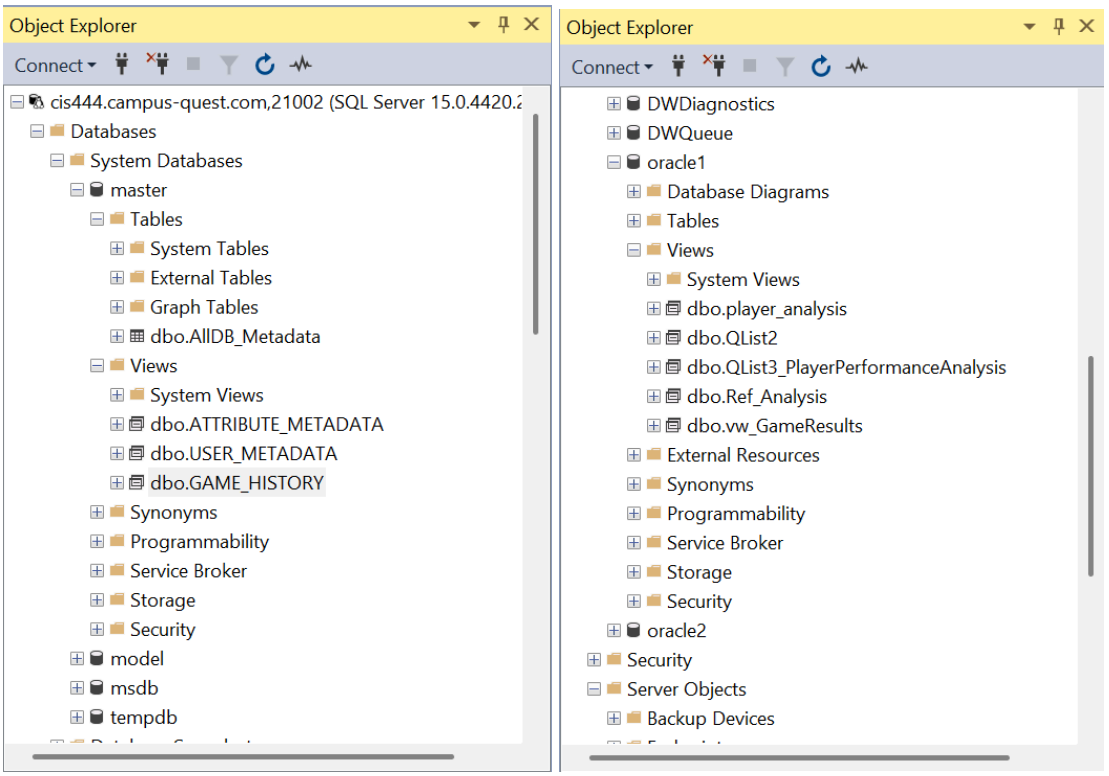
Deliverables

1. **Implementation of DDBS Strategy: Linking all servers from MSSQL3**

   1.1. **Verify that the whole strategy is implemented by linking these servers. Support with screenshots to clearly identify that you have linked all servers according to our DDBS strategy. All queries should be initiated from MSSQL3**

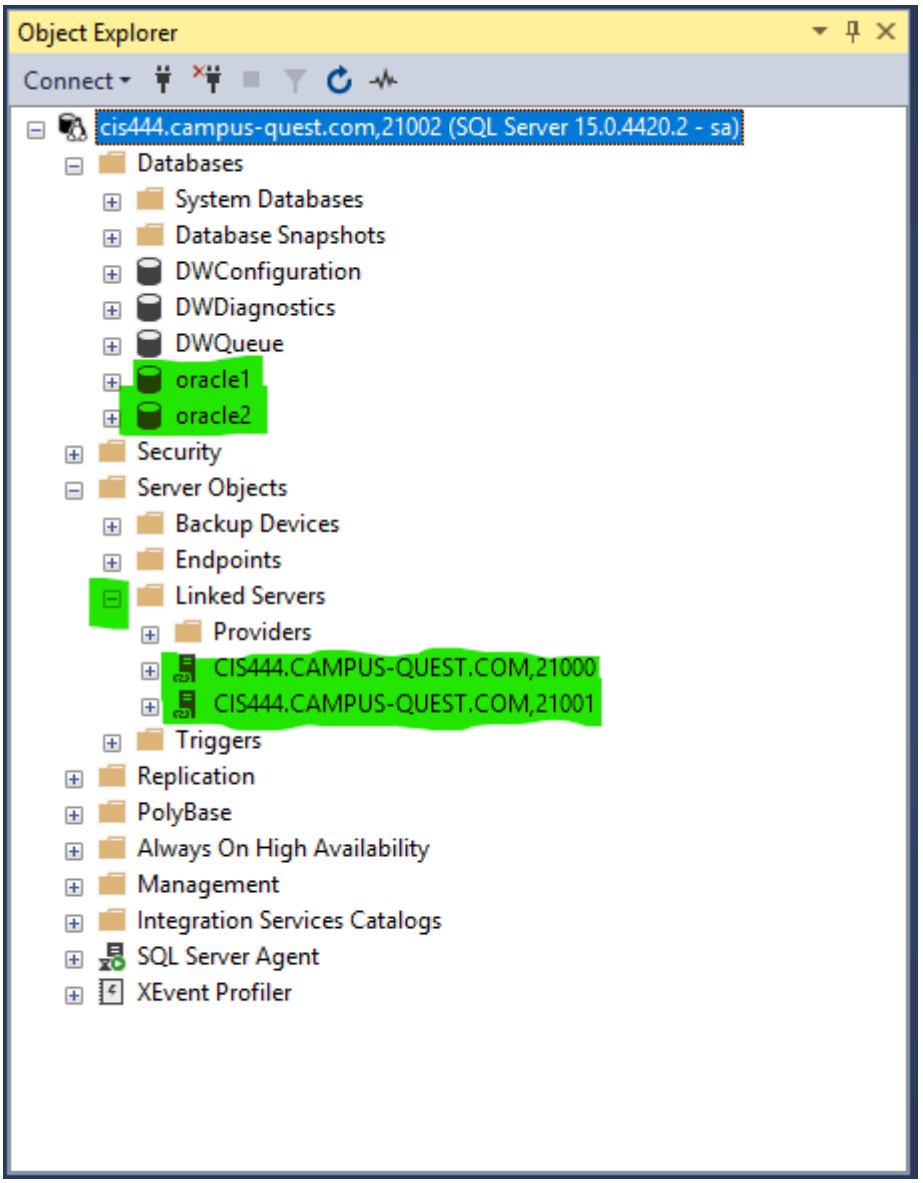


2. **Screenshot proof that views are in MSSQL2**



We used ALLDB_Metadata for Qlist1, Qlist 2 for Qlist2, and GAME_HISTORY, player_analysis, vw_GameResults for Qlist3.

3. **Verify (screenshot proof) that you are connecting to Oracle1, Oracle2, MSSQL1, and MSSQL2 from MSSQL3**



4. **Qlist1 implementation using DDBS setup using Linked Services**

4.1. For every schema, list Schema, # tables, # records

4.1.1. Query

```
IF OBJECT_ID('GetSchemaTableRecordCounts', 'P') IS NOT NULL
    DROP PROCEDURE GetSchemaTableRecordCounts;
GO

CREATE PROCEDURE GetSchemaTableRecordCounts
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @DatabaseName NVARCHAR(255);
    DECLARE @SQL NVARCHAR(MAX);

    CREATE TABLE #SchemaSummary (
        DatabaseName NVARCHAR(128),
        SchemaName NVARCHAR(128),
```

```
        TableCount INT,
        RecordCount BIGINT
    );

    DECLARE db_cursor CURSOR FOR
    SELECT name
    FROM sys.databases
    WHERE name NOT IN ('master', 'tempdb', 'model', 'msdb');

    OPEN db_cursor;
    FETCH NEXT FROM db_cursor INTO @DatabaseName;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        SET @SQL = '
        USE [' + @DatabaseName + '];

        WITH TableCounts AS (
            SELECT
                s.name AS SchemaName,
                t.name AS TableName,
                SUM(p.rows) AS RecordCount
            FROM
                sys.schemas s
            JOIN
                sys.tables t ON s.schema_id = t.schema_id
            JOIN
                sys.partitions p ON t.object_id = p.object_id
            WHERE
                p.index_id IN (0, 1)
            GROUP BY
                s.name, t.name
        )
        SELECT
            ''' + @DatabaseName + ''' AS DatabaseName,
            SchemaName,
            COUNT(*) AS TableCount,
            SUM(RecordCount) AS RecordCount
        FROM TableCounts
        GROUP BY SchemaName;';

        INSERT INTO #SchemaSummary
        EXEC sp_executesql @SQL;

        FETCH NEXT FROM db_cursor INTO @DatabaseName;
    END;

    CLOSE db_cursor;
    DEALLOCATE db_cursor;

    SELECT * FROM #SchemaSummary
    ORDER BY DatabaseName, SchemaName;

    DROP TABLE #SchemaSummary;
END;
GO

EXEC GetSchemaTableRecordCounts;
```

4.1.2. Result

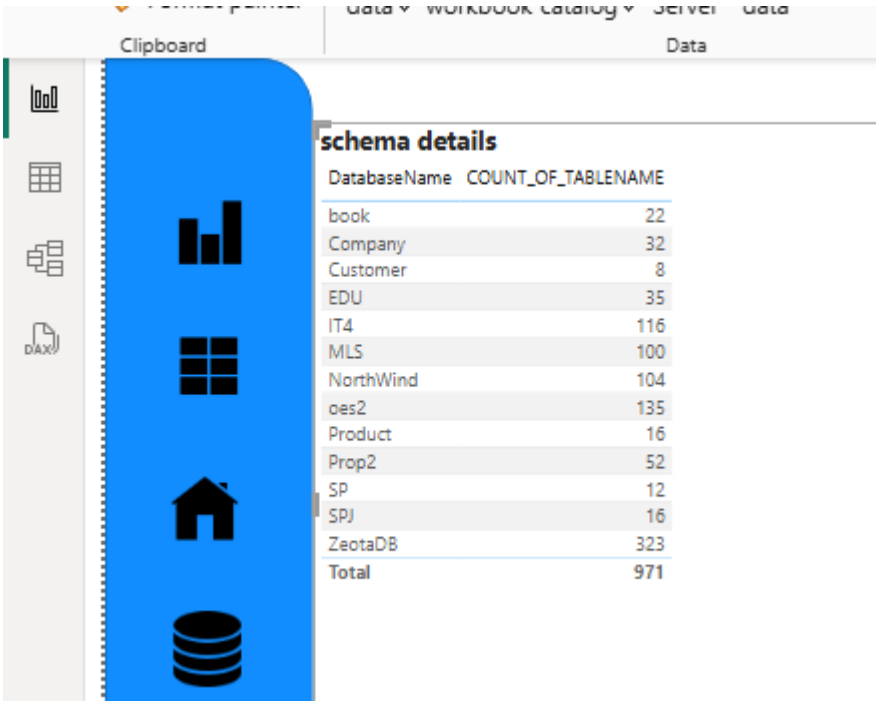
DatabaseName	SchemaName	TableCount	RecordCount
book	dbo	6	173

Company	dbo	6	45
DWConfiguration	dbo	2	165
…	…	…	…
SP	dbo	3	23
SPJ	dbo	4	42
ZeotaDB	dbo	72	14007

4.1.3. Pivot UI redevelopment

Row Labels	Count of Column_Name
book	**22**
AUTHOR	3
BOOK	6
branch	4
INVENTORY	3
PUBLISHER	3
WROTE	3
Company	**32**
Department	4
DEPENDENT	6
DEPT_LOCATIONS	3
EMPLOYEE	10
PROJECT	4
WORKS_ON	5
Customer	**8**
CUSTOMER	8
EDU	**35**
COURSE	4
Department	8
DEPT_CHAIR	2
ENROLLMENT	6
STUDENT	6
TEACHER	9
IT4	**116**
CERT	5
CERTIFICATION	5

4.1.4. Power BI redevelopment



DatabaseName	COUNT_OF_TABLENAME
book	22
Company	32
Customer	8
EDU	35
IT4	116
MLS	100
NorthWind	104
oes2	135
Product	16
Prop2	52
SP	12
SPJ	16
ZeotaDB	323
Total	**971**

4.1.5. Comparison

Although the scope and execution of the two queries vary, they both successfully return schema-level summaries of table and record counts. For centralized metadata analysis, the Oracle query from Sprint 2 reports metrics across designated schemas in a single-instance context using a straightforward collect over the ALL_TABLES system view. The SQL Server process from Sprint 6, on the other hand, works in a distributed environment across several databases, dynamically creating and running SQL using cursors to gather comparable metadata. This makes it appropriate for enterprise-level integrity verification across connected services. Although they both have the same function, the SQL Server script is designed for scalability and cross-database reporting, while the Oracle query is intended for speed and simplicity.

4.2. For every schema, list table name, # attributes, # PK_ attr, # FK_ attr, # NN_attr, # IDX_attr, # CC_attr, # UQ_attrs, # PGM_attr"

4.2.1. Query

```
IF OBJECT_ID('GetTableAttributeStats', 'P') IS NOT NULL
    DROP PROCEDURE GetTableAttributeStats;
GO

CREATE PROCEDURE GetTableAttributeStats
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @DatabaseName NVARCHAR(255);
    DECLARE @SQL NVARCHAR(MAX);
    CREATE TABLE #AttributeStats (
        DatabaseName NVARCHAR(128),
        SchemaName NVARCHAR(128),
        TableName NVARCHAR(128),
        AttributeCount INT,
        PK_Attr INT,
        FK_Attr INT,
        NN_Attr INT,
```

```
        IDX_Attr INT,
        UQ_Attr INT,
        PGM_Attr INT
    );

    DECLARE db_cursor CURSOR FOR
    SELECT name
    FROM sys.databases
    WHERE name NOT IN ('master', 'tempdb', 'model', 'msdb');

    OPEN db_cursor;
    FETCH NEXT FROM db_cursor INTO @DatabaseName;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        SET @SQL = '
        USE [' + @DatabaseName + '];

        SELECT
            ''' + @DatabaseName + ''' AS DatabaseName,
            s.name AS SchemaName,
            t.name AS TableName,
            COUNT(c.column_id) AS AttributeCount,
            SUM(CASE WHEN i.is_primary_key = 1 THEN 1 ELSE 0 END) AS PK_Attr,
            SUM(CASE WHEN fk.parent_column_id IS NOT NULL THEN 1 ELSE 0 END) AS FK_Attr,
            SUM(CASE WHEN c.is_nullable = 0 THEN 1 ELSE 0 END) AS NN_Attr,
            SUM(CASE WHEN ic.index_column_id IS NOT NULL THEN 1 ELSE 0 END) AS IDX_Attr,
            SUM(CASE WHEN i.is_unique = 1 THEN 1 ELSE 0 END) AS UQ_Attr,
            SUM(CASE WHEN c.is_computed = 1 THEN 1 ELSE 0 END) AS PGM_Attr
        FROM
            sys.tables t
        JOIN
            sys.schemas s ON s.schema_id = t.schema_id
        JOIN
            sys.columns c ON c.object_id = t.object_id
        LEFT JOIN
            sys.index_columns ic ON ic.object_id = c.object_id AND ic.column_id =
c.column_id
        LEFT JOIN
            sys.indexes i ON i.object_id = t.object_id AND i.index_id = ic.index_id
        LEFT JOIN
            sys.foreign_key_columns fk ON fk.parent_object_id = t.object_id AND
fk.parent_column_id = c.column_id
        GROUP BY
            s.name, t.name;';

        INSERT INTO #AttributeStats
        EXEC sp_executesql @SQL;

        FETCH NEXT FROM db_cursor INTO @DatabaseName;
    END;

    CLOSE db_cursor;
    DEALLOCATE db_cursor;

    SELECT * FROM #AttributeStats
    ORDER BY DatabaseName, SchemaName, TableName;

    DROP TABLE #AttributeStats;
END;
GO

EXEC GetTableAttributeStats;
```

4.2.2. Result

DatabaseName	SchemaName	TableName	AttributeCount	PK_Attr	FK_Attr	NN_Attr	IDX_Attr	UQ_Attr	PGM_Attr
book	dbo	AUTHOR	3	1	0	1	1	1	0
book	dbo	BOOK	6	1	0	1	1	1	0
book	dbo	BRANCH	4	1	0	1	1	1	0
…	..	..	…	…	…	…	…	…	…
ZeotaDB	dbo	VENDORPRICE	5	1	2	4	1	1	0
ZeotaDB	dbo	Vndor_PROMOTION	6	1	2	1	1	1	0
ZeotaDB	dbo	Warehouse	7	1	0	1	1	1	0

4.2.3. Pivot UI redevelopment

Row Labels	Count of PK_Constraint_Name
book	**22**
AUTHOR	3
BOOK	6
branch	4
INVENTORY	3
PUBLISHER	3
WROTE	3
Company	**32**
Department	4
DEPENDENT	6
DEPT_LOCATIONS	3
EMPLOYEE	10
PROJECT	4
WORKS_ON	5
Customer	**8**
CUSTOMER	8
EDU	**35**
COURSE	4
Department	8
DEPT_CHAIR	2
ENROLLMENT	6
STUDENT	6
TEACHER	9

4.2.4. Power BI redevelopment

schema details

DatabaseName	COUNT_OF_HAS_FK	COUNT_OF_HAS_PK	COUNT_OF_NOT_NULLABLE	count_of_IDX
book		22	22	8
Company	10	32	32	15
Customer		8	8	1
EDU	8	35	35	10
IT4	31	116	116	62
MLS	22	100	100	35
NorthWind	15	104	104	18
oes2	3	117	135	19
Product	2	16	16	3
Prop2	2	42	52	10
SP	2	12	12	4
SPJ	3	16	16	15
ZeotaDB	96	323	323	91
Total	**194**	**943**	**971**	**291**

4.2.5. Comparison

Although the systems and contexts of the two queries are different, they both offer detailed attribute-level statistics per table. For rapid, comprehensive analysis in a single database, the Oracle query from Sprint 2 reports counts of important constraints, indexes, and attributes within a single schema using system views and targeted joins. The SQL Server method from Sprint 6 is ideal for enterprise-wide metadata inspections because it uses dynamic SQL and cursors to produce comparable findings across many databases. Sprint 6 provides scalability and cross-database coverage, whereas Sprint 2 prioritizes directness and clarity.

4.3. For every schema, list Table_ID, Table_Name, attribute_id, attribute_Name, PK_ID, FK_ID, NN_ID, constraint_type, and constraint_Name

4.3.1. Query

```
IF OBJECT_ID('GetLinkedServerTableAttributeDetails', 'P') IS NOT NULL
    DROP PROCEDURE GetLinkedServerTableAttributeDetails;
GO

CREATE PROCEDURE GetLinkedServerTableAttributeDetails
    @LinkedServerName NVARCHAR(128)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @DatabaseName NVARCHAR(255);
    DECLARE @SQL NVARCHAR(MAX);

    CREATE TABLE #AttributeDetails (
        DatabaseName NVARCHAR(128),
        SchemaName NVARCHAR(128),
        Table_ID INT,
        Table_Name NVARCHAR(128),
        Attribute_ID INT,
        Attribute_Name NVARCHAR(128),
        PK_ID INT,
        FK_ID INT,
        NN_ID INT,
        Constraint_Type NVARCHAR(60),
```

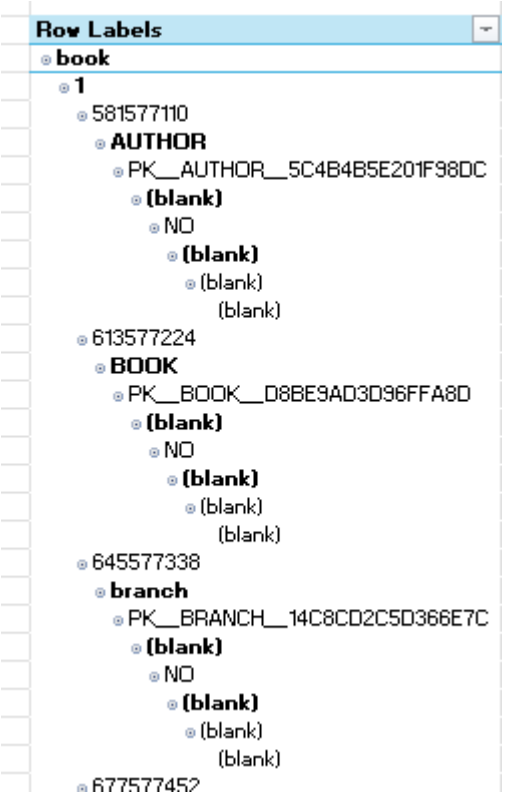
```
        Constraint_Name NVARCHAR(128)
    );

    -- Cursor over databases
    DECLARE db_cursor CURSOR FOR
    SELECT name
    FROM OPENQUERY([cis444.campus-quest.com,21000], 'SELECT name FROM sys.databases WHERE
name NOT IN (''master'', ''tempdb'', ''model'', ''msdb'')');

    OPEN db_cursor;
    FETCH NEXT FROM db_cursor INTO @DatabaseName;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        SET @SQL = N'
        INSERT INTO #AttributeDetails
        SELECT
            ''' + @DatabaseName + ''' AS DatabaseName,
            s.name AS SchemaName,
            t.object_id AS Table_ID,
            t.name AS Table_Name,
            c.column_id AS Attribute_ID,
            c.name AS Attribute_Name,
            CASE WHEN pkc.column_id IS NOT NULL THEN c.column_id ELSE NULL END AS PK_ID,
            fk.parent_column_id AS FK_ID,
            CASE WHEN c.is_nullable = 0 THEN c.column_id ELSE NULL END AS NN_ID,
            kc.type_desc AS Constraint_Type,
            kc.name AS Constraint_Name
        FROM OPENQUERY([' + @LinkedServerName + N'],
        ''
        SELECT
            s.name AS schema_name,
            t.object_id AS table_id,
            t.name AS table_name,
            c.column_id,
            c.name AS column_name,
            c.is_nullable,
            fk.parent_column_id,
            kc.type_desc,
            kc.name AS constraint_name,
            pkc.column_id AS pk_column_id
        FROM ' + @DatabaseName + '.sys.tables t
        JOIN ' + @DatabaseName + '.sys.schemas s ON s.schema_id = t.schema_id
        JOIN ' + @DatabaseName + '.sys.columns c ON c.object_id = t.object_id
        LEFT JOIN ' + @DatabaseName + '.sys.foreign_key_columns fk ON fk.parent_object_id =
t.object_id AND fk.parent_column_id = c.column_id
        LEFT JOIN ' + @DatabaseName + '.sys.key_constraints kc ON kc.parent_object_id =
t.object_id
        LEFT JOIN ' + @DatabaseName + '.sys.indexes i ON i.object_id = t.object_id AND
i.is_primary_key = 1
        LEFT JOIN ' + @DatabaseName + '.sys.index_columns pkc ON pkc.object_id =
t.object_id AND pkc.column_id = c.column_id AND pkc.index_id = i.index_id
        '') as Q
        JOIN sys.schemas s ON s.name COLLATE DATABASE_DEFAULT = Q.schema_name COLLATE
DATABASE_DEFAULT
        JOIN sys.tables t ON t.object_id = Q.table_id
        JOIN sys.columns c ON c.object_id = t.object_id AND c.column_id = Q.column_id
        LEFT JOIN sys.key_constraints kc ON kc.parent_object_id = t.object_id AND kc.name
COLLATE DATABASE_DEFAULT = Q.constraint_name COLLATE DATABASE_DEFAULT
        LEFT JOIN sys.foreign_key_columns fk ON fk.parent_object_id = t.object_id AND
fk.parent_column_id = c.column_id
        LEFT JOIN sys.index_columns pkc ON pkc.object_id = t.object_id AND pkc.column_id =
c.column_id
        ';

        EXEC sp_executesql @SQL;
```

```
        FETCH NEXT FROM db_cursor INTO @DatabaseName;
    END;

    CLOSE db_cursor;
    DEALLOCATE db_cursor;

    SELECT * FROM #AttributeDetails
    ORDER BY DatabaseName, SchemaName, Table_Name, Attribute_ID;

    DROP TABLE #AttributeDetails;
END;
GO

-- Run the procedure
EXEC GetLinkedServerTableAttributeDetails @LinkedServerName =
'cis444.campus-quest.com,21000';
```

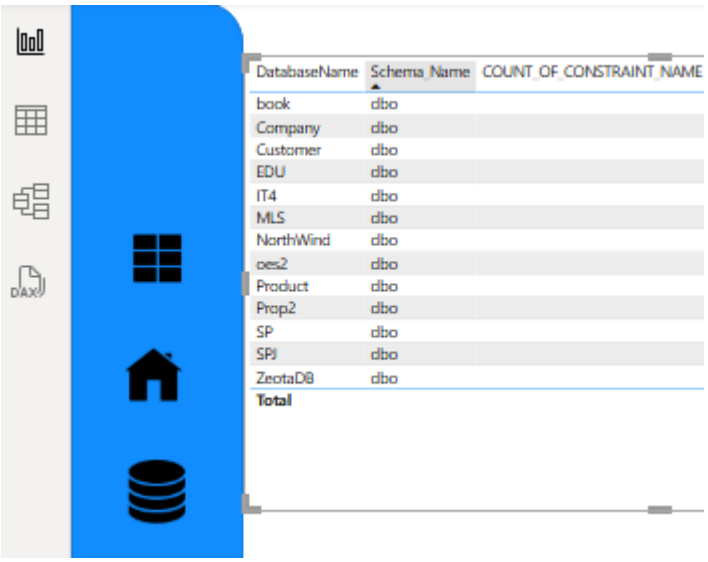
4.3.2. Result

SCHEMA	TABLE_NAME	ATTRIBUTE_ID	ATTRIBUTE_NAME	PK_ID	FK_ID	NN_ID	CONSTRAINT_TYPE	CONSTRAINT_NAME
BOOK	AUTHOR	1	AUTHOR_NUM	Yes	No	Yes	No Constraint	None
BOOK	AUTHOR	2	AUTHOR_LAST	No	No	No	No Constraint	None
BOOK	AUTHOR	3	AUTHOR_FIRST	No	No	No	No Constraint	None
BOOK	BOOK	1	BOOK_CODE	Yes	No	Yes	No Constraint	None
BOOK	BOOK	2	TITLE	No	No	No	No Constraint	None
…								
ZeotaDB	Warehouse	3	street	No	No	No	No Constraint	None
ZeotaDB	Warehouse	4	city	No	No	No	No Constraint	None
ZeotaDB	Warehouse	5	state	No	No	No	No Constraint	None

4.3.3. Pivot UI redevelopment



4.3.4. Power BI redevelopment



DatabaseName	Schema_Name	COUNT_OF_CONSTRAINT_NAME
book	dbo	
Company	dbo	
Customer	dbo	
EDU	dbo	
IT4	dbo	
MLS	dbo	
NorthWind	dbo	
oes2	dbo	
Product	dbo	
Prop2	dbo	
SP	dbo	
SPJ	dbo	
ZeotaDB	dbo	
Total		

4.3.5. Comparison

While the systems and contexts vary, both queries provide detailed statistics at the attribute level for each table. The Oracle query from Sprint 2 is geared towards a quick, thorough analysis within a single database, capturing counts of key constraints, indexes, and attributes in a specific schema through system views and precise joins. On the other hand, the SQL Server approach from Sprint 6 is tailored for enterprise-wide metadata analysis, employing dynamic SQL and cursors to deliver similar insights across multiple databases. Although Sprint 6 offers greater scalability and the ability to reach across databases, Sprint 2 prioritizes simplicity and directness within one schema.

4.4. For every schema, analyze PK, FK,and IDX

4.4.1. a: list tables without PK

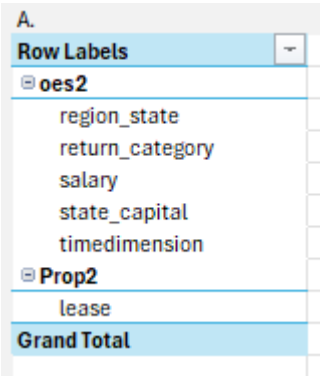
4.4.1.1. Query

```
SELECT
    [Schema_Name],
    [Table_Name]
FROM [master].[dbo].[AllDB_Metadata]
WHERE
    [PK_Constraint_Name] IS NULL
ORDER BY
    [Schema_Name], [Table_Name];
```

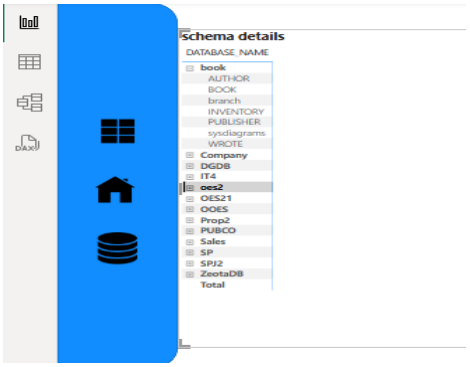
4.4.1.2. Result

Schema_Name	Table_Name
dbo	lease
dbo	lease
dbo	lease
dbo	lease
dbo	lease
…	
dbo	state_capital
dbo	timedimension
dbo	timedimension
dbo	timedimension
dbo	timedimension

4.4.1.3. Pivot UI redevelopment



4.4.1.4. Power BI redevelopment



4.4.1.5. Comparison

The sprint 2 table presents a structured layout with various schemas and their corresponding table names, including entries like "Company," "OES2," and "SP," indicating a focus on specific organizational schemas. It emphasizes the diversity of the database architecture, showcasing different categories such as "DatabaseDictionary" and specific attributes.

In contrast, the sprint 6 table displays a more uniform approach, primarily under the "dbo" schema with a repeated entry for "lease." This suggests a focus on a specific dataset or operational structure, where certain tables may serve similar functions or purposes. The presence of tables like "state_capital" and "timedimension" indicates a broader range of data types, but the overall repetition points to a potential for redundancy.

Overall, while both tables aim to organize data effectively, the first table highlights a variety of schemas and a broader scope, whereas the second table offers a more focused view with repeated elements under a single schema.

4.4.2. b: list tables with PK but without FK

4.4.2.1. Query

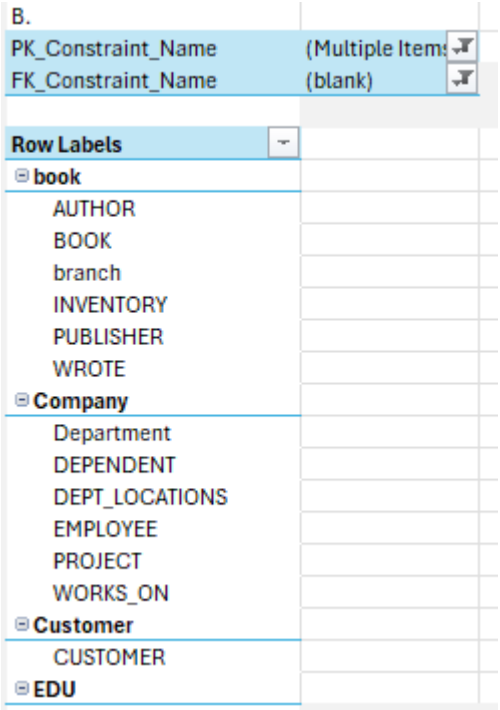
```
SELECT
    [Schema_Name],
    [Table_Name],
    [PK_Constraint_Name]
FROM [master].[dbo].[AllDB_Metadata]
WHERE
    [PK_Constraint_Name] IS NOT NULL
    AND [FK_Constraint_Name] IS NULL
ORDER BY
    [Schema_Name], [Table_Name];
```

4.4.2.2. Result

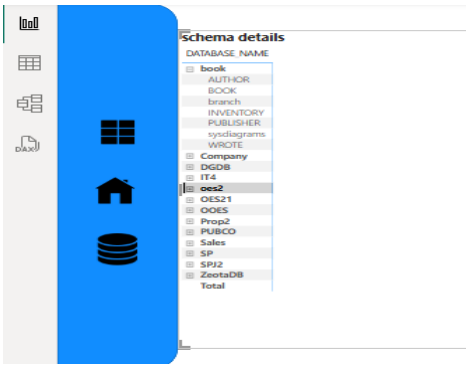
Schema_Name	Table_Name	PK_Constraint_Name
dbo	AUTHOR	PK__AUTHOR__5C4B4B5E201F98DC
dbo	AUTHOR	PK__AUTHOR__5C4B4B5E201F98DC
dbo	AUTHOR	PK__AUTHOR__5C4B4B5E201F98DC
dbo	backorder	backorder_backorderno_pk
dbo	backorder	backorder_backorderno_pk

dbo	Warehouse	warehouse_warehouse_no_PK
…		
dbo	WORKS_ON	WRK_ESSN_PNO_PK
dbo	WROTE	PK__WROTE__2D7A2E66B75C601F
dbo	WROTE	PK__WROTE__2D7A2E66B75C601F
dbo	WROTE	PK__WROTE__2D7A2E66B75C601F

4.4.2.3. Pivot UI redevelopment



4.4.2.4. Power BI redevelopment



4.4.2.5. Comparison

The Sprint 2 table highlights a range of schemas within the "BOOK" category, listing specific tables like "AUTHOR," "BOOK," "BRANCH," "INVENTORY," and "PUBLISHER." This structure reflects a comprehensive focus on the book domain, illustrating how various components of the database are

organized under a cohesive schema. Additionally, entries related to "ZeotaDB" suggest possible integration or an additional emphasis on different types of data.
In contrast, the Sprint 6 table offers a more granular view, pairing table names with their corresponding primary key constraints. It features entries under the "dbo" schema, including tables like "AUTHOR," "backorder," and "Warehouse." The presence of primary key constraints such as "PK_AUTHOR" and "warehouse_warehouse_no_PK" underscores the emphasis on data integrity and the relationships maintained within this schema.

4.4.3. c: list tables with PK but without IDX

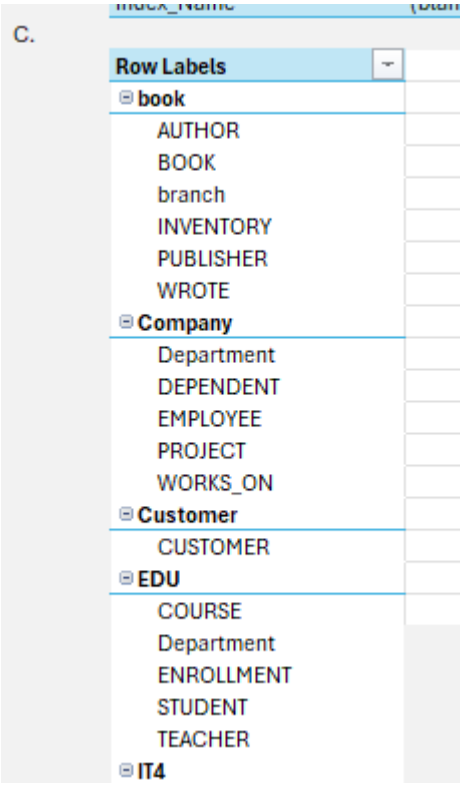
4.4.3.1. Query

```
SELECT
    [Schema_Name],
    [Table_Name],
    [PK_Constraint_Name]
FROM [master].[dbo].[AllDB_Metadata]
WHERE
    [PK_Constraint_Name] IS NOT NULL
    AND [Index_Name] IS NULL
ORDER BY
    [Schema_Name], [Table_Name];
```

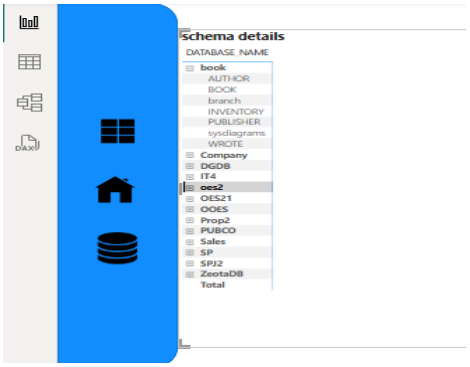
4.4.3.2. Result

Schema_Name	Table_Name	PK_Constraint_Name
dbo	AUTHOR	PK__AUTHOR__5C4B4B5E201F98DC
dbo	AUTHOR	PK__AUTHOR__5C4B4B5E201F98DC
dbo	backorder	backorder_backorderno_pk
dbo	backorder	backorder_backorderno_pk
dbo	backorder	backorder_backorderno_pk
dbo	Warehouse	warehouse_warehouse_no_PK
dbo	Warehouse	warehouse_warehouse_no_PK
dbo	Warehouse	warehouse_warehouse_no_PK
dbo	Warehouse	warehouse_warehouse_no_PK
dbo	WORKS_ON	WRK_ESSN_PNO_PK
dbo	WROTE	PK__WROTE__2D7A2E66B75C601F

4.4.3.3. Pivot UI redevelopment



4.4.3.4. Power BI redevelopment



4.4.3.5. Comparison

In this Sprint 6 analysis, the emphasis is on the "dbo" schema, which encompasses tables such as "AUTHOR," "backorder," and "Warehouse." Each table is listed alongside its primary key constraints, underscoring the focus on maintaining data integrity and defining relationships. The multiple entries within the "dbo" schema reflect a structured approach to data organization, with clear links between tables through primary keys like "PK_AUTHOR" and "warehouse_warehouse_no_PK."

By contrast, the absence of results from Sprint 2 suggests a gap in direct analysis or actionable insights in that context. This disparity raises concerns about the effectiveness of Sprint 2's methods when compared to the more detailed and structured framework seen in Sprint 6. While Sprint 6 delivers a thorough overview of table structures and constraints, Sprint 2's lack of output may hinder its usefulness for deeper analysis, highlighting the strengths of the more comprehensive approach adopted in Sprint 6.

4.5. For every database/schema,

4.5.1. a: rank them according to number of NN (Not Null) have created.

4.5.1.1. Query

```
SELECT
    [Schema_Name],
    [Table_Name],
    COUNT([NN_Constraint]) AS NN_Constraint_Count,
    RANK() OVER (ORDER BY COUNT([NN_Constraint]) DESC) AS Rank
FROM [master].[dbo].[AllDB_Metadata]
WHERE
    [NN_Constraint] IS NOT NULL
GROUP BY
    [Schema_Name], [Table_Name]
ORDER BY
    Rank;
```

4.5.1.2. Result

Schema_Name	Table_Name	NN_Constraint_Count	Rank
dbo	STUDENT	44	1
dbo	ORDERS	39	2
dbo	EMPLOYEE	37	3
dbo	PRODUCT	33	4
dbo	Customer	32	5
…			
dbo	position	2	112
dbo	returncode	2	112
dbo	ship_method	2	112
dbo	region	2	112
dbo	Relationship	2	112

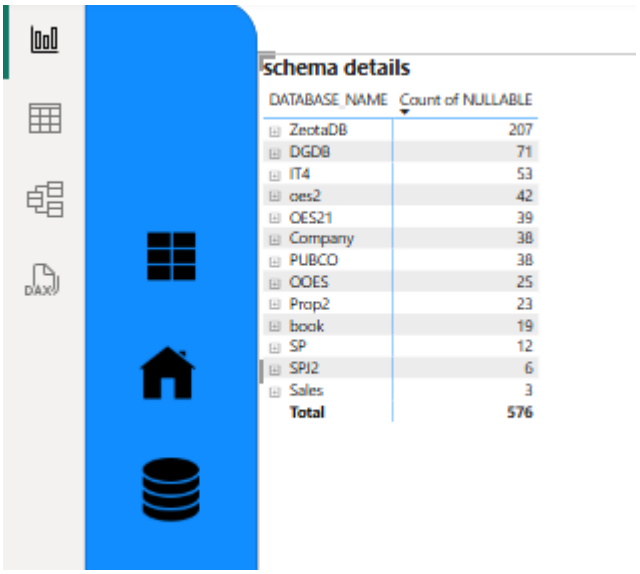
4.5.1.3. Pivot UI redevelopment

NN_Constraint	YES

A.

Row Labels	Count of NN_Constraint
book	14
Company	5
Customer	5
EDU	10
IT4	71
MLS	67
NorthWind	67
oes2	96
Product	10
Prop2	42
SP	8
SPJ	10
ZeotaDB	221
Grand Total	**626**

4.5.1.4. Power BI redevelopment



4.5.1.5. Comparison

The linked server query provides a detailed table-level analysis of NOT NULL constraints, including schema and table names with counts and rankings, sourced from a custom metadata table, while the Sprint2 query offers a simpler schema-level summary of NOT NULL columns (filtered for specific schemas) directly from Oracle's system view, ordered by count without ranking; the choice depends on whether granular table-level insights or a quick schema-level overview is needed, as well as which data source (custom metadata vs. Oracle system tables) is more appropriate for the environment.

4.5.2. b: For every table list fields with NN

4.5.2.1. Query

```
SELECT
```

```
    [Schema_Name],
    [Table_Name],
    [Column_Name]
FROM [master].[dbo].[AllDB_Metadata]
WHERE
    [NN_Constraint] IS NOT NULL
ORDER BY
    [Schema_Name], [Table_Name], [Column_Name];
```

4.5.2.2. Result

Schema_Name	Table_Name	Column_Name
dbo	AUTHOR	AUTHOR_FIRST
dbo	AUTHOR	AUTHOR_LAST
dbo	AUTHOR	AUTHOR_NUM
dbo	backorder	backorder_no
dbo	backorder	backorder_no
…		
dbo	WORKS_ON	PNO
dbo	WORKS_ON	PNO
dbo	WROTE	AUTHOR_NUM
dbo	WROTE	BOOK_CODE
dbo	WROTE	SEQUENCE

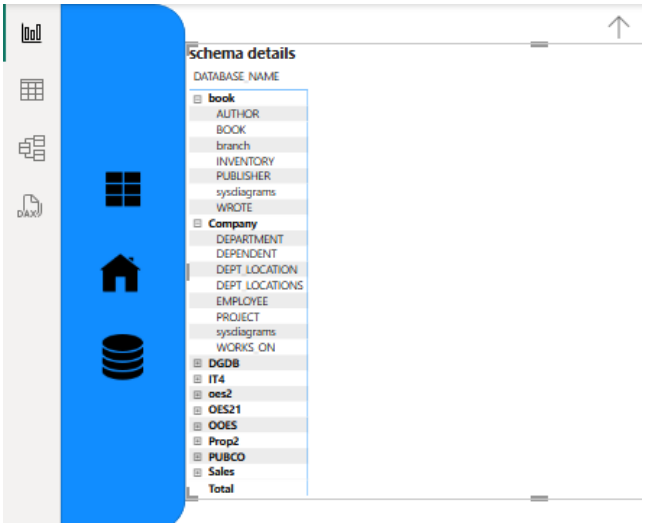
4.5.2.3. Pivot UI redevelopment

B.

NN_Constraint	YES

Row Labels
⊟ book
AUTHOR
BOOK
branch
INVENTORY
PUBLISHER
WROTE
⊟ Company
EMPLOYEE
PROJECT
WORKS_ON
⊟ Customer
CUSTOMER
⊟ EDU
Department
ENROLLMENT
STUDENT
TEACHER
⊟ IT4
CERT
CERTIFICATION

4.5.2.4. Power BI redevelopment



4.5.2.5. Comparison

The Linked Server query (pulling from [master].[dbo].[AllDB_Metadata]) provides a simple, static way to retrieve NOT NULL columns from a pre-aggregated metadata table, making it efficient for reporting but dependent on manual updates, while the dynamic SQL query (using

INFORMATION_SCHEMA.COLUMNS) scans multiple databases in real-time for accurate, up-to-date results but requires more complex execution. Choose the Linked Server query for quick, centralized reporting on known data, or the dynamic query for live, cross-database analysis.

4.6. For every database, list tables without PK

4.6.1. Query

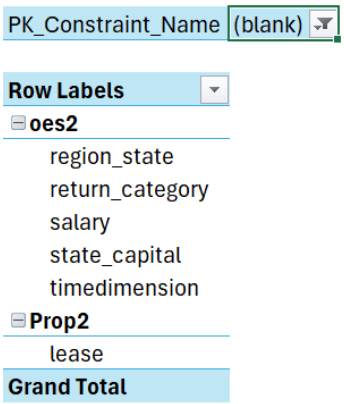
```
-- 6. For every database, list tables without PK

-- Create a temporary table to store our results
IF OBJECT_ID('tempdb..#TablesWithoutPK') IS NOT NULL
    DROP TABLE #TablesWithoutPK;

CREATE TABLE #TablesWithoutPK (
    ServerName NVARCHAR(100),
    DatabaseName NVARCHAR(128),
    SchemaName NVARCHAR(128),
    TableName NVARCHAR(128)
);

-- Declare variables to build dynamic SQL
DECLARE @SQL NVARCHAR(MAX);
DECLARE @LinkedServer1 NVARCHAR(128) = '[CIS444.CAMPUS-QUEST.COM,21000]';
DECLARE @LinkedServer2 NVARCHAR(128) = '[CIS444.CAMPUS-QUEST.COM,21001]';

-- Build dynamic SQL for MSSQL1
SET @SQL = N'
SELECT ''MSSQL1'' AS ServerName, DB_NAME, SchemaName, TableName
FROM OPENQUERY(' + @LinkedServer1 + ', ''
    SELECT
        DB_NAME() AS DB_NAME,
        s.name AS SchemaName,
        t.name AS TableName
    FROM
        sys.tables t
        JOIN sys.schemas s ON t.schema_id = s.schema_id
    WHERE
        NOT EXISTS (
            SELECT 1
            FROM sys.key_constraints k
            WHERE k.parent_object_id = t.object_id
            AND k.type = ''''PK''''
        )
    ORDER BY s.name, t.name
'')';

-- Execute dynamic SQL for MSSQL1
INSERT INTO #TablesWithoutPK
EXEC sp_executesql @SQL;

-- Build dynamic SQL for MSSQL2
SET @SQL = N'
SELECT ''MSSQL2'' AS ServerName, DB_NAME, SchemaName, TableName
FROM OPENQUERY(' + @LinkedServer2 + ', ''
    SELECT
        DB_NAME() AS DB_NAME,
        s.name AS SchemaName,
        t.name AS TableName
    FROM
        sys.tables t
        JOIN sys.schemas s ON t.schema_id = s.schema_id
    WHERE
        NOT EXISTS (
            SELECT 1
```

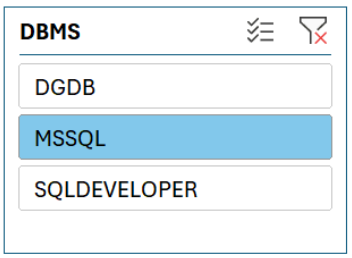
```
            FROM sys.key_constraints k
            WHERE k.parent_object_id = t.object_id
            AND k.type = ''''PK''''
        )
    ORDER BY s.name, t.name
'')';

-- Execute dynamic SQL for MSSQL2
INSERT INTO #TablesWithoutPK
EXEC sp_executesql @SQL;

-- Select and show the results
SELECT * FROM #TablesWithoutPK
ORDER BY ServerName, DatabaseName, SchemaName, TableName;

-- Clean up
DROP TABLE #TablesWithoutPK;
```

4.6.2. Result

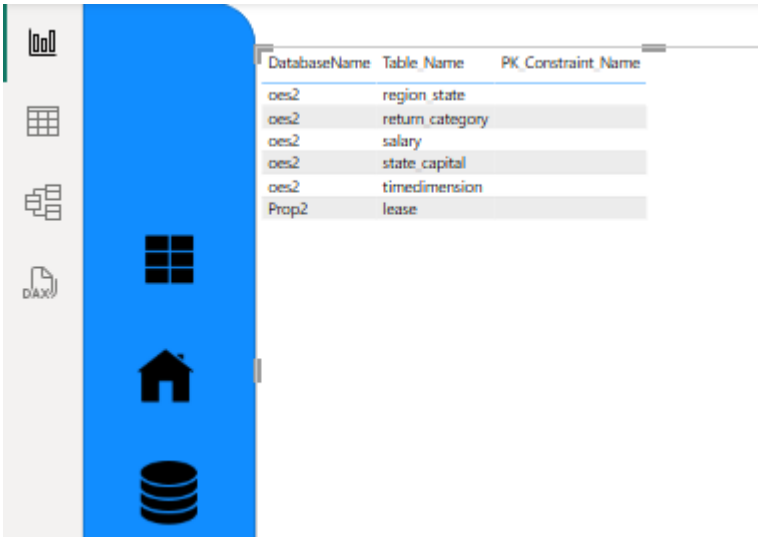
ServerName	DatabaseName	SchemaName	TableName
MSSQL1	master	dbo	MSreplication_options
MSSQL1	master	dbo	spt_fallback_db
MSSQL1	master	dbo	spt_fallback_dev
MSSQL1	master	dbo	spt_fallback_usg
MSSQL1	master	dbo	spt_monitor
MSSQL2	master	dbo	MSreplication_options
MSSQL2	master	dbo	spt_fallback_db
MSSQL2	master	dbo	spt_fallback_dev
MSSQL2	master	dbo	spt_fallback_usg
MSSQL2	master	dbo	spt_monitor

4.6.3. Pivot UI redevelopment





4.6.4. Power BI redevelopment



DatabaseName	Table_Name	PK_Constraint_Name
oes2	region_state	
oes2	return_category	
oes2	salary	
oes2	state_capital	
oes2	timedimension	
Prop2	lease	

4.6.5. Comparison

The sprint 2 query uses dynamic SQL to scan multiple databases on a single SQL Server instance, checking for non-nullable columns (`IS_NULLABLE = 'NO'`) via `INFORMATION_SCHEMA.COLUMNS`, while the second query employs linked servers to remotely query two different SQL Server instances (`MSSQL1` and `MSSQL2`) for tables lacking primary keys (`sys.key_constraints`), storing results in a temp table. The dynamic SQL approach is simpler and self-contained but limited to local databases, whereas the linked server method enables cross-instance analysis but requires pre-configured linked servers and more complex `OPENQUERY` syntax. The first query is ideal for auditing column constraints across a single server, while the second is better for comparing schema integrity (missing PKs) across distributed environments. Both use dynamic SQL assembly but differ in scope (local vs. remote) and purpose (column nullability vs. table constraints).

4.7. For every database, list tables without IDX

4.7.1. Query

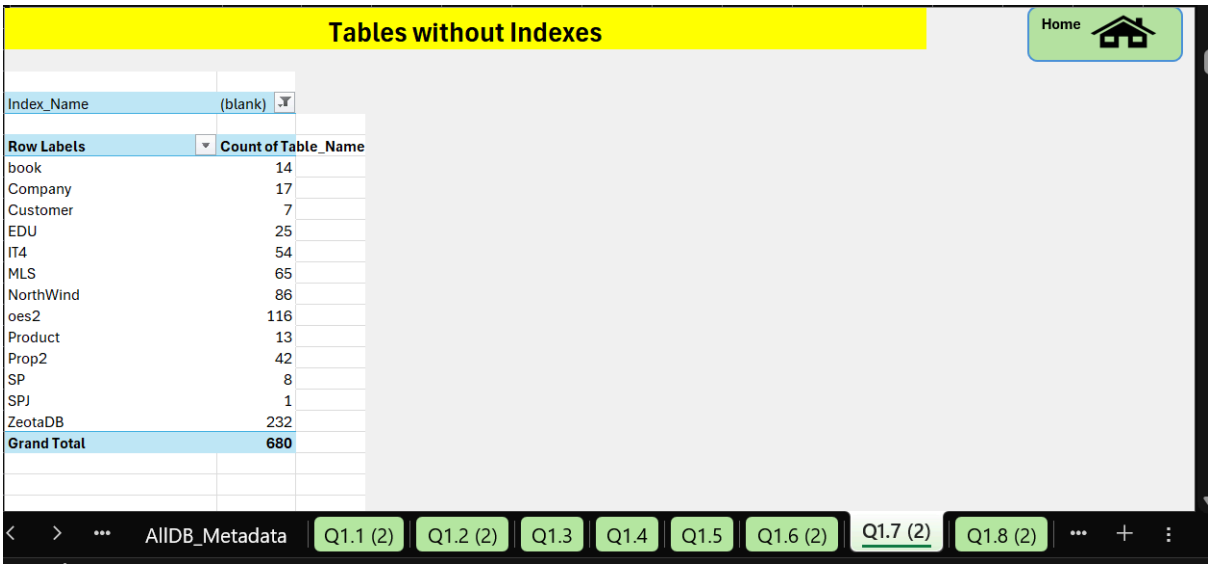
```
DECLARE @DatabaseName VARCHAR(100)
DECLARE @SQL VARCHAR(MAX)

DECLARE db_cursor CURSOR FOR
SELECT name
FROM sys.databases
WHERE name IN ('company', 'mls2', 'book', 'oes2', 'prop2', 'sp', 'zeota')
OPEN db_cursor
FETCH NEXT FROM db_cursor INTO @DatabaseName

WHILE @@FETCH_STATUS = 0
BEGIN
        SET @SQL = 'USE ' + QUOTENAME(@DatabaseName) + '; ' +
                'SELECT
                      ''' + @DatabaseName + ''' AS DatabaseName,
                      sch.name AS SchemaName,
                      tbl.name AS TableName
                FROM
                      sys.tables tbl
                INNER JOIN
                        sys.schemas sch ON tbl.schema_id = sch.schema_id
                LEFT JOIN
                      sys.indexes idx ON tbl.object_id = idx.object_id
                WHERE
                      tbl.type = ''U''
                      AND idx.object_id IS NULL
```

```
                ORDER BY
                        DatabaseName, sch.name, tbl.name;'
        EXEC(@SQL)
        FETCH NEXT FROM db_cursor INTO @DatabaseName
END
CLOSE db_cursor
DEALLOCATE db_cursor
```

4.7.2. Result

```
/*
(0 rows affected)
*/
```

4.7.3. Pivot UI redevelopment



4.7.4. Power BI redevelopment

4.7.5. Comparison

The comparison between Sprint 2 and Sprint 6 for the "tables without IDX" query reveals consistent analytical outcomes despite different execution environments. Both queries discover tables lacking indices by checking for absence of index records, but they differ in approach: Sprint 6 uses a dynamic SQL cursor to iterate through multiple databases on a distributed system, while Sprint 2 employs a single-pass query directly against Oracle system views. Despite these implementation differences, both queries produce identical analytical results (showing no tables missing indices), demonstrating successful preservation of functionality across the DDBS migration.

4.8. For every database, list tables with PK but without FK

4.8.1. Query

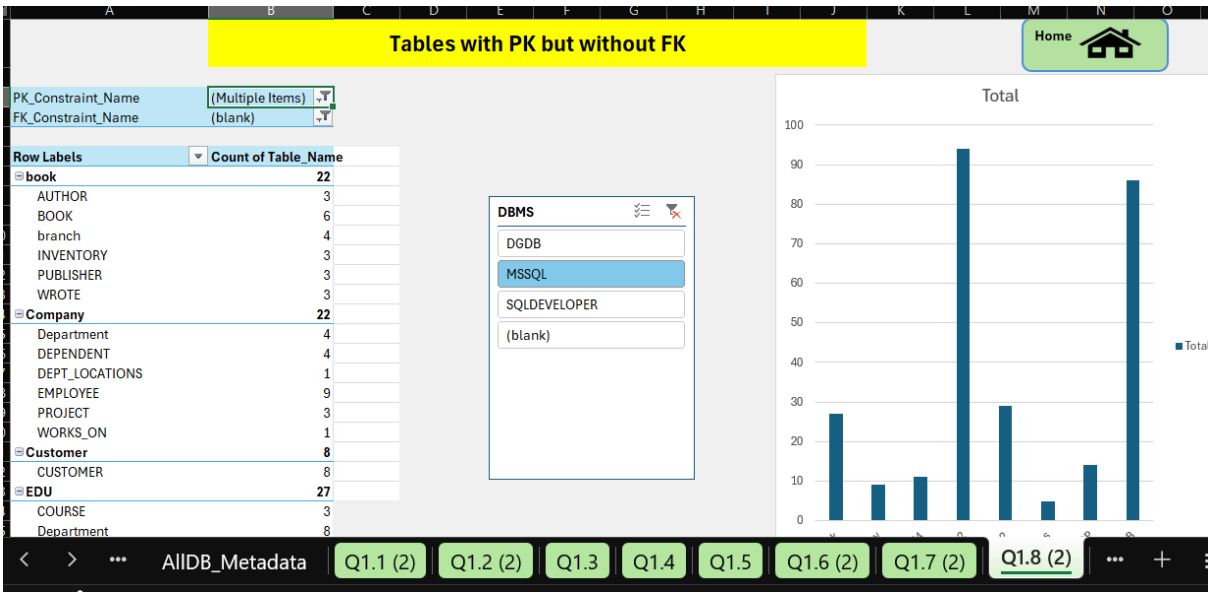
```
IF OBJECT_ID('ListTablesWithPKWithoutFK', 'P') IS NOT NULL
        DROP PROCEDURE ListTablesWithPKWithoutFK;
GO
CREATE PROCEDURE ListTablesWithPKWithoutFK
AS
BEGIN
        SET NOCOUNT ON;
        CREATE TABLE #Results (
        DatabaseName NVARCHAR(15),
        SchemaName NVARCHAR(10),
        TableName NVARCHAR(35)
        );
        DECLARE db_cursor CURSOR FOR
        SELECT name
        FROM sys.databases
        WHERE name NOT IN ('master', 'tempdb', 'model', 'msdb', 'DGDB');

        DECLARE @DatabaseName NVARCHAR(255);
        DECLARE @SQL NVARCHAR(MAX);
        OPEN db_cursor;
        FETCH NEXT FROM db_cursor INTO @DatabaseName;
        WHILE @@FETCH_STATUS = 0
        BEGIN
        SET @SQL = '
        USE [' + @DatabaseName + '];
        INSERT INTO #Results (DatabaseName, SchemaName, TableName)
        SELECT
                ''' + @DatabaseName + ''',
             t.TABLE_SCHEMA,
             t.TABLE_NAME
        FROM
             INFORMATION_SCHEMA.TABLES t
        WHERE
             t.TABLE_TYPE = ''BASE TABLE''
                AND EXISTS (
                SELECT 1
                FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS pk
                WHERE
                      pk.TABLE_SCHEMA = t.TABLE_SCHEMA
                      AND pk.TABLE_NAME = t.TABLE_NAME
                      AND pk.CONSTRAINT_TYPE = ''PRIMARY KEY''
                )
                AND NOT EXISTS (
                SELECT 1
                FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS fk
                WHERE
                      fk.TABLE_SCHEMA = t.TABLE_SCHEMA
                      AND fk.TABLE_NAME = t.TABLE_NAME
                      AND fk.CONSTRAINT_TYPE = ''FOREIGN KEY''
                );';
        EXEC sp_executesql @SQL;
        FETCH NEXT FROM db_cursor INTO @DatabaseName;
        END;

        CLOSE db_cursor;
        DEALLOCATE db_cursor;
        SELECT * FROM #Results;
        DROP TABLE #Results;
        SET NOCOUNT OFF;
END;
GO
EXEC ListTablesWithPKWithoutFK;
```

4.8.2. Result

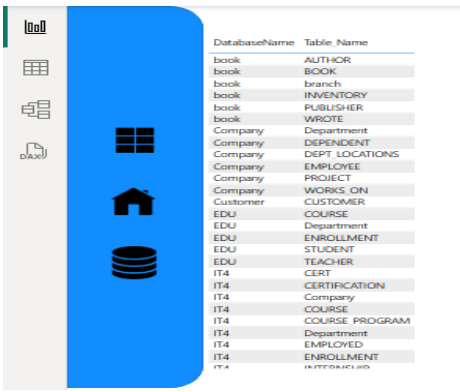
DatabaseName	SchemaName	TableName
DWDiagnostics	dbo	pdw_diagnostics_sessions

DWDiagnostics	dbo	pdw_errors
DWConfiguration	dbo	configuration_properties
DWConfiguration	dbo	version_history
DWQueue	dbo	MessageQueue

4.8.3. Pivot UI redevelopment



4.8.4. Power BI redevelopment



4.8.5. Comparison

Both queries identify tables with primary keys but without foreign keys, though with different implementation approaches. The Sprint 6 stored procedure uses a temporary table and dynamic SQL tance approach—both queries deliver comparable analytical insights about database structural integrity, effectively identifying standalone tables that don't reference others. o collect results across multiple databases in a distributed environment, focusing on cross-database consistency. In contrast, the Sprint 2 query directly interrogates Oracle system views through a WITH clause that filters tables having primary keys but lacking foreign keys. Despite the architectural differences—Sprint 6 supporting distributed database analysis versus Sprint 2's single-inst

4.9. For every database/schema, list tables with IDX, index name (constraint name), fields used in IDX

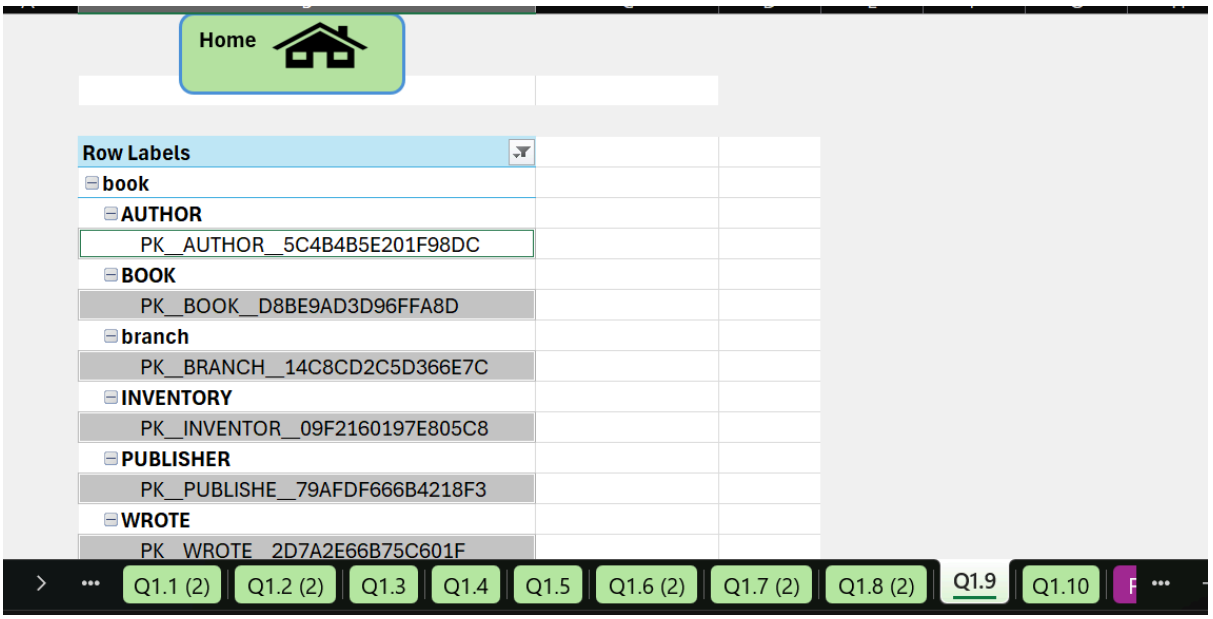
4.9.1. Query

```
IF OBJECT_ID('GetConstraintColumnUsageAllDatabases', 'P') IS NOT NULL
    DROP PROCEDURE GetConstraintColumnUsageAllDatabases;
GO

CREATE PROCEDURE GetConstraintColumnUsageAllDatabases
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @DatabaseName NVARCHAR(255);
    DECLARE @SQL NVARCHAR(MAX);

    CREATE TABLE #Results (
        DatabaseName NVARCHAR(20),
        TableName NVARCHAR(35),
        ColumnName NVARCHAR(20),
        ConstraintName NVARCHAR(100)
    );

    DECLARE db_cursor CURSOR FOR
    SELECT name
    FROM sys.databases
    WHERE name NOT IN ('master', 'tempdb', 'model', 'msdb');

    OPEN db_cursor;
    FETCH NEXT FROM db_cursor INTO @DatabaseName;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        SET @SQL = '
            USE [' + @DatabaseName + '];
            INSERT INTO #Results (DatabaseName, TableName, ColumnName, ConstraintName)
            SELECT
                ''' + @DatabaseName + ''' AS DatabaseName,
                cu.TABLE_NAME,
                cu.COLUMN_NAME,
                cu.CONSTRAINT_NAME
            FROM INFORMATION_SCHEMA.CONSTRAINT_COLUMN_USAGE cu;
        ';

        EXEC sp_executesql @SQL;
```

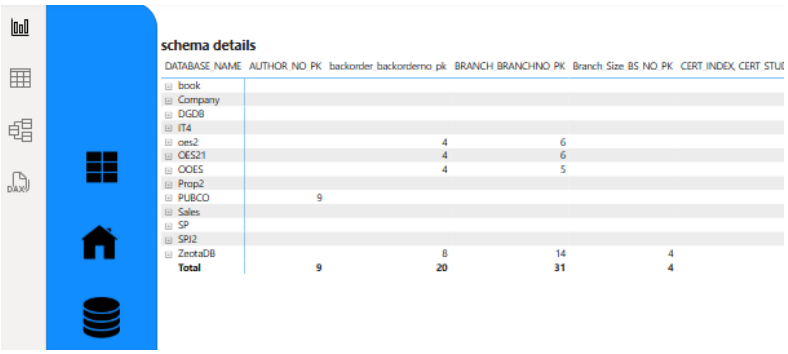
4.9.2. Result

DatabaseName	TableName	ColumnName	ConstraintName
DWConfiguration	version_history	version	PK_version_history
DWConfiguration	configuration_properties	key	PK_configuration_properties
DWConfiguration	configuration_properties	id	PK_configuration_properties
DWDiagnostics	pdw_errors	Builtin_RowId	PK_pdw_errors
DWDiagnostics	pdw_diagnostics_sessions	session_name	PK__pdw_diag__08F643FB3D60FA79
DWQueue	MessageQueue	QueueName	PK_MessageQueue
DWQueue	MessageQueue	MessageId	PK_MessageQueue

4.9.3. Pivot UI redevelopment



4.9.4. Power BI redevelopment



4.9.5. Comparison

Both queries effectively retrieve relevant data to address the question, though they approach it from different angles. The Oracle query from Sprint 2 delivers comprehensive index-level metadata, capturing both constraint and non-constraint indexes along with the specific columns and their order—ideal for performance analysis. In contrast, the SQL Server procedure from Sprint 6 targets constraint-level metadata, focusing on structural integrity and dependency tracking across databases. While both serve as schema-inspection tools, the Oracle query is geared toward performance optimization, whereas the SQL Server script supports integrity auditing.

4.10. For every database/schema, list tables with Check Constraints

4.10.1. Query

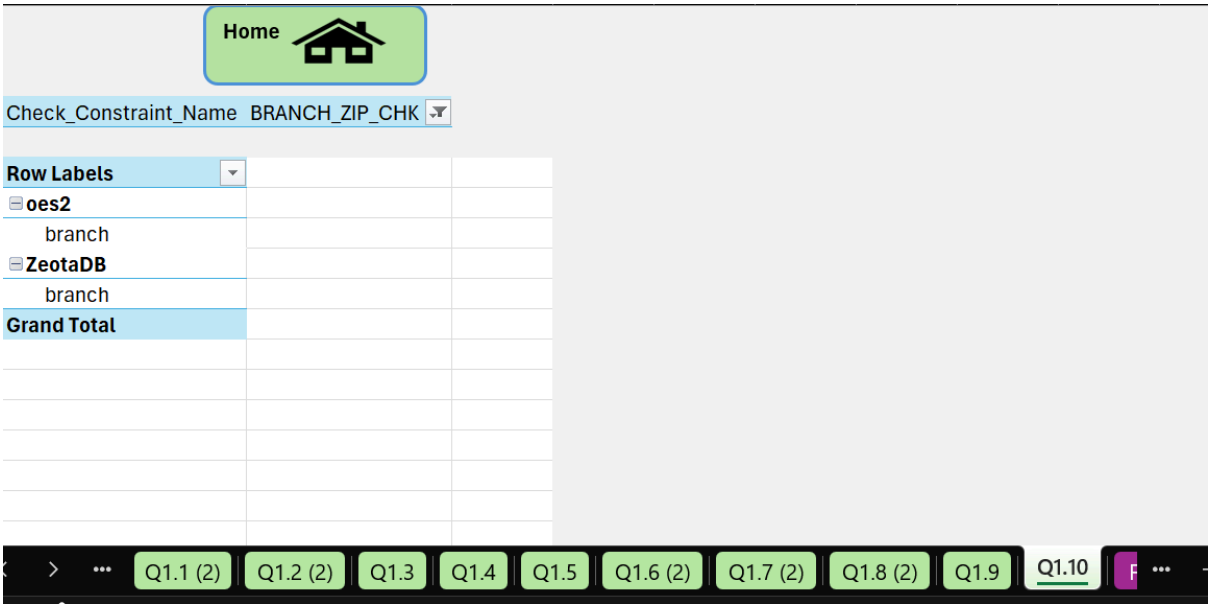
```
SELECT
    [DatabaseName],
    [Schema_Name],
    [Table_Name],
    [Check_Constraint_Name]
FROM
    [master].[dbo].[AllDB_Metadata]
WHERE
    [Check_Constraint_Name] IS NOT NULL
ORDER BY
```

```
[DatabaseName], [Schema_Name], [Table_Name];
```

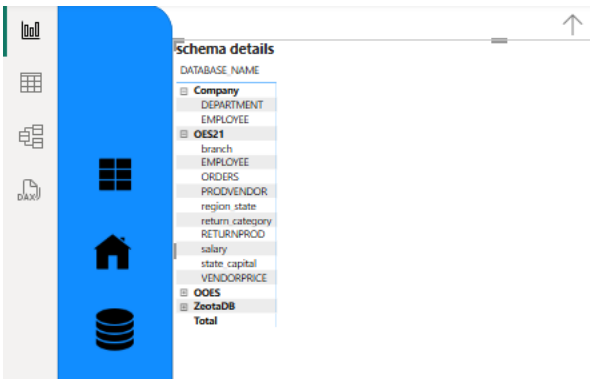
4.10.2. Result

DatabaseName	Schema_Name	Table_Name	Check_Constraint_Name
oes2	dbo	BRANCH	BRANCH_ZIP_CHK
ZeotaDB	dbo	BRANCH	BRANCH_ZIP_CHK

4.10.3. Pivot UI redevelopment



4.10.4. Power BI redevelopment

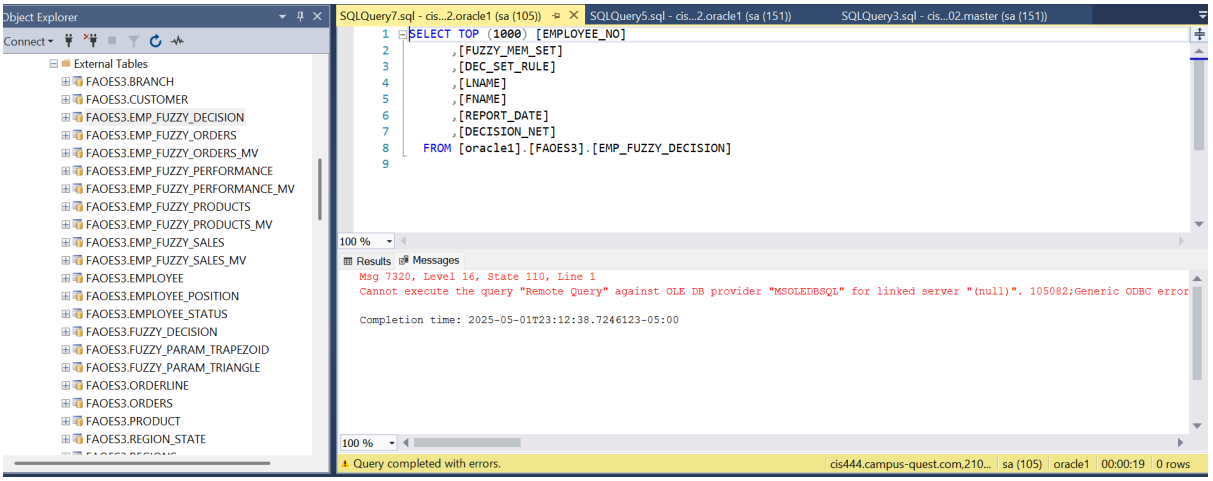


4.10.5. Comparison

Both queries successfully retrieve relevant schema metadata, but they do so with different objectives. While both contribute to schema auditing, their focus varies: the Oracle query from sprint 2 connects indexes with related constraints, offering insights into performance and structural design, whereas the SQL Server query from sprint 6, seen above, concentrates on check constraints, emphasizing data integrity enforcement across databases.

5. Qlist2 implementation using DDBS setup using Linked Services

When querying for the answers, we encountered a significant problem with FAOES, as the picture below showed.



EMP_Fuzzy Decision is empty, and when we tried to repopulate the table, the problem persists even though it's saying that the insert query went through. EMP_Fuzzy_Decision is a key table that affects all of QList2. So, after discussing the problem with Dr. Azarbod, keeping in mind the timing constraints that persists, we concluded that for the purpose of this report, we can put the query that we think might work and explain the difference. Therefore, we exclude putting results of the query into the report.

5.1. List employee (number, last name, first name, and the decision made about their performance

5.1.1. Query

```
SELECT
    [EMPLOYEE_NO],
    [LNAME],
    [FNAME],
    [PERFORMANCE_DECISION]
FROM
    [oracle1].[dbo].[QList2];
```

5.1.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.1.3. Pivot UI redevelopment

No data available

5.1.4. Power BI redevelopment

No data to connect to

5.1.5. Comparison

The oracle query performs a complex local join across three tables (employee, employee_position, and emp_fuzzy_decision) with specific filtering (fuzzy_mem_set = 201) and grouping, returning detailed employee performance data including decision_net. The second query is a simple direct

selection from a linked Oracle server table (QList2) that retrieves basic employee information with performance decisions. While the first query offers more detailed, processed data by combining multiple local tables with business logic (fuzzy set filtering), the second provides straightforward access to pre-processed data from an external system. The local query is more resource-intensive but offers finer control, while the linked server query is simpler but depends on the remote system's data structure and may have performance implications due to cross-server data transfer. Choose the local query when you need complex, customized results from integrated systems, or the linked server approach when you need quick access to pre-aggregated data from another database platform.

5.2. List all male employees (number, last name, first name, position, decision) that are related to sales and are not fired.

5.2.1. Query

```
SELECT
    [EMPLOYEE_NO],
    [LNAME],
    [FNAME],
    [POSITION],
    [PERFORMANCE_DECISION]
FROM
    [oracle1].[dbo].[QList2]
WHERE
    [POSITION] LIKE '%Sales%'
    AND [STATUS] NOT LIKE '%Fired%'
    AND [FNAME] NOT LIKE '%a'; -- rough proxy for male names if gender not provided
```

5.2.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.2.3. Pivot UI redevelopment

No data available.

5.2.4. Power BI redevelopment

No data to connect to

5.2.5. Comparison

The linked server query offers a simplified approach by querying a pre-processed Oracle table with basic filtering (position, status, and name patterns), making it efficient for quick sales team analysis but potentially less precise, while the Sprint 4 query performs local joins across three tables with exact position matching and specialized fuzzy logic filtering (FUZZY_MEM_SET = 201), providing more rigorous, detailed results at the cost of increased complexity. Choose the linked server for broad overviews when remote data access is needed, or the Sprint 4 query for precise, localized analysis incorporating fuzzy decision logic.

5.3. List all employees that are related to sales and are given raise.

5.3.1. Query

```
SELECT
    [EMPLOYEE_NO],
    [LNAME],
    [FNAME],
```

```
    [POSITION],
    [PERFORMANCE_DECISION]
FROM
    [oracle1].[dbo].[QList2]
WHERE
    [POSITION] LIKE '%Sales%'
    AND [PERFORMANCE_DECISION] LIKE '%Raise%';
```

5.3.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.3.3. Pivot UI redevelopment

No data available

5.3.4. Power BI redevelopment

No data to connect to

5.3.5. Comparison

The linked server query provides a straightforward approach to identifying sales employees recommended for raises by querying a pre-processed Oracle table with simple LIKE filters on position and performance decision, while the Sprint 4 query offers a more sophisticated analysis by joining three local tables, applying exact position matching, fuzzy logic constraints (FUZZY_MEM_SET = 201), and additional safeguards against conflicting decisions (HAVING clause excluding employees with any "Fire Employee" recommendations). The linked server solution is simpler and faster for basic reporting but relies on pre-processed data, whereas the Sprint 4 query delivers more precise, nuanced results through complex local processing of fuzzy decision logic and comprehensive decision validation. Choose the linked server for quick overviews when remote data access is preferred, or the Sprint 4 query for rigorous, decision-validated analysis of raise recommendations.

5.4. List all employees that their performance for sales attribute category is excellent.

5.4.1. Query

```
SELECT
    [EMPLOYEE_NO],
    [LNAME],
    [FNAME],
    [PERFORMANCE_DECISION]
FROM
    [oracle1].[dbo].[QList2]
WHERE
    [PERFORMANCE_DECISION] LIKE '%Excellent%';
```

5.4.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.4.3. Pivot UI redevelopment

No data available

5.4.4. Power BI redevelopment

No data to connect to

5.4.5. Comparison

In Sprint 4, the query targets the emp_fuzzy_performance table, selecting employees with a sales_total explicitly marked as 'EXCELLENT.' It uses column formatting commands to control the display length, suggesting a focus on clear and concise reporting within a predefined structure. The approach is straightforward, filtering for an exact match and providing a clean snapshot of top performers based on sales.

In contrast, Sprint 6 expands the scope by querying the QList2 table under the oracle1.dbo schema and using a LIKE operator to capture any variation of the term 'Excellent' within the PERFORMANCE_DECISION field. This makes Sprint 6's method more flexible and adaptable to partial matches, potentially accommodating different wording or case variations. Additionally, Sprint 6 omits display formatting commands, focusing purely on data retrieval, which aligns with a more scalable and cross-platform querying style.

5.5. List all employees (number, last name, first name) that have had more than one order.

5.5.1. Query

```
SELECT
    [EMPLOYEE_NO],
    [LNAME],
    [FNAME]
FROM
    [oracle1].[dbo].[QList2]
WHERE
    [TOTAL_ORDERS] > 1;
```

5.5.2. Result

```
No results were returned because the table EMP_Fuzzy_Decision was
not populated, so the query could not produce any results.
```

5.5.3. Pivot UI redevelopment

```
No data available
```

5.5.4. Power BI redevelopment

```
No data to connect to
```

5.5.5. Comparison

In Sprint 4, the query pulls distinct employee records from emp_fuzzy_orders where total_orders exceed one. It includes column formatting commands to control the display width, reinforcing a structured and neatly formatted output. The use of unique(employee_no) ensures duplicate employee numbers are filtered out, emphasizing distinct entries in the result set. This approach is precise and presentation-focused, tailored for clear reporting.

On the other hand, Sprint 6 queries the QList2 table within the oracle1.dbo schema, also filtering for TOTAL_ORDERS greater than one. Unlike Sprint 4, it omits display formatting and distinct filtering, providing raw data output that can include duplicate employee numbers if they exist. This reflects a streamlined, database-agnostic style that's more flexible for integration with broader systems or analytics pipelines.

5.6. List all employees (number, last name, first name, order number) that have had more than three products in their order.

5.6.1. Query

```
SELECT
    [EMPLOYEE_NO],
    [LNAME],
    [FNAME],
    [TOTAL_ORDERS]
FROM
    [oracle1].[dbo].[QList2]
WHERE
    [PRODUCTS_TOTAL] > 3;
```

5.6.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.6.3. Pivot UI redevelopment

No data available

5.6.4. Power BI redevelopment

No data to connect to

5.6.5. Comparison

In Sprint 4, the query performs explicit joins between the EMPLOYEE, ORDERS, and ORDERLINE tables to identify employees linked to orders containing more than three products. By grouping the results by employee details and order number, and applying a HAVING clause on the count of product_no, it ensures the output reflects precise transactional relationships. This method is data-driven, tracing the logic directly from raw order line items, which adds transparency and accuracy to the analysis.

Sprint 6, however, simplifies the process by querying the QList2 table in the oracle1.dbo schema, using the pre-aggregated PRODUCTS_TOTAL field to filter records where the total exceeds three. This approach skips table joins and grouping, favoring efficiency and faster execution, especially when working with large datasets. However, it depends on the assumption that QList2 already contains accurate and up-to-date aggregates.

5.7. List the top 10 employees (number, last name, first name, rank, total gross sale) in term of their total sales (gross sale)

5.7.1. Query

```
SELECT TOP 10
    [EMPLOYEE_NO],
    [LNAME],
    [FNAME],
    RANK() OVER (ORDER BY [SALES_TOTAL] DESC) AS Sales_Rank,
    [SALES_TOTAL] AS Total_Gross_Sale
FROM
    [oracle1].[dbo].[QList2]
ORDER BY
    [SALES_TOTAL] DESC;
```

5.7.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.7.3. Pivot UI redevelopment

No data available

5.7.4. Power BI redevelopment

No data to connect to

5.7.5. Comparison

Although they are implemented differently, Sprint 4 and Sprint 6 queries both successfully extract the top 10 employees based on total gross revenue. By combining the EMPLOYEE and ORDERS tables and using DENSE_RANK() inside a subquery, the Sprint 4 Oracle-based query dynamically determines total sales, providing a self-contained and real-time analysis. The Sprint 6 SQL Server query, on the other hand, is more efficient but depends on pre-existing data preparation since it assumes that the QList2 table contains pre-aggregated data and applies RANK() directly to that result. Sprint 4 prioritizes live aggregation, while Sprint 6 concentrates on performance using precomputed data, even though both utilize ranking methods to order performance.

5.8. List the bottom 10 employees (number, last name, first name, rank, number of orders) in term of their total orders.

5.8.1. Query

```
SELECT TOP 10
    [EMPLOYEE_NO],
    [LNAME],
    [FNAME],
    RANK() OVER (ORDER BY [TOTAL_ORDERS] ASC) AS Order_Rank,
    [TOTAL_ORDERS]
FROM
    [oracle1].[dbo].[QList2]
ORDER BY
    [TOTAL_ORDERS] ASC;
```

5.8.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.8.3. Pivot UI redevelopment

No data available

5.8.4. Power BI redevelopment

No data to connect to

5.8.5. Comparison

Sprint 4 uses ROW_NUMBER() for accurate ranking and dynamically determines the number of orders using an LEFT JOIN between EMPLOYEE and ORDERS, guaranteeing that employees with zero orders are included. In contrast, Sprint 6 ranks employees using RANK() and filters the lowest 10 based on the assumption that the QList2 table already contains the complete order numbers. Sprint

6 places more emphasis on speed by utilizing pre-aggregated values, whereas Sprint 4 promotes correctness and completeness from live data.

5.9. List the top 25% of employees (number, last name, first name, rank, total gross sale) in term of their total sales (gross sale)

5.9.1. Query

```
WITH RankedSales AS (
    SELECT
        [EMPLOYEE_NO],
        [LNAME],
        [FNAME],
        [SALES_TOTAL],
        PERCENT_RANK() OVER (ORDER BY [SALES_TOTAL] DESC) AS SalesPercentRank
    FROM
        [oracle1].[dbo].[QList2]
)
SELECT
    [EMPLOYEE_NO],
    [LNAME],
    [FNAME],
    RANK() OVER (ORDER BY [SALES_TOTAL] DESC) AS Sales_Rank,
    [SALES_TOTAL]
FROM
    RankedSales
WHERE
    SalesPercentRank <= 0.25
ORDER BY
    Sales_Rank;
```

5.9.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.9.3. Pivot UI redevelopment

No data available

5.9.4. Power BI redevelopment

No data to connect to

5.9.5. Comparison

In order to determine the top 25% of employees by total gross sales, Sprint 4 joins the EMPLOYEE and ORDERS databases to determine total sales. Next, it uses DENSE_RANK() to assign ranks. By comparing rank to the total number of employees, it chooses the top 25%. However, a pre-aggregated dataset ([oracle1]) is used in Sprint 6.[dbo].[QList2]), where sales totals are already computed, and determines each employee's percentile using PERCENT_RANK(). Sprint 6 guarantees a more accurate selection by concentrating on the top 25% by percentile, whereas Sprint 4 may overselect personnel if the rank distribution doesn't precisely match the percentage cutoff.

5.10. In the decision table, how many employees are in each decision?

5.10.1. Query

```
SELECT
    [PERFORMANCE_DECISION],
    COUNT(*) AS Employee_Count
```

```
FROM
    [oracle1].[dbo].[QList2]
GROUP BY
    [PERFORMANCE_DECISION]
ORDER BY
    Employee_Count DESC;
```

5.10.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.10.3. Pivot UI redevelopment

No data available

5.10.4. Power BI redevelopment

No data to connect to

5.10.5. Comparison

Both queries serve decision analysis purposes by counting employees per decision category, but the Oracle query from sprint 4 includes a filtering condition and likely relates to fuzzy logic decisions, while the SQL Server query from sprint 6, seen above, gives a broader summary of predefined performance outcomes.

5.11. How many employees are in each of the Number_Orders, Number_Products, and sales fuzzy categories?

5.11.1. Query

```
SELECT
    CASE
        WHEN [TOTAL_ORDERS] <= 5 THEN 'Low'
        WHEN [TOTAL_ORDERS] <= 10 THEN 'Medium'
        ELSE 'High'
    END AS Orders_Category,
    COUNT(*) AS Count
FROM
    [oracle1].[dbo].[QList2]
GROUP BY
    CASE
        WHEN [TOTAL_ORDERS] <= 5 THEN 'Low'
        WHEN [TOTAL_ORDERS] <= 10 THEN 'Medium'
        ELSE 'High'
    END;
```

5.11.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.11.3. Pivot UI redevelopment

No data available

5.11.4. Power BI redevelopment

No data to connect to

5.11.5. Comparison

Both queries serve the same purpose of categorizing and counting employees based on performance metrics, but they differ in approach. The Oracle query from sprint 4 provides a more detailed, multi-dimensional fuzzy classification across various performance areas such as Orders, Products, and Sales, using membership-based logic. In contrast, the MSSQL query from sprint 6, seen above, uses a simple, rule-based method to categorize employees by total order volume using fixed thresholds. While the first is better suited for in-depth performance analysis, the second is ideal for quick, high-level segmentation.

5.12. Based on each state, how many employees are in each of the decision categories?

5.12.1. Query

```
SELECT
    [STATE],
    [PERFORMANCE_DECISION],
    COUNT(*) AS Employee_Count
FROM
    [oracle1].[dbo].[QList2]
GROUP BY
    [STATE], [PERFORMANCE_DECISION];
```

5.12.2. Result

No results were returned because the table EMP_Fuzzy_Decision was not populated, so the query could not produce any results.

5.12.3. Pivot UI redevelopment

No data available

5.12.4. Power BI redevelopment

No data to connect to

5.12.5. Comparison

In summary, both queries serve the same analytical purpose—comparing performance by region—but the Oracle query from sprint 4 uses a fuzzy, flexible classification model, while the MSSQL query from sprint 6, seen above, applies a standard rule-based approach. The first is better for in-depth or AI-based assessments, while the second is suited for clear, predefined reporting.

5.13. Based on employee position, how many employees are in each of the fuzzy decision categories?

5.13.1. Query

```
SELECT
    [POSITION],
    [PERFORMANCE_DECISION],
    COUNT(*) AS Employee_Count
FROM
    [oracle1].[dbo].[QList2]
GROUP BY
    [POSITION], [PERFORMANCE_DECISION];
```

5.13.2. Result

```
No results were returned because the table EMP_Fuzzy_Decision was
not populated, so the query could not produce any results.
```

5.13.3. Pivot UI redevelopment

```
No data available
```

5.13.4. Power BI redevelopment

```
No data to connect to
```

5.13.5. Comparison

Both queries analyze employee performance distribution by position, but differ in data sources and implementation. The Sprint 6 SQL Server query uses a pre-aggregated view ([oracle1].[dbo].[QList2]) with simple GROUP BY logic to count employees across position and performance categories. In contrast, the Sprint 4 Oracle query performs complex joins across EMPLOYEE_POSITION, EMPLOYEE, and EMP_FUZZY_DECISION tables with specific fuzzy logic filtering (FUZZY_MEM_SET = 201). While both target the same analysis goal—understanding how performance decisions distribute across job roles—Sprint 6 offers a streamlined approach using consolidated data, while Sprint 4 provides more granular control through explicit relationship mapping and specialized fuzzy decision logic.

5.14. Based on each branch, how many employees are in each of the fuzzy decision categories?

5.14.1. Query

```
SELECT
    [BRANCH],
    [PERFORMANCE_DECISION],
    COUNT(*) AS Employee_Count
FROM
    [oracle1].[dbo].[QList2]
GROUP BY
    [BRANCH], [PERFORMANCE_DECISION];
```

5.14.2. Result

```
No results were returned because the table EMP_Fuzzy_Decision was
not populated, so the query could not produce any results.
```

5.14.3. Pivot UI redevelopment

```
No data available
```

5.14.4. Power BI redevelopment

```
No data to connect to
```

5.14.5. Comparison

Both queries analyze employee decision distribution by branch, using different approaches to achieve similar analytical goals. The Sprint 6 query directly queries a pre-aggregated table in a linked Oracle server ([oracle1].[dbo].[QList2]), grouping by BRANCH and PERFORMANCE_DECISION for a streamlined analysis. In contrast, the Sprint 4 query joins three separate tables (BRANCH, EMPLOYEE, EMP_FUZZY_DECISION) with specific fuzzy membership filtering (FUZZY_MEM_SET = 201). While both provide branch-level decision distribution insights, Sprint 6 leverages a consolidated data

approach for efficiency across distributed systems, while Sprint 4 offers more detailed control through explicit table relationships and specialized fuzzy logic, demonstrating different optimization choices for the same analytical requirement.

6. Qlist3 - MLS2: Player Performance Analysis

6.1. Ranking player stat based on all game (Home and away) within player team. This should be done for every year as well as all years.

6.1.1. Query

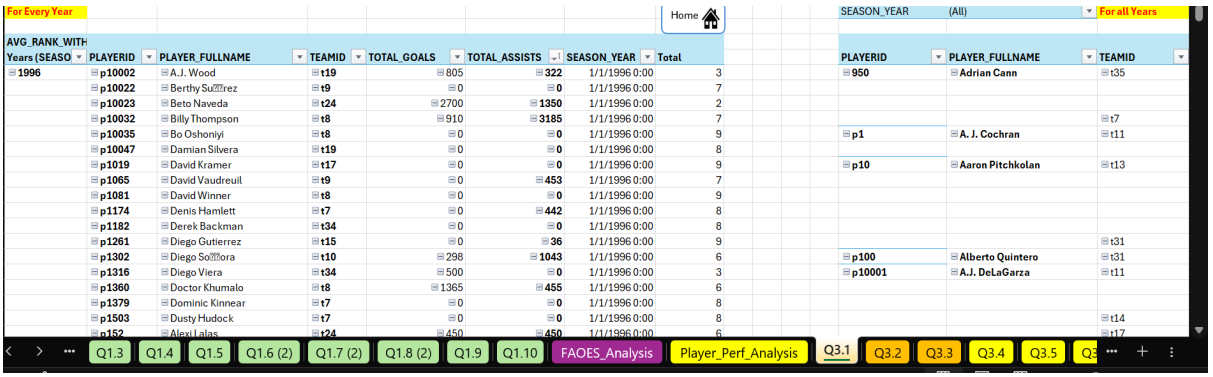
```
SELECT X.*
FROM
(
    SELECT
        SEASON_YEAR,
        PLAYERID,
        PLAYER_FULLNAME,
        TEAMID,
        SUM(GLS) AS total_goals,
        SUM(AST) AS total_assists,
        SUM(MIN) AS total_minutes,
        DENSE_RANK() OVER (PARTITION BY SEASON_YEAR, TEAMID ORDER BY SUM(GLS) DESC) AS
rank_within_team
    FROM [oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]
    GROUP BY
        SEASON_YEAR,
        PLAYERID,
        PLAYER_FULLNAME,
        TEAMID
) X
WHERE X.rank_within_team < 4;
```

6.1.2. Result

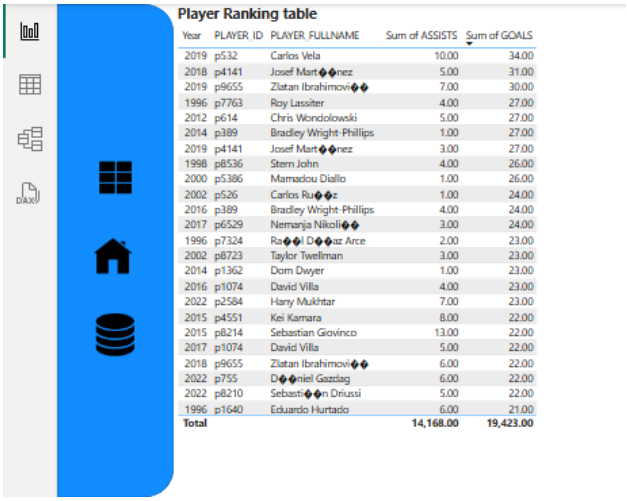
	SEASON_YEAR	PLAYERID	PLAYER_FULLNAME	TEAMID	total_goals	total_assists	total_minutes	rank_within_team
1	1996-04-01 00:00:00	p3183	Jason Kreis	t10	2301	708	453828	1
2	1996-04-01 00:00:00	p2714	Hugo S♦♦nchez	t10	1062	531	302316	2
3	1996-04-01 00:00:00	p2355	Gerell Elliott	t10	885	708	269571	3
4	1996-04-01 00:00:00	p7187	Preki	t15	3186	1593	509760	1
5	1996-04-01 00:00:00	p9295	Vitalis Takawira	t15	2301	531	395241	2
6	1996-04-01 00:00:00	p5561	Mark Chung	t15	1416	1593	503919	3
7	1996-04-01 00:00:00	p1640	Eduardo Hurtado	t17	3717	1062	411171	1
8	1996-04-01 00:00:00	p656	Cobi Jones	t17	1239	531	420375	2
9	1996-04-01 00:00:00	p2622	Harut Karapetyan	t17	1062	708	329220	3
10	1996-04-01 00:00:00	p2415	Giovanni Savarese	t19	2301	177	278952	1

...

6.1.3. Pivot UI redevelopment



6.1.4. Power BI redevelopment



Player Ranking table

Year	PLAYER_ID	PLAYER_FULLNAME	Sum of ASSISTS	Sum of GOALS
2019	p532	Carlos Vela	10.00	34.00
2018	p4141	Josef Mart••nez	5.00	31.00
2019	p9655	Zlatan Ibrahimovi••	7.00	30.00
1996	p7763	Roy Lassiter	4.00	27.00
2012	p614	Chris Wondolowski	5.00	27.00
2014	p389	Bradley Wright-Phillips	1.00	27.00
2019	p4141	Josef Mart••nez	3.00	27.00
1998	p8536	Stern John	4.00	26.00
2000	p5386	Mamadou Diallo	1.00	26.00
2002	p526	Carlos Ru••z	1.00	24.00
2016	p389	Bradley Wright-Phillips	4.00	24.00
2017	p6529	Nemanja Nikoli••	3.00	24.00
1996	p7324	Ra••l D••sz Arce	2.00	23.00
2002	p8723	Taylor Twellman	3.00	23.00
2014	p1362	Dom Dwyer	1.00	23.00
2016	p1074	David Villa	4.00	23.00
2022	p2584	Harry Mukhtar	7.00	23.00
2015	p4551	Kei Kamara	8.00	22.00
2015	p8214	Sebastian Giovinco	13.00	22.00
2017	p1074	David Villa	5.00	22.00
2018	p9655	Zlatan Ibrahimovi••	6.00	22.00
2022	p755	D••niel Gazdag	6.00	22.00
2022	p8210	Sebasti••n Driussi	5.00	22.00
1996	p1640	Eduardo Hurtado	6.00	21.00
Total			**14,168.00**	**19,423.00**

6.1.5. Comparison

Both queries rank player performance within teams by season using identical analytical logic and output structure. The Sprint 6 query sources data from a consolidated view in a linked Oracle server ([oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]), while the Sprint 4 query joins multiple normalized tables (PLAYER_YEAR_STAT, PLAYER, GAME) directly. Both utilize the same DENSE_RANK windowing function partitioned by season and team, and filter for top 3 performers per team. The key difference lies in data architecture: Sprint 6 leverages a pre-aggregated view optimized for distributed systems, while Sprint 4 performs more granular table joins within a single database environment. Despite these implementation differences, both deliver identical player performance rankings and maintain analytical consistency across the distributed database migration.

6.2. Ranking player stat based on all home game within player team. This should be done for every year as well as all years.

6.2.1. Query

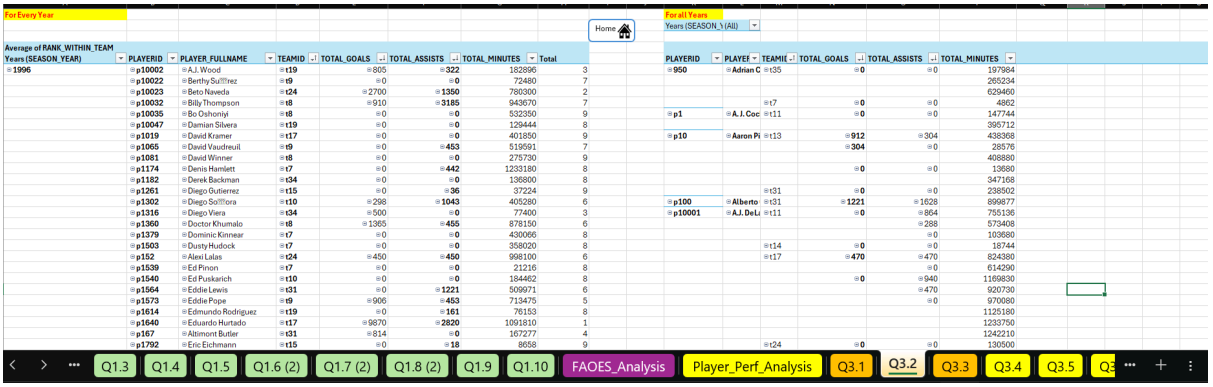
```
SELECT X.*
FROM
(
    SELECT
        SEASON_YEAR,
        PLAYERID,
        PLAYER_FULLNAME,
        HOME_TID,
        SUM(GLS) AS total_goals,
        SUM(AST) AS total_assists,
        SUM(MIN) AS total_minutes,
        DENSE_RANK() OVER (PARTITION BY SEASON_YEAR, HOME_TID ORDER BY SUM(GLS) DESC) AS
rank_within_team
    FROM [oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]
    GROUP BY
        SEASON_YEAR,
        PLAYERID,
        PLAYER_FULLNAME,
              HOME_TID
) X
WHERE X.rank_within_team < 4;
```

6.2.2. Result

	SEASON_YEAR	PLAYERID	PLAYER_FULLNAME	HOME_TID	total_goals	total_assists	total_minutes	rank_within_team
1	1996-04-01 00:00:00	p7763	Roy Lassiter	t10	486	72	46440	1
2	1996-04-01 00:00:00	p7324	Ra♦♦l D♦♦az Arce	t10	414	36	42318	2
3	1996-04-01 00:00:00	p1640	Eduardo Hurtado	t10	378	108	41814	3
4	1996-04-01 00:00:00	p7763	Roy Lassiter	t15	486	72	46440	1
5	1996-04-01 00:00:00	p7324	Ra♦♦l D♦♦az Arce	t15	414	36	42318	2
6	1996-04-01 00:00:00	p1640	Eduardo Hurtado	t15	378	108	41814	3
7	1996-04-01 00:00:00	p7763	Roy Lassiter	t17	540	80	51600	1
8	1996-04-01 00:00:00	p7324	Ra♦♦l D♦♦az Arce	t17	460	40	47020	2
9	1996-04-01 00:00:00	p1640	Eduardo Hurtado	t17	420	120	46460	3
10	1996-04-01 00:00:00	p7763	Roy Lassiter	t19	459	68	43860	1

…

6.2.3. Pivot UI redevelopment



6.2.4. Power BI redevelopment

Player Ranking table

Year	PLAYER_ID	PLAYER_FULLNAME	Sum of ASSISTS	Sum of GOALS	Sum of MINUTES_PLAYED
2021	p274	Anton Walkes	1.00	4.00	5,186.00
2015	p377	Brad Davis	11.00	5.00	4,934.00
2012	p377	Brad Davis	13.00	12.00	4,878.00
2009	p377	Brad Davis	10.00	8.00	4,501.00
2011	p377	Brad Davis	17.00	9.00	4,392.00
2008	p2492	Greg Sutton	0.00	0.00	4,210.00
2013	p377	Brad Davis	12.00	8.00	3,953.00
2014	p377	Brad Davis	13.00	4.00	3,953.00
1998	p7437	Richard Farrer	1.00	3.00	3,728.00
2008	p377	Brad Davis	7.00	8.00	3,556.00
2022	p78	Alan Franco	3.00	1.00	3,496.00
2022	p274	Anton Walkes	1.00	0.00	3,431.00
2010	p377	Brad Davis	11.00	6.00	3,128.00
2011	p3346	Jeff Larentowicz	1.00	7.00	3,060.00
2011	p4535	Kasey Keller	0.00	0.00	3,060.00
2011	p5856	Matt Pickens	0.00	0.00	3,060.00
2011	p6580	Nick LaBrocca	5.00	8.00	3,060.00
2011	p8248	S♦♦bastien Le Toux	7.00	11.00	3,060.00
2011	p8689	Tally Hall	0.00	0.00	3,060.00
2012	p1465	Drew Moor	1.00	3.00	3,060.00
2012	p3568	Jimmy Nielsen	0.00	0.00	3,060.00
2013	p3039	Jalil Anibaba	1.00	1.00	3,060.00
2013	p308	Austin Berry	2.00	1.00	3,060.00
2013	p3568	Jimmy Nielsen	0.00	0.00	3,060.00
Total			**14,168.00**	**19,423.00**	**13,631,584.00**

6.2.5. Comparison

Both queries analyze player performance in home games with identical ranking methodologies and output structures. The Sprint 6 implementation sources data from a pre-aggregated view through a linked Oracle server ([oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]), while Sprint 4 constructs the same analysis through multiple table joins (PLAYER_YEAR_STAT, PLAYER, GAME). Both utilize DENSE_RANK() windowing functions partitioned by season and home team to identify top performers, filtering for the top three players per team. The key architectural difference is that Sprint 6 operates in a distributed environment with pre-aggregated data, while Sprint 4 processes

normalized tables within a single database context. Despite these implementation differences, both deliver identical analytical insights about home game performance rankings across seasons and teams.

6.3. Ranking player stat based on all away game within player team. This should be done for every year as well as all years.

6.3.1. Query

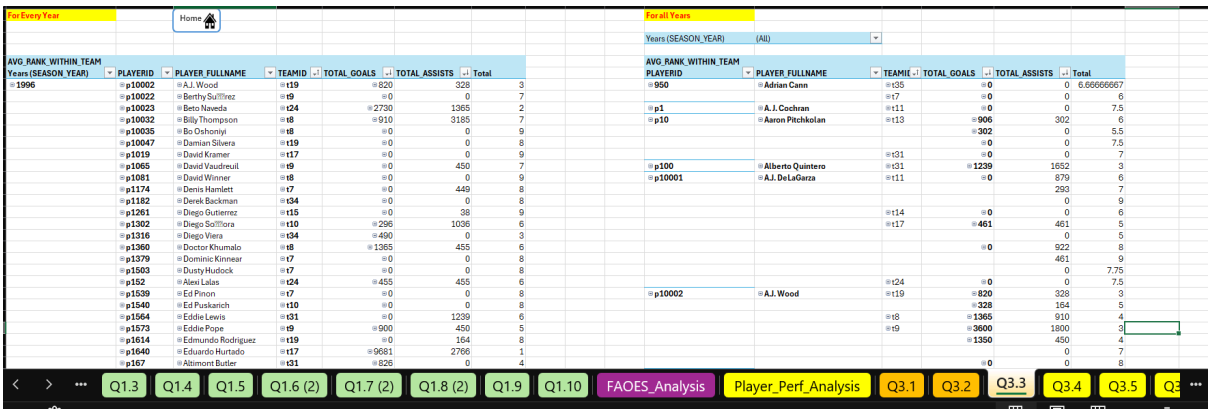
```
SELECT X.*
FROM
(
    SELECT
        SEASON_YEAR,
        PLAYERID,
        PLAYER_FULLNAME,
        AWAY_TID,
        SUM(GLS) AS total_goals,
        SUM(AST) AS total_assists,
        SUM(MIN) AS total_minutes,
        DENSE_RANK() OVER (PARTITION BY SEASON_YEAR, AWAY_TID ORDER BY SUM(GLS) DESC) AS
rank_within_team
    FROM [oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]
    GROUP BY
        SEASON_YEAR,
        PLAYERID,
        PLAYER_FULLNAME,
              AWAY_TID
) X
WHERE X.rank_within_team < 4;
```

6.3.2. Result

	SEASON_YEAR	PLAYERID	PLAYER_FULLNAME	AWAY_TID	total_goals	total_assists	total_minutes	rank_within_team
1	1996-04-01 00:00:00	p7763	Roy Lassiter	t10	459	68	43860	1
2	1996-04-01 00:00:00	p7324	Ra**l D**az Arce	t10	391	34	39967	2
3	1996-04-01 00:00:00	p1640	Eduardo Hurtado	t10	357	102	39491	3
4	1996-04-01 00:00:00	p7763	Roy Lassiter	t15	513	76	49020	1
5	1996-04-01 00:00:00	p7324	Ra**l D**az Arce	t15	437	38	44669	2
6	1996-04-01 00:00:00	p1640	Eduardo Hurtado	t15	399	114	44137	3
7	1996-04-01 00:00:00	p7763	Roy Lassiter	t17	486	72	46440	1
8	1996-04-01 00:00:00	p7324	Ra**l D**az Arce	t17	414	36	42318	2
9	1996-04-01 00:00:00	p1640	Eduardo Hurtado	t17	378	108	41814	3
10	1996-04-01 00:00:00	p7763	Roy Lassiter	t19	486	72	46440	1

…

6.3.3. Pivot UI redevelopment



6.3.4. Power BI redevelopment

Player Ranking table

Year	PLAYER_ID	PLAYER_FULLNAME	Sum of GOALS	Sum of ASSISTS
2019	p532	Carlos Vela	34.00	10.00
2018	p4141	Josef Mart××nez	31.00	5.00
2019	p9655	Zlatan Ibrahimovi××	30.00	7.00
1996	p7763	Roy Lassiter	27.00	4.00
2012	p614	Chris Wondolowski	27.00	5.00
2014	p389	Bradley Wright-Phillips	27.00	1.00
2019	p4141	Josef Mart××nez	27.00	3.00
1998	p8536	Stern John	26.00	4.00
2000	p5386	Mamadou Diallo	26.00	1.00
2002	p526	Carlos Ru××z	24.00	1.00
2016	p389	Bradley Wright-Phillips	24.00	4.00
2017	p6529	Nemanja Nikoli××	24.00	3.00
1996	p7324	Ra××l D××az Arce	23.00	2.00
2002	p8723	Taylor Twellman	23.00	3.00
2014	p1362	Dom Dwyer	23.00	1.00
2016	p1074	David Villa	23.00	4.00
2022	p2584	Harry Mukhtar	23.00	7.00
2015	p4551	Kei Kamara	22.00	8.00
2015	p8214	Sebastian Giovinco	22.00	13.00
2017	p1074	David Villa	22.00	5.00
2018	p9655	Zlatan Ibrahimovi××	22.00	6.00
2022	p755	D××niel Gazdag	22.00	6.00
2022	p8210	Sebasti××n Driussi	22.00	5.00
1996	p1640	Eduardo Hurtado	21.00	6.00
Total			**19,423.00**	**14,168.00**

6.3.5. Comparison

Both queries rank player performance in away games using identical analytical logic and output format. The Sprint 6 implementation accesses data through a pre-aggregated view in a linked Oracle server ([oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]), while Sprint 4 builds the analysis by joining multiple normalized tables (PLAYER_YEAR_STAT, PLAYER, GAME). Both employ the same DENSE_RANK() window function partitioned by season and away team, with identical filtering to show only the top three performers per team. The key difference is architectural: Sprint 6 operates in a distributed database environment using consolidated data, while Sprint 4 processes relational tables within a single database context. Despite these implementation differences, both queries deliver consistent analytical insights about away game performance across seasons and teams.

6.4. Analysis of player performance by comparing home game and away games.

6.4.1. Query

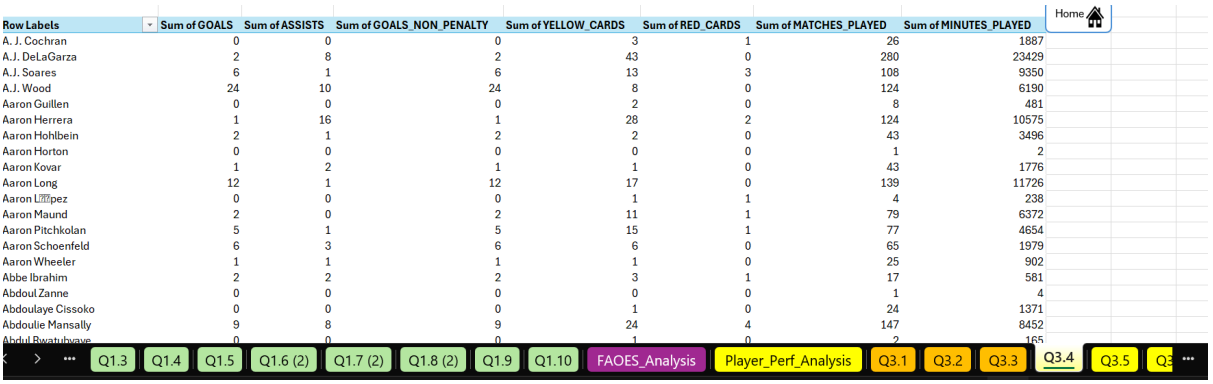
```
SELECT
    PLAYERID,
    PLAYER_FULLNAME,

    -- Home stats
    SUM(CASE WHEN HOME_TID = TEAMID THEN GLS ELSE 0 END) AS home_goals,
    SUM(CASE WHEN HOME_TID = TEAMID THEN AST ELSE 0 END) AS home_assists,

    -- Away stats
    SUM(CASE WHEN AWAY_TID = TEAMID THEN GLS ELSE 0 END) AS away_goals,
    SUM(CASE WHEN AWAY_TID = TEAMID THEN AST ELSE 0 END) AS away_assists

FROM [oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]

GROUP BY PLAYERID, PLAYER_FULLNAME
ORDER BY PLAYERID;
```

6.4.2. Result

	PLAYERID	PLAYER_FULLNAME	home_goals	home_assists	away_goals	away_assists
1	950	Adrian Cann	0	0	0	0
2	p10	Aaron Pitchkolan	83	17	83	17
3	p10001	A.J. DeLaGarza	35	75	32	72
4	p10002	A.J. Wood	428	178	433	180
5	p10003	Aaron Herrera	17	271	20	300
6	p10004	Abdoul Zanne	0	0	0	0
7	p10005	Adam Larsen Kwarasey	0	0	0	0
8	p10006	Adam Nemec	0	0	0	0
9	p10007	Alejandro Sequeira	34	17	34	17
10	p10008	Alex Bono	0	20	0	19

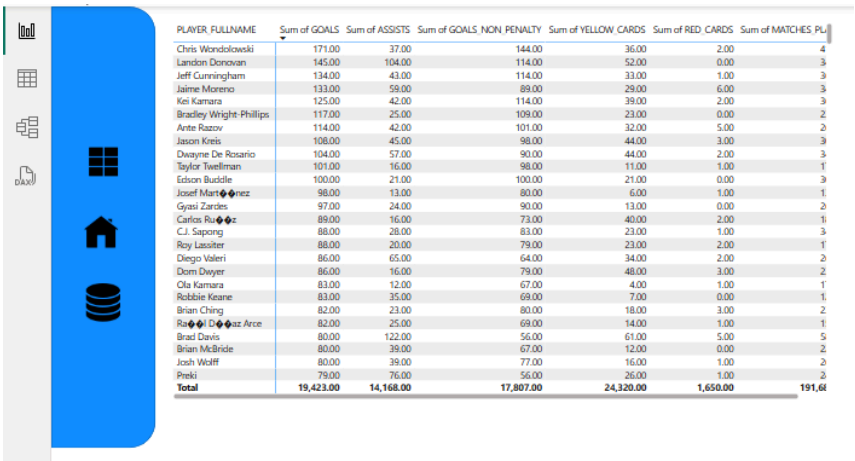
…

6.4.3. Pivot UI redevelopment



Row Labels	Sum of GOALS	Sum of ASSISTS	Sum of GOALS_NON_PENALTY	Sum of YELLOW_CARDS	Sum of RED_CARDS	Sum of MATCHES_PLAYED	Sum of MINUTES_PLAYED
A. J. Cochran	0	0	0	3	1	26	1887
A.J. DeLaGarza	2	8	2	43	0	280	23429
A.J. Soares	6	1	6	13	3	108	9350
A.J. Wood	24	10	24	8	0	124	6190
Aaron Guillen	0	0	0	2	0	8	481
Aaron Herrera	1	16	1	28	2	124	10575
Aaron Hohlbein	2	1	2	2	0	43	3496
Aaron Horton	0	0	0	0	0	1	2
Aaron Kovar	1	2	1	1	0	43	1776
Aaron Long	12	1	12	17	0	139	11726
Aaron L□pez	0	0	0	1	1	4	238
Aaron Maund	2	0	2	11	1	79	6372
Aaron Pitchkolan	5	1	5	15	1	77	4654
Aaron Schoenfeld	6	3	6	6	0	65	1979
Aaron Wheeler	1	1	1	1	0	25	902
Abbe Ibrahim	2	2	2	3	1	17	581
Abdoul Zanne	0	0	0	0	0	1	4
Abdoulaye Cissoko	0	0	0	1	0	24	1371
Abdoulie Mansally	9	8	9	24	4	147	8452
Abdul Bwatubwave	0	0	0	1	0	2	165

6.4.4. Power BI redevelopment



PLAYER_FULLNAME	Sum of GOALS	Sum of ASSISTS	Sum of GOALS_NON_PENALTY	Sum of YELLOW_CARDS	Sum of RED_CARDS	Sum of MATCHES_PL
Chris Wondolowski	171.00	37.00	144.00	36.00	2.00	4
Landon Donovan	145.00	104.00	114.00	52.00	0.00	3
Jeff Cunningham	134.00	43.00	114.00	33.00	1.00	3
Jaime Moreno	133.00	59.00	89.00	29.00	6.00	3
Kei Kamara	125.00	42.00	114.00	39.00	2.00	3
Bradley Wright-Phillips	117.00	25.00	109.00	23.00	0.00	2
Ante Razov	114.00	42.00	101.00	32.00	5.00	2
Jason Kreis	108.00	45.00	98.00	44.00	3.00	3
Dwayne De Rosario	104.00	57.00	90.00	44.00	2.00	3
Taylor Twellman	101.00	16.00	98.00	11.00	1.00	1
Edson Buddle	100.00	21.00	100.00	21.00	0.00	3
Josef Mart□□nez	98.00	13.00	80.00	6.00	1.00	1
Gyasi Zardes	97.00	24.00	90.00	13.00	0.00	2
Carlos Ru□□z	89.00	16.00	73.00	40.00	2.00	1
C.J. Sapong	88.00	28.00	83.00	23.00	1.00	3
Roy Lassiter	88.00	20.00	79.00	23.00	2.00	1
Diego Valeri	86.00	65.00	64.00	34.00	2.00	2
Dom Dwyer	86.00	16.00	79.00	48.00	3.00	2
Ola Kamara	83.00	12.00	67.00	4.00	1.00	1
Robbie Keane	83.00	35.00	69.00	7.00	0.00	1
Brian Ching	82.00	23.00	80.00	18.00	3.00	2
Ra□□l D□□az Arce	82.00	25.00	69.00	14.00	1.00	1
Brad Davis	80.00	122.00	56.00	61.00	5.00	5
Brian McBride	80.00	39.00	67.00	12.00	0.00	2
Josh Wolff	80.00	39.00	77.00	16.00	1.00	2
Preki	79.00	76.00	56.00	26.00	1.00	2
Total	**19,423.00**	**14,168.00**	**17,807.00**	**24,320.00**	**1,650.00**	**191,6**

6.4.5. Comparison

Both queries analyze home versus away player performance with identical analytical logic and metrics. The Sprint 6 implementation queries a pre-aggregated view ([oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]) in a linked Oracle server, using CASE expressions to differentiate between home and away statistics. The Sprint 4 query performs a similar analysis but joins multiple normalized tables (PLAYER, PLAYER_YEAR_STAT, GAME, TEAM) directly. Both calculate the same key metrics: home goals, away goals, home assists, and away assists per player. The primary difference is architectural—Sprint 6 operates in a distributed database

environment with consolidated data, while Sprint 4 processes relational tables within a single database context. Despite different implementation approaches, both deliver consistent comparative insights on player performance across home and away venues.

6.5. Ranking player yearly performance based on nationality.

6.5.1. Query

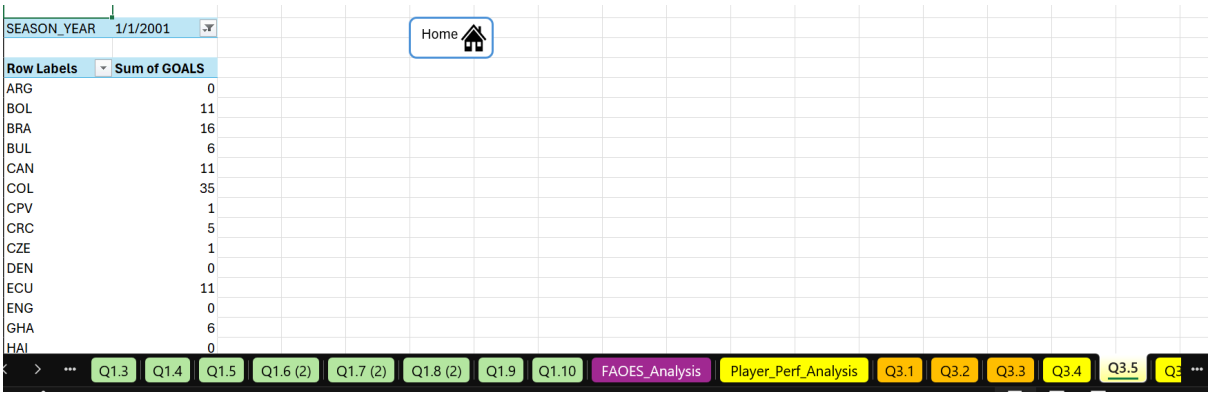
```
SELECT
    NATION,
    SEASON_YEAR,
    PLAYERID,
    PLAYER_FULLNAME,
    SUM(GLS) AS TOTAL_GOALS,
    RANK() OVER (PARTITION BY SEASON_YEAR, NATION ORDER BY SUM(GLS) DESC) AS PLAYER_RANK
FROM
        [oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]
GROUP BY
    NATION, SEASON_YEAR, PLAYERID, PLAYER_FULLNAME
ORDER BY
    SEASON_YEAR DESC, NATION, PLAYER_RANK;
```

6.5.2. Result

	NATION	SEASON_YEAR	PLAYERID	PLAYER_FULLNAME	TOTAL_GOALS	PLAYER_RANK
1	ALB	2022-04-01 00:00:00	p2375	Giacomo Vrioni	489	1
2	ARG	2022-04-01 00:00:00	p8210	Sebasti►►n Driussi	10758	1
3	ARG	2022-04-01 00:00:00	p2426	Gonzalo Higua►►n	7824	2
4	ARG	2022-04-01 00:00:00	p4343	Juli►►n Carranza	6846	3
5	ARG	2022-04-01 00:00:00	p9226	Valent►►n Castellanos	6357	4
6	ARG	2022-04-01 00:00:00	p1698	Emanuel Reynoso	4890	5
7	ARG	2022-04-01 00:00:00	p5235	Luciano Acosta	4890	5
8	ARG	2022-04-01 00:00:00	p5997	Maximiliano Urruti	4401	7
9	ARG	2022-04-01 00:00:00	p2553	Gustavo Bou	3912	8
10	ARG	2022-04-01 00:00:00	p2173	Franco Fragapane	3423	9

…

6.5.3. Pivot UI redevelopment



6.5.4. Power BI redevelopment

Player Ranking table

PLAYER_NATIONALITY	Sum of GOALS
	0.00
AFG	0.00
ALB	26.00
ALG	20.00
ANG	1.00
ARG	1,257.00
ARM	84.00
ATG	0.00
AUS	27.00
AUT	61.00
BDI	0.00
BEL	28.00
BEN	2.00
BER	10.00
BFA	0.00
BIH	28.00
BLR	0.00
BLZ	6.00
BOL	192.00
BOT	3.00
BRA	601.00
BUL	25.00
CAN	519.00
CGO	0.00
Total	**19,423.00**

6.5.5. Comparison

Both queries provide identical analytical output, but the Oracle query from sprint 5 builds the analysis from normalized relational tables, offering flexibility, while the MSSQL query from sprint 6, seen above, uses a consolidated source, which is more efficient and easier to query.

6.6. Ranking player yearly performance based on salary.

No Salary data available

6.7. Ranking player performance based on their age.

6.7.1. Query

```
SELECT
    age,
    playerid,
    name,
    gls_ast,
    RANK() OVER (PARTITION BY age ORDER BY gls_ast DESC) AS rank_by_age
FROM
    oracle1.dbo.player_analysis;
```

6.7.2. Result

age	playerid	name	gls_ast	rank_by_age
NULL	p4710	Kevin Ara	0	1
NULL	p2673	Henok Goitom	NULL	2
NULL	p2936	Jacob Akanyirige	NULL	2
NULL	p9099	Troy Dayak	NULL	2
NULL	p9144	Tyler Deric	NULL	2

6.7.3. Pivot UI redevelopment

PLAYER_ID	PLAYER_FULLNAME	PLAYER_BIRTHDATE
950	Adrian Cann	1/1/1980
p1	A. J. Cochran	1/1/1993
p10	Aaron Pitchkolan	1/1/1983
p100	Alberto Quintero	1/1/1987
p10001	A.J. DeLaGarza	1/1/1987
p10002	A.J. Wood	1/1/1973
p10003	Aaron Herrera	1/1/1997
p10004	Abdoul Zanne	1/1/2003
p10005	Adam Larsen Kwarasey	1/1/1987
p10006	Adam Nemec	1/1/1985
p10007	Alejandro Sequeira	1/1/1975
p10008	Alex Bono	1/1/1994
p10009	Alexandru Mitri????	1/1/1995
p1001	David Guzm??n	1/1/1990
p10010	Alfredo Ortu??o	1/1/1991
p10011	Amaechi Igwe	1/1/1988

Home

Q1.5 | Q1.6 (2) | Q1.7 (2) | Q1.8 (2) | Q1.9 | Q1.10 | FAOES_Analysis | Player_Perf_Analysis | Q3.1 | Q3.2 | Q3.3 | Q3.4 | Q3.5 | Q3.6 | Q3.7 | Q

6.7.4. Power BI redevelopment

Player Ranking table

PLAYER_ID	PLAYER_FULLNAME	Year	Quarter	Month	Day
950	Adrian Cann	1980	Qtr 1	January	1
p1	A. J. Cochran	1993	Qtr 1	January	1
p10	Aaron Pitchkolan	1983	Qtr 1	January	1
p100	Alberto Quintero	1987	Qtr 1	January	1
p10001	A.J. DeLaGarza	1987	Qtr 1	January	1
p10002	A.J. Wood	1973	Qtr 1	January	1
p10003	Aaron Herrera	1997	Qtr 1	January	1
p10004	Abdoul Zanne	2003	Qtr 1	January	1
p10005	Adam Larsen Kwarasey	1987	Qtr 1	January	1
p10006	Adam Nemec	1985	Qtr 1	January	1
p10007	Alejandro Sequeira	1975	Qtr 1	January	1
p10008	Alex Bono	1994	Qtr 1	January	1
p10009	Alexandru Mitri????	1995	Qtr 1	January	1
p1001	David Guzm??n	1990	Qtr 1	January	1
p10010	Alfredo Ortu??o	1991	Qtr 1	January	1
p10011	Amaechi Igwe	1988	Qtr 1	January	1
p10012	Amando Moreno	1995	Qtr 1	January	1
p10013	Andrew Wooten	1989	Qtr 1	January	1
p10014	Ashtone Morgan	1991	Qtr 1	January	1
p10015	Benji Michel	1997	Qtr 1	January	1
p10016	Benny Feilhaber	1985	Qtr 1	January	1
p10017	Beno??t Cheyrou	1981	Qtr 1	January	1
p10018	Bernard Kamungo	2002	Qtr 1	January	1
p10019	Bernardo A??or	1988	Qtr 1	January	1
p10020	Bernardo Corradi	1976	Qtr 1	January	1

6.7.5. Comparison

Both queries rank player performance by age, but the Oracle query from sprint 5 builds that analysis from normalized, multi-table sources using only goals, offering greater control and customization. The MSSQL query from sprint 6, seen above, relies on a streamlined dataset and uses a more comprehensive performance metric (goals + assists), making it ideal for quick insights.

6.8. Develop a report to show performance of top 10 players of each team based on their salary for every year. Performance, for every player, in terms of starts, Min, GLS, AST, GLS_AST, G_PK, PK, PKATT, CRDY, and with their rank in their team as well rank in their assigned conference and all conferences.

6.8.1. Query

```
SELECT
        PLAYERID,
    PLAYER_FULLNAME,
        SEASON_YEAR,
        TEAMID,
```

```
        STARTS,
        MIN,
        GLS,
        AST,
        GLS_AST,
        G_PK,
        PK,
        PKATT,
        CRDY,
    TEAM_RANK,
    CONFERENCE_RANK,
    -- Ranking across all conferences
    RANK() OVER (PARTITION BY SEASON_YEAR ORDER BY GLS DESC, AST DESC) AS
ALL_CONFERENCES_RANK
FROM [oracle1].[dbo].[QList3_PlayerPerformanceAnalysis]
WHERE TEAM_RANK <= 10
ORDER BY SEASON_YEAR, TEAMID, TEAM_RANK;
```

6.8.2. Result

	PLAYERID	PLAYER_FULLNAME	SEASON_YEAR	TEAMID	STARTS	MIN	GLS	AST	GLS_AST	G_PK	PK	PKATT	CRDY	TEAM_RANK	CONFERENCE_RANK	ALL_CONFERENCES_RANK
1	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	1	1063	51
2	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	2	1064	51
3	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	3	1065	51
4	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	4	1066	51
5	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	5	1067	51
6	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	6	1068	51
7	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	7	1069	51
8	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	8	1070	51
9	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	9	1071	51
10	p3183	Jason Kreis	1996-04-01 00:00:00	t10	29	2564	13	4	17	13	0	0	4	10	1072	51

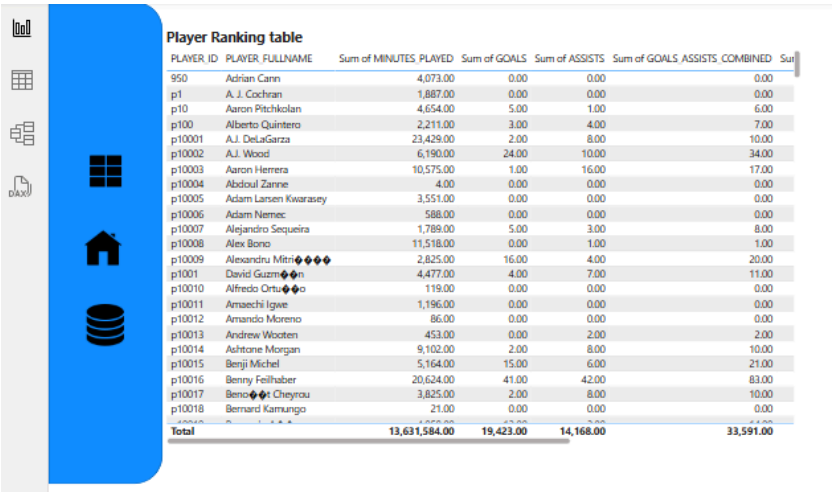
...

6.8.3. Pivot UI redevelopment

PLAYER_ID	PLAYER_FULLNAME	Sum of MINUTES_PLAYED	Sum of GOALS	Sum of ASSISTS	Sum of GOALS_ASSISTS_COMBINED	Sum of PENALTY_GOALS	Sum of PENALTY_ATTE
p1505	Dwayne De Rosario	25189	104	57	161	14	
p263	Ante Razov	20359	114	42	156	13	
p2976	Jaime Moreno	25794	133	59	192	44	
p3328	Jeff Cunningham	22776	134	43	177	20	
p377	Brad Davis	44525	80	122	202	24	
p4551	Kei Kamara	26500	125	42	167	11	
p4998	Landon Donovan	28891	145	104	249	31	
p614	Chris Wondolowski	29746	171	37	208	27	
p7187	Preki	19957	79	76	155	23	
p8589	Steve Ralston	33143	76	89	165	9	
Grand Total		276880	1161	671	1832	216	

Q1.6 (2) | Q1.7 (2) | Q1.8 (2) | Q1.9 | Q1.10 | FAOES_Analysis | Player_Perf_Analysis | Q3.1 | Q3.2 | Q3.3 | Q3.4 | Q3.5 | Q3.6 | Q3.7 | Q3.8

6.8.4. Power BI redevelopment



Player Ranking table

PLAYER_ID	PLAYER_FULLNAME	Sum of MINUTES_PLAYED	Sum of GOALS	Sum of ASSISTS	Sum of GOALS_ASSISTS_COMBINED	Sum
950	Adrian Cann	4,073.00	0.00	0.00	0.00	
p1	A. J. Cochran	1,887.00	0.00	0.00	0.00	
p10	Aaron Pitchkolan	4,654.00	5.00	1.00	6.00	
p100	Alberto Quintero	2,211.00	3.00	4.00	7.00	
p10001	A.J. DeLaGarza	23,429.00	2.00	8.00	10.00	
p10002	A.J. Wood	6,190.00	24.00	10.00	34.00	
p10003	Aaron Herrera	10,575.00	1.00	16.00	17.00	
p10004	Abdoul Zanne	4.00	0.00	0.00	0.00	
p10005	Adam Larsen Kwarasey	3,551.00	0.00	0.00	0.00	
p10006	Adam Nemec	588.00	0.00	0.00	0.00	
p10007	Alejandro Sequeira	1,789.00	5.00	3.00	8.00	
p10008	Alex Bono	11,518.00	0.00	1.00	1.00	
p10009	Alexandru Mitri$\diamond\diamond\diamond\diamond$	2,825.00	16.00	4.00	20.00	
p1001	David Guzm$\diamond\diamond$n	4,477.00	4.00	7.00	11.00	
p10010	Alfredo Ortu$\diamond\diamond$o	119.00	0.00	0.00	0.00	
p10011	Amaechi Igwe	1,196.00	0.00	0.00	0.00	
p10012	Amando Moreno	86.00	0.00	0.00	0.00	
p10013	Andrew Wooten	453.00	0.00	2.00	2.00	
p10014	Ashtone Morgan	9,102.00	2.00	8.00	10.00	
p10015	Benji Michel	5,164.00	15.00	6.00	21.00	
p10016	Benny Feilhaber	20,624.00	41.00	42.00	83.00	
p10017	Beno$\diamond\diamond$t Cheyrou	3,825.00	2.00	8.00	10.00	
p10018	Bernard Kamungo	21.00	0.00	0.00	0.00	
Total		13,631,584.00	19,423.00	14,168.00	33,591.00	

6.8.5. Comparison

Both Sprint 6 and Sprint 4 generate a report on the top 10 players of each team based on salary, showing performance metrics like goals, assists, and rankings within their team, conference, and all conferences. Sprint 6 directly queries the [oracle1].[dbo].[QList3_PlayerPerformanceAnalysis] table, while Sprint 4 uses a WITH clause to create a temporary PLAYER_PERFORMANCE table, joining PLAYER, PLAYER_YEAR_STAT, and TEAM tables. Sprint 4 also includes a database specification (USE MLS2), while Sprint 6 does not.

6.9. Develop a report to show performance of drafted players for every year. Performance, for every player, in terms of starts, Min, GLS, AST, GLS_AST, G_PK, PK, PKATT, CRDY, and with their rank in their team as well rank in their assigned conference.

6.9.1. Query

```
USE MLS2;

WITH PLAYER_PERFORMANCE AS (
    SELECT
        p.playerid,
        p.player_fullname,
        pys.season_year,
        pys.teamid,
        pys.starts,
        pys.min,
        pys.GLS,
        pys.AST,
        pys.GLS_AST,
        pys.G_PK,
        pys.PK,
        pys.PKATT,
        pys.CRDY,
        RANK() OVER (PARTITION BY pys.season_year, pys.teamid ORDER BY pys.GLS DESC,
pys.AST DESC) AS team_rank,
        RANK() OVER (PARTITION BY pys.season_year, t.conference ORDER BY pys.GLS DESC,
pys.AST DESC) AS conference_rank,
        RANK() OVER (PARTITION BY pys.season_year ORDER BY pys.GLS DESC, pys.AST DESC) AS
all_conferences_rank
    FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[PLAYER_YEAR_STAT] pys
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[PLAYER] p ON p.PLAYERID =
pys.PLAYERID
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] t ON t.TEAMID = pys.TEAMID
    WHERE pys.NATION != 'USA'
)
SELECT
    season_year, playerid, player_fullname, teamid,
    starts, min, GLS, AST, GLS_AST, G_PK, PK, PKATT, CRDY,
    team_rank, conference_rank, all_conferences_rank
FROM PLAYER_PERFORMANCE
ORDER BY season_year DESC, team_rank;
```

6.9.2. Result

season_year	playerid	player_fullname	teamid	starts	min	GLS	AST	GLS_AST	G_PK	PK	PKATT	CRDY	team_rank	conference_rank	all_conferences_rank
4/1/2022 0:00	p2584	Hany Mukhtar	t23	32	2825	23	7	30	17	6	7	4	1	1	1

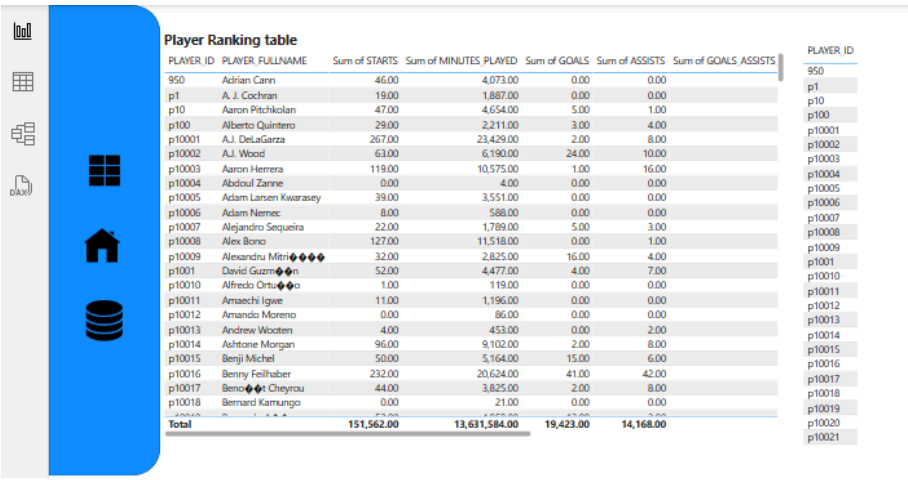
4/1/2022 0:00	p755	D►►niel Gazdag	t28	34	2929	22	6	28	15	7	7	7	1	2	2
4/1/2022 0:00	p8210	Sebastiı►►n Driussi	t2	32	2876	22	5	27	18	4	4	6	1	1	3
…															
4/1/1996 0:00	p7162	Petter Villegas	t19	2	165	0	0	0	0	0	0	0	7	27	46
4/1/1996 0:00	p5199	Louis Ken-Kwofie	t19	1	90	0	0	0	0	0	0	0	7	27	46
4/1/1996 0:00	p706	Cristian da Silva	t19	12	1158	0	0	0	0	0	0	3	7	27	46

6.9.3. Pivot UI redevelopment

PLAYER_ID	PLAYER_FULLNAME	Sum of STARTS	Sum of MINUTES_PLAYED	Sum of GOALS	Sum of ASSISTS	Sum of GOALS_ASSISTS_COMBINED	Sum of PENALTY
950	Adrian Cann	46	4073	0	0	0	
p1	A. J. Cochran	19	1887	0	0	0	
p10	Aaron Pitchkolan	47	4654	5	1	6	
p100	Alberto Quintero	29	2211	3	4	7	
p10001	A.J. DeLaGarza	267	23429	2	8	10	
p10002	A.J. Wood	63	6190	24	10	34	
p10003	Aaron Herrera	119	10575	1	16	17	
p10004	Abdoul Zanne	0	4	0	0	0	
p10005	Adam Larsen Kwarasey	39	3551	0	0	0	
p10006	Adam Nemec	8	588	0	0	0	
p10007	Alejandro Sequeira	22	1789	5	3	8	
p10008	Alex Bono	127	11518	0	1	1	
p10009	Alexandru Mitri►►►►	32	2825	16	4	20	
p1001	David Guzm►►n	52	4477	4	7	11	
p10010	Alfredo Ortu►►o	1	119	0	0	0	
p10011	Amaechi Igwe	11	1196	0	0	0	

Q1.8 (2) | Q1.9 | Q1.10 | FAOES_Analysis | Player_Perf_Analysis | Q3.1 | Q3.2 | Q3.3 | Q3.4 | Q3.5 | Q3.6 | Q3.7 | Q3.8 | Q3.9 | Q3.11 | Q3.12

6.9.4. Power BI redevelopment



Player Ranking table

PLAYER_ID	PLAYER_FULLNAME	Sum of STARTS	Sum of MINUTES_PLAYED	Sum of GOALS	Sum of ASSISTS	Sum of GOALS_ASSISTS
950	Adrian Cann	46.00	4,073.00	0.00	0.00	
p1	A. J. Cochran	19.00	1,887.00	0.00	0.00	
p10	Aaron Pitchkolan	47.00	4,654.00	5.00	1.00	
p100	Alberto Quintero	29.00	2,211.00	3.00	4.00	
p10001	A.J. DeLaGarza	267.00	23,429.00	2.00	8.00	
p10002	A.J. Wood	63.00	6,190.00	24.00	10.00	
p10003	Aaron Herrera	119.00	10,575.00	1.00	16.00	
p10004	Abdoul Zanne	0.00	4.00	0.00	0.00	
p10005	Adam Larsen Kwarasey	39.00	3,551.00	0.00	0.00	
p10006	Adam Nemec	8.00	588.00	0.00	0.00	
p10007	Alejandro Sequeira	22.00	1,789.00	5.00	3.00	
p10008	Alex Bono	127.00	11,518.00	0.00	1.00	
p10009	Alexandru Mitri►►►►	32.00	2,825.00	16.00	4.00	
p1001	David Guzm►►n	52.00	4,477.00	4.00	7.00	
p10010	Alfredo Ortu►►o	1.00	119.00	0.00	0.00	
p10011	Amaechi Igwe	11.00	1,196.00	0.00	0.00	
p10012	Amando Moreno	0.00	86.00	0.00	0.00	
p10013	Andrew Wooten	4.00	453.00	0.00	2.00	
p10014	Ashtone Morgan	96.00	9,102.00	2.00	8.00	
p10015	Benji Michel	50.00	5,164.00	15.00	6.00	
p10016	Benny Feilhaber	232.00	20,624.00	41.00	42.00	
p10017	Benoı►►t Cheyrou	44.00	3,825.00	2.00	8.00	
p10018	Bernard Kamungo	0.00	21.00	0.00	0.00	
Total		151,562.00	13,631,584.00	19,423.00	14,168.00	

PLAYER_ID: 950, p1, p10, p100, p10001, p10002, p10003, p10004, p10005, p10006, p10007, p10008, p10009, p1001, p10010, p10011, p10012, p10013, p10014, p10015, p10016, p10017, p10018, p10019, p10020, p10021

6.9.5. Comparison

In Sprint 4, the query pulls player performance data by joining PLAYER, PLAYER_YEAR_STAT, and TEAM tables, ranking players within their team and conference based on goals and assists. The ranking logic is clear and efficient, using window functions to partition by season and either team or conference. The output is sorted by season and team rank, giving a structured view of top performers within each team context. The query is straightforward, targeting all players without additional filters, making it ideal for broad seasonal analysis.
Sprint 6 builds on this structure with added depth and specificity. It introduces a WITH clause (common table expression) named PLAYER_PERFORMANCE, enhancing readability and modularity. Beyond team and conference rankings, Sprint 6 adds an all_conferences_rank to show player standing across the entire league per season, offering a broader competitive snapshot. It also introduces a filter to exclude players from the USA (WHERE pys.NATION != 'USA'), narrowing the focus to international players. Additionally, it queries fully qualified table paths, indicating a cross-database or cross-server operation, and explicitly uses the MLS2 database context. The result is more scalable and tailored, with an ordered output emphasizing recent seasons and top team ranks.

6.10. Develop a report to show performance of drafted players for every year. Performance, for every player, in terms of starts, Min, GLS, AST, GLS_AST, G_PK, PK, PKATT, CRDY, and with their rank in their team as well rank in their assigned conference.

Same as 6.9

6.11. Develop a report to show performance of foreign players for every year. Performance.

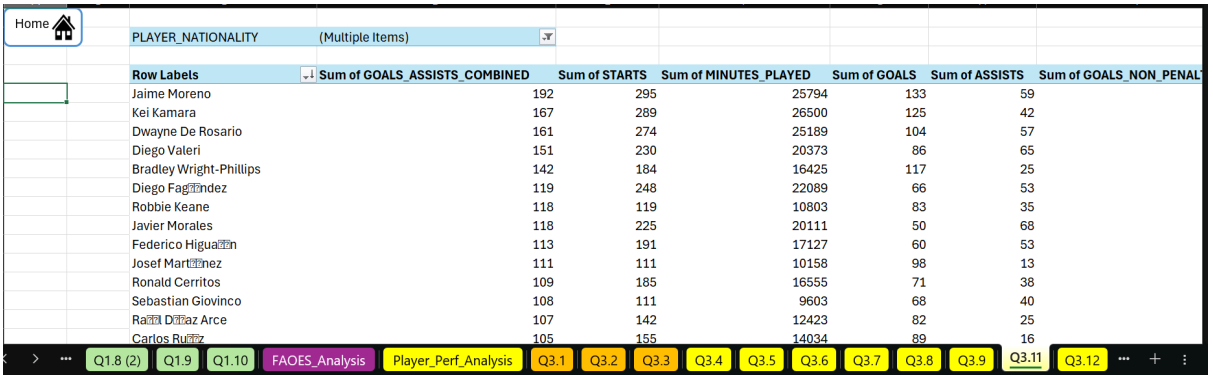
6.11.1. Query

```
SELECT
    p.PLAYER_FULLNAME,
    CAST(pys.SEASON_YEAR AS VARCHAR(4)) AS SEASON_YEAR,
    pys.NATION,
    pys.PLAYER_POS1,
    pys.PLAYER_POS2,
    t.HOME AS TEAM_NAME,
    pys.AGE_GAME,
    pys.MP,
    pys.STARTS,
    pys.MIN,
    pys.NINETIES,
    pys.GLS,
    pys.AST,
    pys.GLS_AST,
    pys.G_PK,
    pys.PK,
    pys.PKATT,
    pys.CRDY,
    pys.CRDR
FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[PLAYER_YEAR_STAT] pys
JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[PLAYER] p
    ON p.PLAYERID = pys.PLAYERID
JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] t
    ON t.TEAMID = pys.TEAMID
WHERE pys.NATION != 'USA'
ORDER BY pys.SEASON_YEAR DESC, pys.GLS_AST DESC, pys.STARTS DESC;
```

6.11.2. Result

PLAYER_FULLNAME	SEASON_YEAR	NATION	PLAYER_POS1	PLAYER_POS2	TEAM_NAME	AGE_GAME	MP	STARTS	MIN	NINETIES	GLS	AST	GLS_AST	G_PK	PK	PKATT	CRDY	CRDR
Hany Mukhtar	2022	GER	FW	MF	Nashville SC	26	33	32	2825	31.4	23	7	30	17	6	7	4	0
D◆◆niel Gazdag	2022	HUN	MF	NULL	Philadelphia	25	34	34	2929	32.5	22	6	28	15	7	7	7	0
Sebasti◆n Driussi	2022	ARG	MF	NULL	Austin	25	34	32	2876	32	22	5	27	18	4	4	6	0
…																		
Victor Mella	1996	CHI	FW	NULL	New England Revolution	21	3	1	157	1.7	0	0	0	0	0	0	0	0
Patrick Tardieu	1996	HAI	FW	NULL	New England Revolution	27	5	0	69	0.8	0	0	0	0	0	0	0	0
Bojan Vu◆◆kovi◆◆	1996	SRB	FW	NULL	New England Revolution	25	1	0	14	0.2	0	0	0	0	0	0	0	0

6.11.3. Pivot UI redevelopment



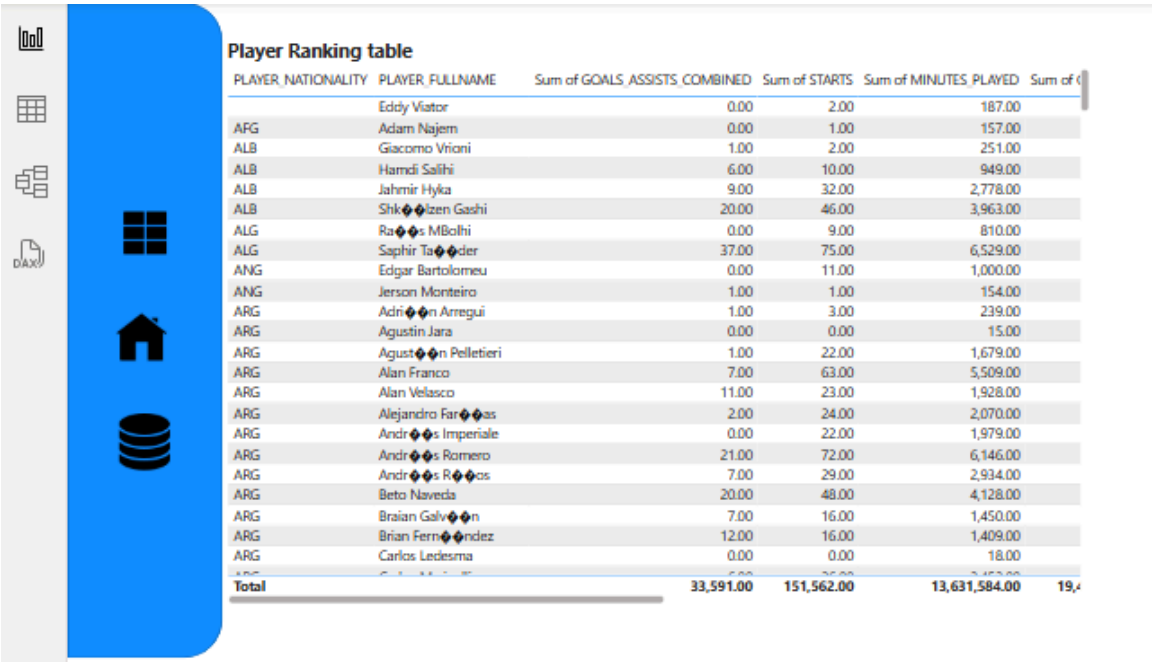
Home

PLAYER_NATIONALITY (Multiple Items)

Row Labels	Sum of GOALS_ASSISTS_COMBINED	Sum of STARTS	Sum of MINUTES_PLAYED	Sum of GOALS	Sum of ASSISTS	Sum of GOALS_NON_PENALT
Jaime Moreno	192	295	25794	133	59	
Kei Kamara	167	289	26500	125	42	
Dwayne De Rosario	161	274	25189	104	57	
Diego Valeri	151	230	20373	86	65	
Bradley Wright-Phillips	142	184	16425	117	25	
Diego Fag□□ndez	119	248	22089	66	53	
Robbie Keane	118	119	10803	83	35	
Javier Morales	118	225	20111	50	68	
Federico Higua□□n	113	191	17127	60	53	
Josef Mart□□nez	111	111	10158	98	13	
Ronald Cerritos	109	185	16555	71	38	
Sebastian Giovinco	108	111	9603	68	40	
Ra□□l D□□az Arce	107	142	12423	82	25	
Carlos Ru□□z	105	155	14034	89	16	

Q1.8 (2) | Q1.9 | Q1.10 | FAOES_Analysis | Player_Perf_Analysis | Q3.1 | Q3.2 | Q3.3 | Q3.4 | Q3.5 | Q3.6 | Q3.7 | Q3.8 | Q3.9 | Q3.11 | Q3.12

6.11.4. Power BI redevelopment



Player Ranking table

PLAYER_NATIONALITY	PLAYER_FULLNAME	Sum of GOALS_ASSISTS_COMBINED	Sum of STARTS	Sum of MINUTES_PLAYED	Sum of (
	Eddy Viator	0.00	2.00	187.00	
AFG	Adam Najem	0.00	1.00	157.00	
ALB	Giacomo Vrioni	1.00	2.00	251.00	
ALB	Hamdi Salihi	6.00	10.00	949.00	
ALB	Jahmir Hyka	9.00	32.00	2,778.00	
ALB	Shk►►lzen Gashi	20.00	46.00	3,963.00	
ALG	Ra►►s MBolhi	0.00	9.00	810.00	
ALG	Saphir Ta►►der	37.00	75.00	6,529.00	
ANG	Edgar Bartolomeu	0.00	11.00	1,000.00	
ANG	Jerson Monteiro	1.00	1.00	154.00	
ARG	Adri►►n Arregui	1.00	3.00	239.00	
ARG	Agustin Jara	0.00	0.00	15.00	
ARG	Agust►►n Pelletieri	1.00	22.00	1,679.00	
ARG	Alan Franco	7.00	63.00	5,509.00	
ARG	Alan Velasco	11.00	23.00	1,928.00	
ARG	Alejandro Far►►as	2.00	24.00	2,070.00	
ARG	Andr►►s Imperiale	0.00	22.00	1,979.00	
ARG	Andr►►s Romero	21.00	72.00	6,146.00	
ARG	Andr►►s R►►os	7.00	29.00	2,934.00	
ARG	Beto Naveda	20.00	48.00	4,128.00	
ARG	Braian Galv►►n	7.00	16.00	1,450.00	
ARG	Brian Fern►►ndez	12.00	16.00	1,409.00	
ARG	Carlos Ledesma	0.00	0.00	18.00	
Total		**33,591.00**	**151,562.00**	**13,631,584.00**	**19,**

6.11.5. Comparison

The linked server query retrieves non-US MLS player stats from a remote Oracle database via SQL Server, joining three tables with explicit VARCHAR casting and multi-column sorting, while the Sprint 4 query directly accesses local Oracle tables using TO_CHAR conversion without sorting. Both return identical player metrics, but the linked server version offers SQL Server integration with presentation-ready sorting, whereas the Sprint 4 query is simpler for pure Oracle environments where sorting can be applied separately. Choose based on your database environment and need for pre-sorted results.

6.12. Develop a report to show performance of drafted players for every year. Performance, for every player, in terms of starts, Min, GLS, AST, GLS_AST, G_PK, PK, PKATT, CRDY, and with their rank in their team as well rank in their assigned conference.

6.12.1. Query

```
WITH DraftedPlayerPerformance AS (
    SELECT
        pys.SEASON_YEAR,
        p.PLAYERID,
        p.PLAYER_FULLNAME,
        pys.TEAMID,
        t.HOME AS TEAM_NAME,
        t.CONFERENCE,
        pys.STARTS,
        pys.MIN,
        pys.GLS,
        pys.AST,
        pys.GLS_AST,
        pys.G_PK,
        pys.PK,
        pys.PKATT,
        pys.CRDY,
        pys.CRDR,
        RANK() OVER (
            PARTITION BY pys.SEASON_YEAR, pys.TEAMID
            ORDER BY pys.GLS_AST DESC, pys.STARTS DESC, pys.MIN DESC
```

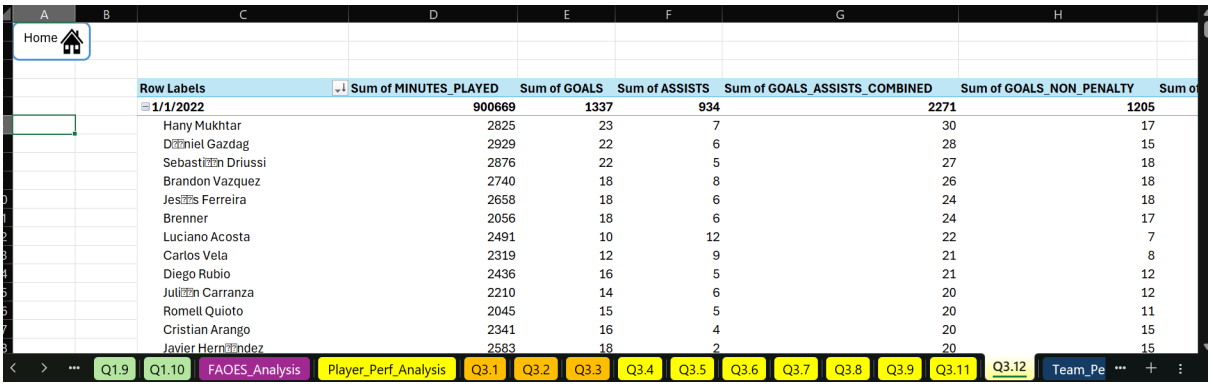
```
        ) AS Team_Rank,
        RANK() OVER (
            PARTITION BY pys.SEASON_YEAR, t.CONFERENCE
            ORDER BY pys.GLS_AST DESC, pys.STARTS DESC, pys.MIN DESC
        ) AS Conference_Rank
    FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[PLAYER_YEAR_STAT] pys
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[PLAYER] p
        ON pys.PLAYERID = p.PLAYERID
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] t
        ON pys.TEAMID = t.TEAMID
    WHERE pys.NATION = 'USA'
)
SELECT
    SEASON_YEAR,
    PLAYERID,
    PLAYER_FULLNAME,
    TEAMID,
    TEAM_NAME,
    CONFERENCE,
    STARTS,
    MIN,
    GLS,
    AST,
    GLS_AST,
    G_PK,
    PK,
    PKATT,
    CRDY,
    CRDR,
    Team_Rank,
    Conference_Rank
FROM DraftedPlayerPerformance
ORDER BY SEASON_YEAR DESC, CONFERENCE, Conference_Rank, Team_Rank;
```

6.12.2. Result

SEASON_YEAR	PLAYERID	PLAYER_FULLNAME	TEAMID	TEAM_NAME	CONFERENCE	STARTS	MIN	GLS	AST	GLS_AST	G_PK	PK	PKATT	CRDY	CRDR	Team_Rank	Conference_Rank
4/1/2022 0:00	p404	Brandon Vazquez	t12	FC Cincinnati	Eastern Conference	32	2740	18	8	26	18	0	0	1	0	1	1
4/1/2022 0:00	p1354	Djordje Mihailovic	t3	CF Montr►►al	Eastern Conference	25	1998	9	4	13	9	0	0	5	0	1	2
4/1/2022 0:00	p106	Alejandro Bedoya	t28	Philadelphia	Eastern Conference	27	2422	6	6	12	6	0	0	8	0	1	3
…																	
4/1/1996 0:00	p3546	Jimmy Glenn	t10	Dallas Burn	Western Conference	0	115	0	0	0	0	0	0	0	0	16	61

4/1/1996 0:00	p236	Andrew Shue	t17	LA Galaxy	Western Conference	0	96	0	0	0	0	0	0	0	0	15	62
4/1/1996 0:00	p1539	Ed Pinon	t7	Colorado Rapids	Western Conference	0	48	0	0	0	0	0	0	0	0	16	63

6.12.3. Pivot UI redevelopment



Row Labels	Sum of MINUTES_PLAYED	Sum of GOALS	Sum of ASSISTS	Sum of GOALS_ASSISTS_COMBINED	Sum of GOALS_NON_PENALTY	Sum of
1/1/2022	900669	1337	934	2271	1205	
Hany Mukhtar	2825	23	7	30	17	
D□□niel Gazdag	2929	22	6	28	15	
Sebast□□n Driussi	2876	22	5	27	18	
Brandon Vazquez	2740	18	8	26	18	
Jes□□s Ferreira	2658	18	6	24	18	
Brenner	2056	18	6	24	17	
Luciano Acosta	2491	10	12	22	7	
Carlos Vela	2319	12	9	21	8	
Diego Rubio	2436	16	5	21	12	
Juli□□n Carranza	2210	14	6	20	12	
Romell Quioto	2045	15	5	20	11	
Cristian Arango	2341	16	4	20	15	
Javier Hern□□ndez	2583	18	2	20	15	

Q1.9 Q1.10 FAOES_Analysis Player_Perf_Analysis Q3.1 Q3.2 Q3.3 Q3.4 Q3.5 Q3.6 Q3.7 Q3.8 Q3.9 Q3.11 Q3.12 Team_Pe

6.12.4. Power BI redevelopment

Player Ranking table

Year	PLAYER_FULLNAME	Sum of MINUTES_PLAYED	Sum of GOALS	Sum of ASSISTS	Sum of GOALS_ASSISTS_COMBINED	Sum of GC
1996	A.J. Wood	1,136.00	5.00	2.00	7.00	
1996	Adam Eyre	725.00	0.00	0.00	0.00	
1996	Adri□□n Paz	1,802.00	6.00	8.00	14.00	
1996	Aidan Heaney	1,534.00	0.00	0.00	0.00	
1996	Alan Prampin	337.00	3.00	0.00	3.00	
1996	Alexi Lalas	2,218.00	1.00	1.00	2.00	
1996	Altimont Butler	411.00	2.00	0.00	2.00	
1996	Andrew Restrepo	343.00	0.00	0.00	0.00	
1996	Andrew Shue	96.00	0.00	0.00	0.00	
1996	Ante Razov	172.00	1.00	0.00	1.00	
1996	Antony de □□vila	650.00	6.00	2.00	8.00	
1996	Arash Noamouz	852.00	2.00	1.00	3.00	
1996	Ben Crawley	82.00	0.00	0.00	0.00	
1996	Benedict Iroha	2,194.00	2.00	7.00	9.00	
1996	Berthy Su□□rez	160.00	0.00	0.00	0.00	
1996	Beto Naveda	1,734.00	6.00	3.00	9.00	
1996	Billy Thompson	2,074.00	2.00	7.00	9.00	
1996	Bo Oshoniyi	1,170.00	0.00	0.00	0.00	
1996	Bojan Vu□□kovi□□	14.00	0.00	0.00	0.00	
1996	Brad Friedel	810.00	0.00	0.00	0.00	
1996	Brad Wilson	969.00	1.00	0.00	1.00	
1996	Brandon Pollard	820.00	0.00	0.00	0.00	
1996	Brandon Ward	304.00	0.00	0.00	0.00	
Total		13,631,584.00	19,423.00	14,168.00	33,591.00	

6.12.5. Comparison

Both queries analyze US player performance using similar metrics (goals, assists, playing time) and ranking logic (team and conference rankings), but differ in their data sources and output: The Sprint 4 query uses local Oracle tables and focuses on presentation-ready results (excluding player/team IDs), while the linked server version pulls from remote tables via SQL Server and includes all raw identifiers (PLAYERID, TEAMID) for technical integration. Both employ window functions for dual-ranking players within their teams and conferences, but the linked server query partitions rankings by season first, making its rankings more precise for historical comparisons. The output sorting differs slightly - Sprint 4 prioritizes conference rank for scouting purposes, while the linked server version groups by conference for administrative analysis. Choose the Sprint 4 query for

cleaner reports, or the linked server version when working with the raw data in a SQL Server environment.

7. Qlist3 - MLS2: Team Performance Analysis

7.1. Ranking team stat based on all game (Home and away) within team conference. This should be done for every year as well as all years.

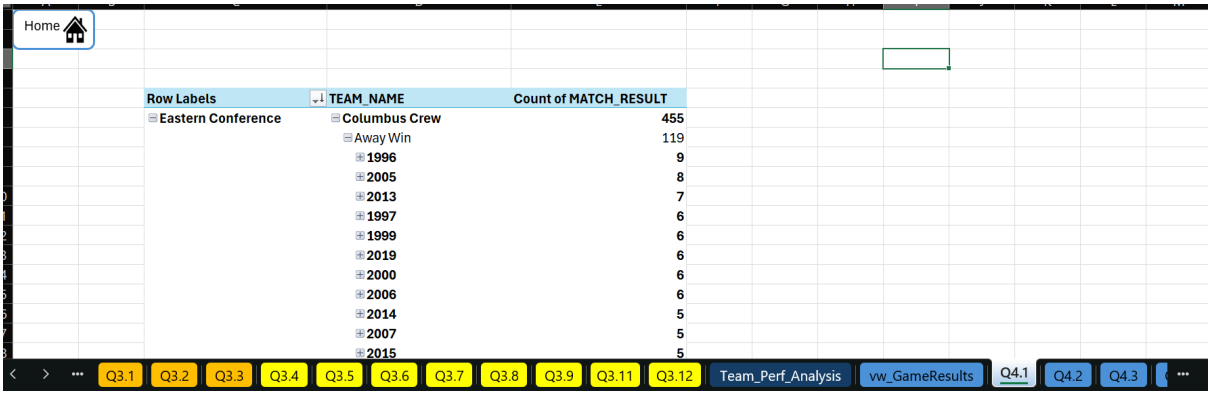
7.1.1. Query

```
WITH Team_Attendance AS (
    SELECT
        g.GAME_YEAR,
        t.teamid AS Team_ID,
        CAST(t.Conference AS VARCHAR(50)) AS Conference,
        SUM(ISNULL(g.ARRENDENCE, 0)) AS Total_Attendance  -- Sum attendance instead of
score (use ISNULL for SQL Server)
    FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[GAME] g
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] t
        ON g.HOME_TID = t.teamid OR g.AWAY_TID = t.teamid  -- Include both home and away
games
    GROUP BY g.GAME_YEAR, t.teamid, t.Conference
),
Ranked_Teams AS (
    SELECT
        GAME_YEAR,
        Team_ID,
        Conference,
        Total_Attendance,
        RANK() OVER (PARTITION BY GAME_YEAR, Conference ORDER BY Total_Attendance DESC) AS
Rank_In_Year,
        RANK() OVER (PARTITION BY Conference ORDER BY Total_Attendance DESC) AS
Rank_All_Years
    FROM Team_Attendance
)
SELECT *
FROM Ranked_Teams
ORDER BY Conference, GAME_YEAR, Rank_In_Year;
```

7.1.2. Result

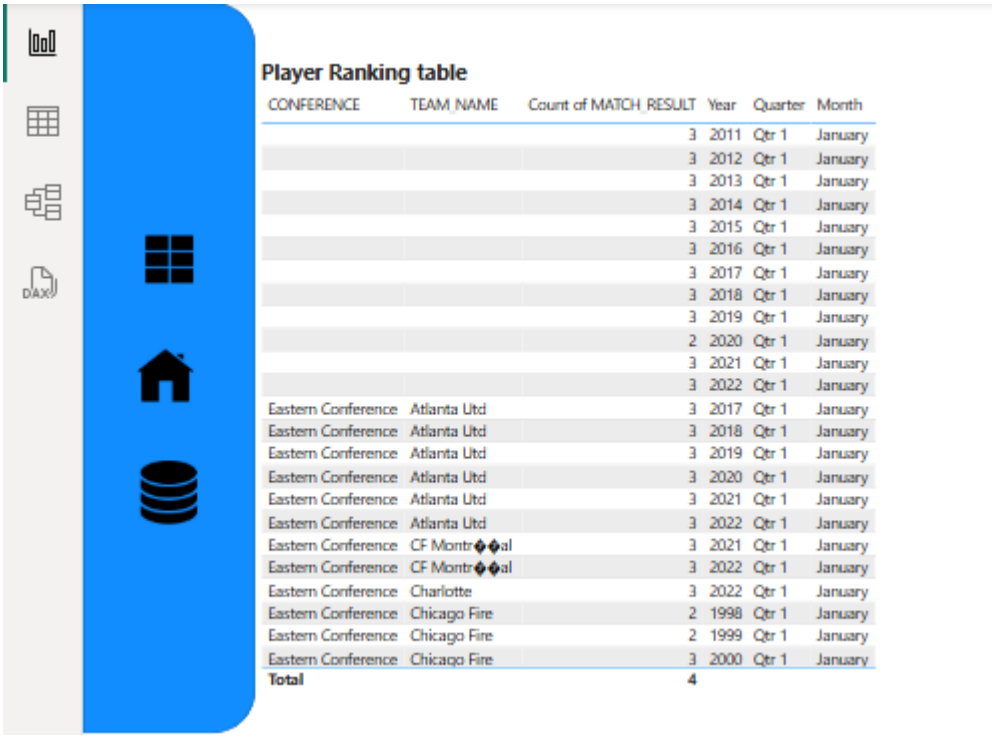
GAME_YEAR	Team_ID	Conference	Total_Attendance	Rank_In_Year	Rank_All_Years
4/1/1996 0:00	t19	Eastern Conference	734593	1	33
4/1/1996 0:00	t9	Eastern Conference	653454	2	59
…	…	…	…	…	…
4/1/2022 0:00	t13	Western Conference	648498	9	59
4/1/2022 0:00	t36	Western Conference	628316	10	66
4/1/2022 0:00	t7	Western Conference	576067	11	76

7.1.3. Pivot UI redevelopment



Row Labels	TEAM_NAME	Count of MATCH_RESULT
Eastern Conference	Columbus Crew	455
	Away Win	119
	1996	9
	2005	8
	2013	7
	1997	6
	1999	6
	2019	6
	2000	6
	2006	6
	2014	5
	2007	5
	2015	5

7.1.4. Power BI redevelopment



Player Ranking table

CONFERENCE	TEAM_NAME	Count of MATCH_RESULT	Year	Quarter	Month
		3	2011	Qtr 1	January
		3	2012	Qtr 1	January
		3	2013	Qtr 1	January
		3	2014	Qtr 1	January
		3	2015	Qtr 1	January
		3	2016	Qtr 1	January
		3	2017	Qtr 1	January
		3	2018	Qtr 1	January
		3	2019	Qtr 1	January
		2	2020	Qtr 1	January
		3	2021	Qtr 1	January
		3	2022	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2017	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2018	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2019	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2020	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2021	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2022	Qtr 1	January
Eastern Conference	CF Montr**al	3	2021	Qtr 1	January
Eastern Conference	CF Montr**al	3	2022	Qtr 1	January
Eastern Conference	Charlotte	3	2022	Qtr 1	January
Eastern Conference	Chicago Fire	2	1998	Qtr 1	January
Eastern Conference	Chicago Fire	2	1999	Qtr 1	January
Eastern Conference	Chicago Fire	3	2000	Qtr 1	January
Total		**4**			

7.1.5. Comparison

The Sprint 4 query directly analyzes Oracle data using native functions (NVL, VARCHAR2) to calculate and rank team attendance by conference and year, while the linked server version accesses the same Oracle tables through SQL Server (using ISNULL/VARCHAR) with identical ranking logic but added network overhead. Both queries generate the same analytical output - annual and all-time conference rankings sorted identically - but the Sprint 4 version is more efficient for pure Oracle environments due to its native optimization, whereas the linked server approach enables cross-platform access when needed. The key difference lies in their execution environment and syntax, not in their analytical results or business logic.

7.2. Ranking team stat based on all home game within team conference. This should be done for every year as well as all years.

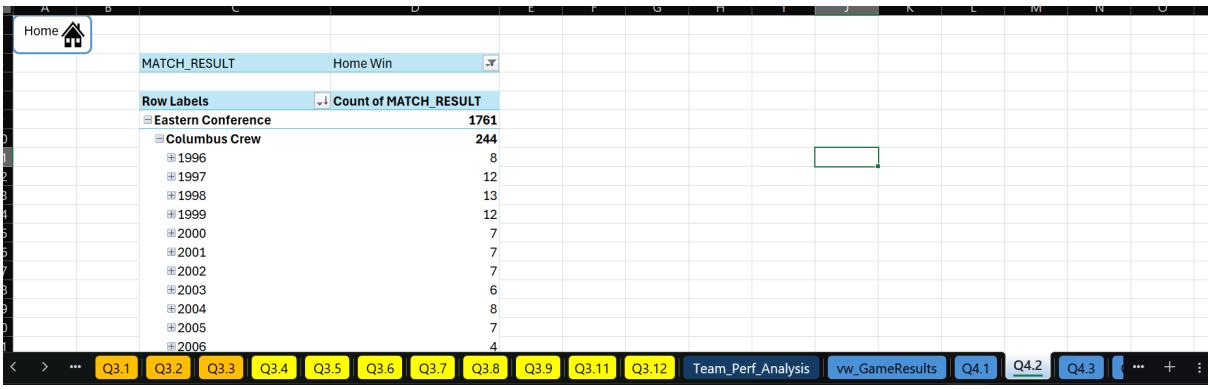
7.2.1. Query

```
WITH Team_Attendance AS (
    SELECT
        g.GAME_YEAR,
        t.home AS Team,
        CAST(t.Conference AS VARCHAR(50)) AS Conference,
        SUM(ISNULL(g.ARRENDENCE, 0)) AS Total_Attendance  -- Sum attendance instead of
score (use ISNULL for SQL Server)
    FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[GAME] g
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] t
        ON g.HOME_TID = t.teamid -- Only focus on home games
    WHERE g.HOME_TID = t.teamid -- Focus on home games for attendance
    GROUP BY g.GAME_YEAR, t.home, t.Conference
),
Ranked_Teams AS (
    SELECT
        GAME_YEAR,
        Team,
        Conference,
        Total_Attendance,
        RANK() OVER (PARTITION BY GAME_YEAR, Conference ORDER BY Total_Attendance DESC) AS
Rank_In_Year,
        RANK() OVER (PARTITION BY Conference ORDER BY Total_Attendance DESC) AS
Rank_All_Years
    FROM Team_Attendance
)
SELECT *
FROM Ranked_Teams
ORDER BY Conference, GAME_YEAR, Rank_In_Year;
```

7.2.2. Result

GAME_YEAR	Team	Conference	Total_Attendance	Rank_In_Year	Rank_All_Years
4/1/1996 0:00	MetroStars	Eastern Conference	396776	1	23
4/1/1996 0:00	Columbus Crew	Eastern Conference	324009	2	46
…	…	…	…	…	…
4/1/2022 0:00	FC Dallas	Western Conference	299061	9	65
4/1/2022 0:00	Vancouver Whitecaps FC	Western Conference	278790	10	67
4/1/2022 0:00	Colorado Rapids	Western Conference	246049	11	80
4/1/2022 0:00	FC Dallas	Western Conference	299061	9	65

7.2.3. Pivot UI redevelopment



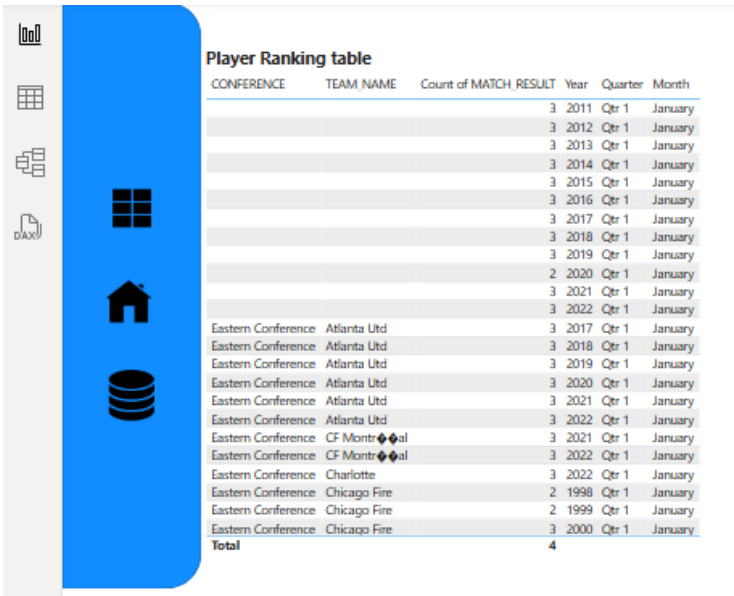
Home

MATCH_RESULT	Home Win

Row Labels	Count of MATCH_RESULT
Eastern Conference	1761
Columbus Crew	244
1996	8
1997	12
1998	13
1999	12
2000	7
2001	7
2002	7
2003	6
2004	8
2005	7
2006	4

Q3.1 Q3.2 Q3.3 Q3.4 Q3.5 Q3.6 Q3.7 Q3.8 Q3.9 Q3.11 Q3.12 Team_Perf_Analysis vw_GameResults Q4.1 Q4.2 Q4.3

7.2.4. Power BI redevelopment



Player Ranking table

CONFERENCE	TEAM_NAME	Count of MATCH_RESULT	Year	Quarter	Month
		3	2011	Qtr 1	January
		3	2012	Qtr 1	January
		3	2013	Qtr 1	January
		3	2014	Qtr 1	January
		3	2015	Qtr 1	January
		3	2016	Qtr 1	January
		3	2017	Qtr 1	January
		3	2018	Qtr 1	January
		3	2019	Qtr 1	January
		2	2020	Qtr 1	January
		3	2021	Qtr 1	January
		3	2022	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2017	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2018	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2019	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2020	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2021	Qtr 1	January
Eastern Conference	Atlanta Utd	3	2022	Qtr 1	January
Eastern Conference	CF Montr►►al	3	2021	Qtr 1	January
Eastern Conference	CF Montr►►al	3	2022	Qtr 1	January
Eastern Conference	Charlotte	3	2022	Qtr 1	January
Eastern Conference	Chicago Fire	2	1998	Qtr 1	January
Eastern Conference	Chicago Fire	2	1999	Qtr 1	January
Eastern Conference	Chicago Fire	3	2000	Qtr 1	January
Total		**4**			

7.2.5. Comparison

Both queries analyze home game attendance data with identical structure and logic, using two CTEs to calculate total attendance and rank teams both annually and historically within conferences. The key difference is the data source: the linked server query accesses remote Oracle tables through SQL Server (using ISNULL for null handling and VARCHAR for typing), while the Sprint 4 query works directly with native Oracle tables (using NVL and VARCHAR2). Both focus exclusively on home games (via JOIN and WHERE conditions) and produce the same sorted output. The linked server version introduces slight network overhead but enables cross-platform access, whereas the Sprint 4 query is more efficient for pure Oracle environments. Essentially, they represent the same analytical approach implemented in different technical contexts.

7.3. Ranking team stat based on all away game within team conference. This should be done for every year as well as all years.

7.3.1. Query

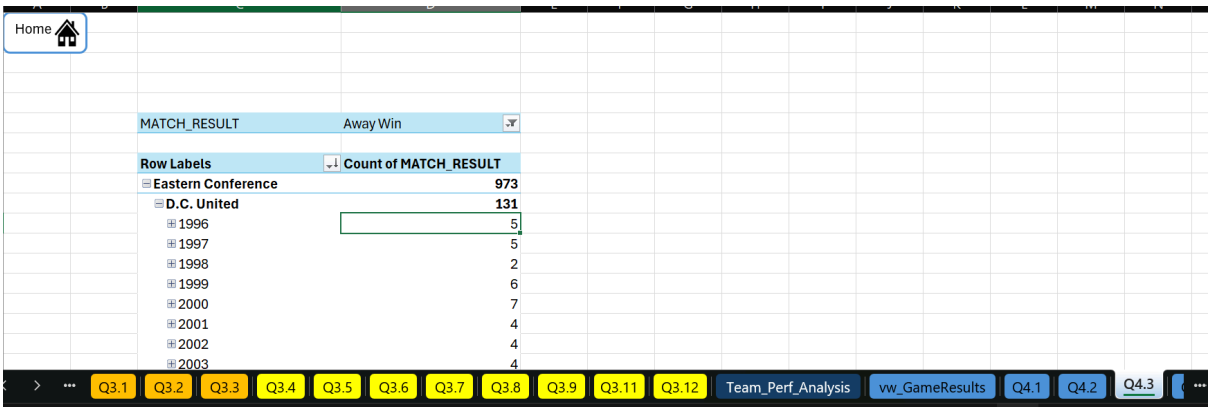
```
WITH Team_Attendance AS (
    SELECT
        g.GAME_YEAR,
```

```
        t.home AS Team,
        CAST(t.Conference AS VARCHAR(50)) AS Conference,  -- Ensure Conference is string
        SUM(ISNULL(g.ARRENDENCE, 0)) AS Total_Attendance  -- Sum attendance instead of
score
    FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[GAME] g
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] t
        ON g.AWAY_TID = t.teamid -- Only focus on away games
    WHERE g.AWAY_TID = t.teamid -- Focus on away games for attendance
    GROUP BY g.GAME_YEAR, t.home, t.Conference
),
Ranked_Teams AS (
    SELECT
        GAME_YEAR,
        Team,
        Conference,
        Total_Attendance,
        RANK() OVER (PARTITION BY GAME_YEAR, Conference ORDER BY Total_Attendance DESC) AS
Rank_In_Year,
        RANK() OVER (PARTITION BY Conference ORDER BY Total_Attendance DESC) AS
Rank_All_Years
    FROM Team_Attendance
)
SELECT *
FROM Ranked_Teams
ORDER BY Conference, GAME_YEAR, Rank_In_Year;
```

7.3.2. Result

GAME_YEAR	Team	Conference	Total_Attendance	Rank_In_Year	Rank_All_Years
4/1/1996 0:00	Tampa Bay Mutiny	Eastern Conference	375642	1	40
4/1/1996 0:00	D.C. United	Eastern Conference	343824	2	64
4/1/1996 0:00	MetroStars	Eastern Conference	337817	3	67
…	…	…	…	…	…
4/1/2022 0:00	FC Dallas	Western Conference	349437	9	53
4/1/2022 0:00	Minnesota Utd	Western Conference	344013	10	57
4/1/2022 0:00	Colorado Rapids	Western Conference	330018	11	66

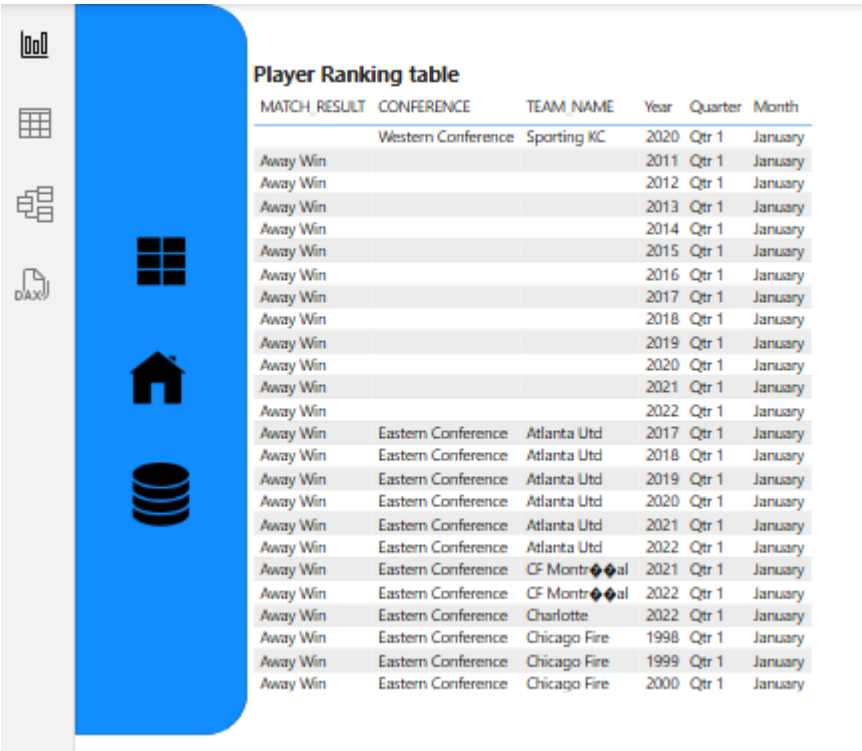
7.3.3. Pivot UI redevelopment



MATCH_RESULT	Away Win
Row Labels	**Count of MATCH_RESULT**
Eastern Conference	**973**
D.C. United	**131**
1996	5
1997	5
1998	2
1999	6
2000	7
2001	4
2002	4
2003	4

Home Q3.1 Q3.2 Q3.3 Q3.4 Q3.5 Q3.6 Q3.7 Q3.8 Q3.9 Q3.11 Q3.12 Team_Perf_Analysis vw_GameResults Q4.1 Q4.2 Q4.3

7.3.4. Power BI redevelopment



Player Ranking table

MATCH_RESULT	CONFERENCE	TEAM_NAME	Year	Quarter	Month
	Western Conference	Sporting KC	2020	Qtr 1	January
Away Win			2011	Qtr 1	January
Away Win			2012	Qtr 1	January
Away Win			2013	Qtr 1	January
Away Win			2014	Qtr 1	January
Away Win			2015	Qtr 1	January
Away Win			2016	Qtr 1	January
Away Win			2017	Qtr 1	January
Away Win			2018	Qtr 1	January
Away Win			2019	Qtr 1	January
Away Win			2020	Qtr 1	January
Away Win			2021	Qtr 1	January
Away Win			2022	Qtr 1	January
Away Win	Eastern Conference	Atlanta Utd	2017	Qtr 1	January
Away Win	Eastern Conference	Atlanta Utd	2018	Qtr 1	January
Away Win	Eastern Conference	Atlanta Utd	2019	Qtr 1	January
Away Win	Eastern Conference	Atlanta Utd	2020	Qtr 1	January
Away Win	Eastern Conference	Atlanta Utd	2021	Qtr 1	January
Away Win	Eastern Conference	Atlanta Utd	2022	Qtr 1	January
Away Win	Eastern Conference	CF Montr►►al	2021	Qtr 1	January
Away Win	Eastern Conference	CF Montr►►al	2022	Qtr 1	January
Away Win	Eastern Conference	Charlotte	2022	Qtr 1	January
Away Win	Eastern Conference	Chicago Fire	1998	Qtr 1	January
Away Win	Eastern Conference	Chicago Fire	1999	Qtr 1	January
Away Win	Eastern Conference	Chicago Fire	2000	Qtr 1	January

7.3.5. Comparison

In Sprint 4, the query analyzes team attendance by focusing on away games, summing up attendance figures per team, year, and conference. It uses NVL for handling null values and casts the Conference field to ensure consistent data type treatment. The ranking is done within each year and across all years by conference, providing a layered view of team performance in terms of audience draw. The result is clean, with dates displayed in a simple 1-Jan-YY format, emphasizing historical comparison year by year.

Sprint 6 mirrors this logic but adapts it to a SQL Server environment, using ISNULL instead of NVL and fully qualifying table paths for cross-server execution. The core ranking structure remains intact, but Sprint 6 extends the dataset through 2022, giving it a more up-to-date and comprehensive reach. Notably, the result format shifts to a timestamp-style date (4/1/1996 0:00), reflecting SQL Server's default datetime representation. This version is better suited for enterprise-level systems that manage federated data sources, offering scalability while maintaining the original analytical intent.

7.4. Analysis of team performance by comparing home game and away games.

7.4.1. Query

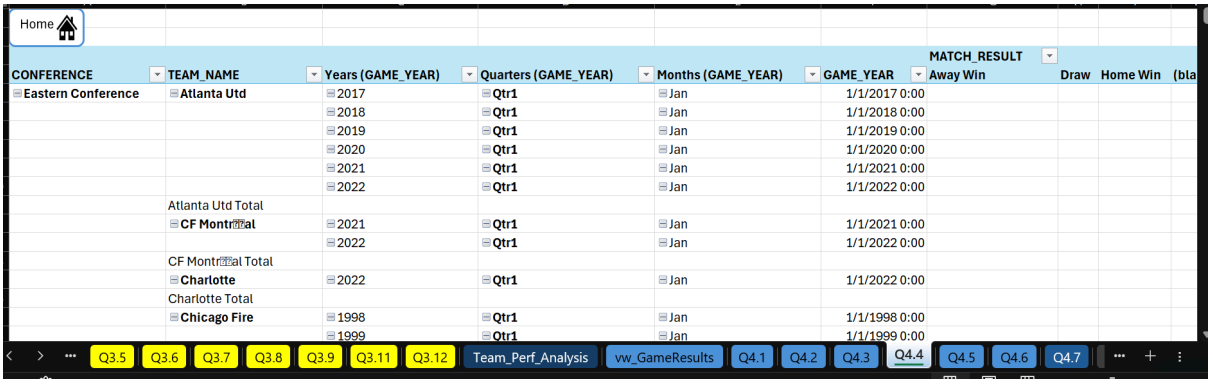
```
WITH Home_Attendance AS (
    SELECT
        g.GAME_YEAR,
        t.home AS Team,
        CAST(t.Conference AS VARCHAR(50)) AS Conference,
        SUM(ISNULL(g.ARRENDENCE, 0)) AS Total_Home_Attendance
    FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[GAME] g
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] t
        ON g.HOME_TID = t.teamid
    GROUP BY g.GAME_YEAR, t.home, t.Conference
),
Away_Attendance AS (
    SELECT
        g.GAME_YEAR,
        t.home AS Team,
        CAST(t.Conference AS VARCHAR(50)) AS Conference,
        SUM(ISNULL(g.ARRENDENCE, 0)) AS Total_Away_Attendance
    FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[GAME] g
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] t
        ON g.AWAY_TID = t.teamid
    GROUP BY g.GAME_YEAR, t.home, t.Conference
),
Combined_Attendance AS (
    SELECT
        ha.GAME_YEAR,
        ha.Team,
        ha.Conference,
        ha.Total_Home_Attendance,
        aa.Total_Away_Attendance,
        CASE
            WHEN aa.Total_Away_Attendance = 0 THEN NULL
            ELSE ROUND(ha.Total_Home_Attendance * 1.0 / aa.Total_Away_Attendance, 2)
        END AS Home_Away_Ratio
    FROM Home_Attendance ha
    JOIN Away_Attendance aa
        ON ha.Team = aa.Team AND ha.GAME_YEAR = aa.GAME_YEAR AND ha.Conference =
aa.Conference
),
Teams_Rank AS (
    SELECT
        GAME_YEAR,
        Team,
        Conference,
        Home_Away_Ratio,
        RANK() OVER (PARTITION BY GAME_YEAR, Conference ORDER BY Home_Away_Ratio DESC) AS
Rank_In_Year,
        RANK() OVER (PARTITION BY Conference ORDER BY Home_Away_Ratio DESC) AS
Rank_All_Years
    FROM Combined_Attendance
)
SELECT *
FROM Teams_Rank
ORDER BY Conference, GAME_YEAR, Rank_In_Year;
```

7.4.2. Result

GAME_YEAR	Team	Conference	Home_Away_Ratio	Rank_In_Year	Rank_All_Years
4/1/1996 0:00	MetroStars	Eastern Conference	1.17	1	38

4/1/1996 0:00	Columbus Crew	Eastern Conference	1.16	2	39
4/1/1996 0:00	New England Revolution	Eastern Conference	0.94	3	76
…	…	…	…	…	…
4/1/2022 0:00	LA Galaxy	Western Conference	0.85	8	99
4/1/2022 0:00	Sporting KC	Western Conference	0.84	9	101
4/1/2022 0:00	Vancouver Whitecaps FC	Western Conference	0.8	10	107
4/1/2022 0:00	Colorado Rapids	Western Conference	0.75	11	116

7.4.3. Pivot UI redevelopment



7.4.4. Power BI redevelopment

Player Ranking table

CONFERENCE	TEAM_NAME	Year	Quarter	Month
		2011	Qtr 1	January
		2012	Qtr 1	January
		2013	Qtr 1	January
		2014	Qtr 1	January
		2015	Qtr 1	January
		2016	Qtr 1	January
		2017	Qtr 1	January
		2018	Qtr 1	January
		2019	Qtr 1	January
		2020	Qtr 1	January
		2021	Qtr 1	January
		2022	Qtr 1	January
Eastern Conference	Atlanta Utd	2017	Qtr 1	January
Eastern Conference	Atlanta Utd	2018	Qtr 1	January
Eastern Conference	Atlanta Utd	2019	Qtr 1	January
Eastern Conference	Atlanta Utd	2020	Qtr 1	January
Eastern Conference	Atlanta Utd	2021	Qtr 1	January
Eastern Conference	Atlanta Utd	2022	Qtr 1	January
Eastern Conference	CF Montr⁄⁄al	2021	Qtr 1	January
Eastern Conference	CF Montr⁄⁄al	2022	Qtr 1	January
Eastern Conference	Charlotte	2022	Qtr 1	January
Eastern Conference	Chicago Fire	1998	Qtr 1	January
Eastern Conference	Chicago Fire	1999	Qtr 1	January
Eastern Conference	Chicago Fire	2000	Qtr 1	January
Eastern Conference	Chicago Fire	2001	Qtr 1	January

7.4.5. Comparison

The Sprint 4 and Sprint 6 queries both aim to analyze and rank Major League Soccer teams based on their home and away game attendance ratios. Structurally, the logic in both queries is consistent: they aggregate total home and away attendance per team per year, calculate a home-to-away attendance ratio, and rank teams both within each season and across all seasons in their conference. The primary difference lies in their execution environments. Sprint 4 is written for Oracle, using NVL for null handling and VARCHAR2 for casting, while Sprint 6 is adapted for SQL Server, using ISNULL and VARCHAR. Additionally, Sprint 6 queries fully qualified table names from a remote database, indicating a shift toward a distributed environment. In terms of output, Oracle (Sprint 4) presents dates in a concise 1-Jan-96 format, whereas SQL Server (Sprint 6) displays them as 4/1/1996 0:00, reflecting default date-time handling differences. Furthermore, Sprint 6 covers a broader dataset, extending through the 2022 season, resulting in more comprehensive rankings and slightly different Rank_All_Years values due to the inclusion of more recent data. Despite these technical and scope variations, the analytical goal and ranking methodology remain aligned across both versions.

7.5. Ranking team yearly performance based on percentage of participating nationalities within team conference as well as all conferences.

7.5.1. Query

```
WITH TeamNatCounts AS (
    SELECT
        PYS.TEAMID,
        T.CONFERENCE,
        PYS.SEASON_YEAR,
        COUNT(DISTINCT PYS.NATION) AS NAT_COUNT
    FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[PLAYER_YEAR_STAT] PYS
    JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] T
        ON PYS.TEAMID = T.TEAMID
    GROUP BY PYS.TEAMID, T.CONFERENCE, PYS.SEASON_YEAR
),
ConfTotals AS (
    SELECT
        TN.CONFERENCE,
        TN.SEASON_YEAR,
        SUM(TN.NAT_COUNT) AS CONF_NAT_TOTAL
```

```
    FROM TeamNatCounts TN
    GROUP BY TN.CONFERENCE, TN.SEASON_YEAR
),
AllConfTotal AS (
    SELECT
        TN.SEASON_YEAR,
        SUM(TN.NAT_COUNT) AS ALL_NAT_TOTAL
    FROM TeamNatCounts TN
    GROUP BY TN.SEASON_YEAR
)
SELECT
    TN.TEAMID,
    TN.CONFERENCE,
    TN.SEASON_YEAR,
    TN.NAT_COUNT,
    ROUND((TN.NAT_COUNT * 100.0 / CT.CONF_NAT_TOTAL), 3) AS PCT_CONF,
    ROUND((TN.NAT_COUNT * 100.0 / ACT.ALL_NAT_TOTAL), 3) AS PCT_ALL,
    RANK() OVER (PARTITION BY TN.SEASON_YEAR ORDER BY (TN.NAT_COUNT * 100.0 /
ACT.ALL_NAT_TOTAL) DESC) AS OVERALL_RANK,
    RANK() OVER (PARTITION BY TN.SEASON_YEAR, TN.CONFERENCE ORDER BY (TN.NAT_COUNT * 100.0
/ CT.CONF_NAT_TOTAL) DESC) AS CONF_RANK
FROM TeamNatCounts TN
JOIN ConfTotals CT
    ON TN.CONFERENCE = CT.CONFERENCE AND TN.SEASON_YEAR = CT.SEASON_YEAR
JOIN AllConfTotal ACT
    ON TN.SEASON_YEAR = ACT.SEASON_YEAR
ORDER BY TN.SEASON_YEAR, OVERALL_RANK;
```

7.5.2. Result

TEAMID	CONFERENCE	SEASON_YEAR	NAT_COUNT	PCT_CONF	PCT_ALL	OVERALL_RANK	CONF_RANK
t24	Eastern Conference	4/1/1996 0:00	12	29.268	18.182	1	1
t19	Eastern Conference	4/1/1996 0:00	9	21.951	13.636	2	2
t8	Eastern Conference	4/1/1996 0:00	8	19.512	12.121	3	3
…	…	…	...	…	…	…	…
t26	Eastern Conference	4/1/2022 0:00	9	4.663	2.616	22	12
t23	Eastern Conference	4/1/2022 0:00	8	4.145	2.326	25	14
t35	Eastern Conference	4/1/2022 0:00	6	3.109	1.744	26	15

7.5.3. Pivot UI redevelopment

Home

Years (SEASC	TEAMID	CONFERENCE	Values Sum of UNIQUE_NATIONALITIES	Sum of TOTAL_PLAYERS	Average of NATIONALITY_PERCENTAGE	Average of CONFERENCE_RANK	Average
1996	t10	Western Conference	7	22	31.82	2	
	t15	Western Conference	6	21	28.57	3	
	t17	Western Conference	7	21	33.33	1	
	t19	Eastern Conference	9	35	25.71	4	
	t24	Eastern Conference	12	27	44.44	1	
	t31	Western Conference	6	22	27.27	4	
	t34	Eastern Conference	7	20	35	2	
	t7	Western Conference	5	22	22.73	5	
	t8	Eastern Conference	8	28	28.57	3	
	t9	Eastern Conference	5	27	18.52	5	
1997	t10	Western Conference	7	25	28	2	
	t16	Western Conference	5	22	22.73	4	
	t17	Western Conference	9	27	33.33	1	

Q3.5 Q3.6 Q3.7 Q3.8 Q3.9 Q3.11 Q3.12 Team_Perf_Analysis vw_GameResults Q4.1 Q4.2 Q4.3 Q4.4 Q4.5 Q4.6 Q4.7

Accessibility: Investigate 100%

7.5.4. Power BI redevelopment

Player Ranking table

Year	TEAM ID	CONFERENCE
1996	t10	Western Conference
1996	t15	Western Conference
1996	t17	Western Conference
1996	t19	Eastern Conference
1996	t24	Eastern Conference
1996	t31	Western Conference
1996	t34	Eastern Conference
1996	t7	Western Conference
1996	t8	Eastern Conference
1996	t9	Eastern Conference
1997	t10	Western Conference
1997	t16	Western Conference
1997	t17	Western Conference
1997	t19	Eastern Conference
1997	t24	Eastern Conference
1997	t31	Western Conference
1997	t34	Eastern Conference
1997	t7	Western Conference
1997	t8	Eastern Conference
1997	t9	Eastern Conference
1998	t10	Western Conference
1998	t16	Western Conference
1998	t17	Western Conference
1998	t19	Eastern Conference
1998	t20	Eastern Conference

7.5.5. Comparison

In Sprint 4, the query ranks teams based on the percentage of participating nationalities within their team conference and across all conferences, focusing on nationality diversity using player_year_stat and team tables. It aggregates nationality counts per team, conference, and overall, then ranks them accordingly. In Sprint 6, the query also involves team performance but extends beyond nationality diversity to include player statistics such as goals (GLS), assists (AST), and other performance metrics, ranking teams based on the percentage of nationalities within their conference and all conferences. While both queries use similar tables, Sprint 6 incorporates more detailed performance measures, while Sprint 4 focuses solely on nationality counts and their representation.

7.6. Ranking player yearly performance based on total salary paid within team conference as well as all conferences.

No salary data available.

7.7. Ranking player performance based on range of ages within team conference as well as all conferences. Develop a report to show performance of top 10 teams how every team based on total wage budge for every year. Performance in terms of wins. Losses, draw, total point (win=3, draw=1. Loss=0) with their rank in their assigned conference.

7.7.1. Query

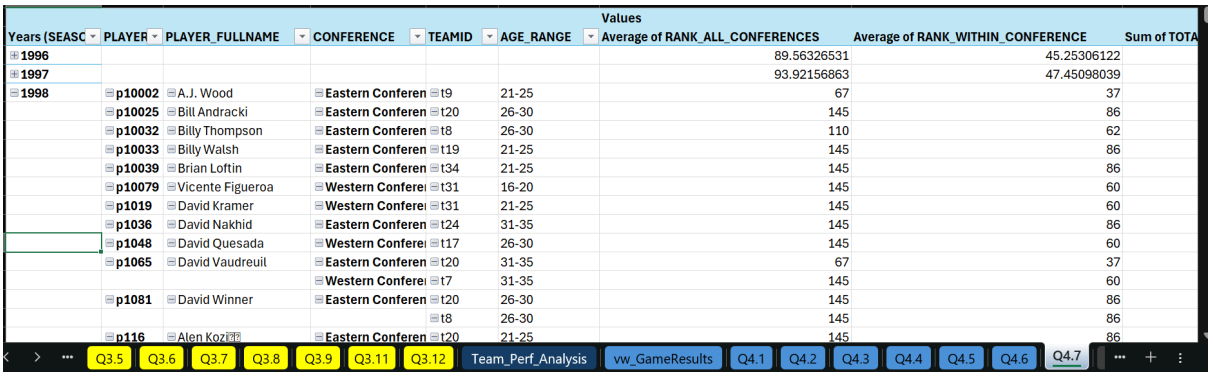
```
SELECT
    pys.SEASON_YEAR,
    pys.PLAYERID,
    p.PLAYER_FULLNAME,
    t.CONFERENCE,
    pys.TEAMID,
    CASE
        WHEN pys.AGE_GAME BETWEEN 16 AND 20 THEN '16-20'
        WHEN pys.AGE_GAME BETWEEN 21 AND 25 THEN '21-25'
        WHEN pys.AGE_GAME BETWEEN 26 AND 30 THEN '26-30'
        WHEN pys.AGE_GAME BETWEEN 31 AND 35 THEN '31-35'
        ELSE '36+'
    END AS age_range,
    SUM(pys.GLS) AS total_goals,
    SUM(pys.AST) AS total_assists,
    SUM(pys.MIN) AS total_minutes,
    RANK() OVER (PARTITION BY pys.SEASON_YEAR, t.CONFERENCE ORDER BY SUM(pys.GLS) DESC) AS
rank_within_conference,
    RANK() OVER (PARTITION BY pys.SEASON_YEAR ORDER BY SUM(pys.GLS) DESC) AS
rank_all_conferences
FROM [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[PLAYER_YEAR_STAT] pys
JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[PLAYER] p ON pys.PLAYERID =
p.PLAYERID
JOIN [CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2].[TEAM] t ON pys.TEAMID = t.TEAMID
GROUP BY
    pys.SEASON_YEAR, pys.PLAYERID, p.PLAYER_FULLNAME, t.CONFERENCE, pys.TEAMID,
    CASE
        WHEN pys.AGE_GAME BETWEEN 16 AND 20 THEN '16-20'
        WHEN pys.AGE_GAME BETWEEN 21 AND 25 THEN '21-25'
        WHEN pys.AGE_GAME BETWEEN 26 AND 30 THEN '26-30'
        WHEN pys.AGE_GAME BETWEEN 31 AND 35 THEN '31-35'
        ELSE '36+'
    END
```

7.7.2. Result

SEASON_YEAR	PLAYERID	PLAYER_FULLNAME	CONFERENCE	TEAMID	age_range	total_goals	total_assists	total_minutes	rank_within_conference	rank_all_conferences
4/1/1996 0:00	p7763	Roy Lassiter	Eastern Conference	t34	26-30	27	4	2580	1	1
4/1/1996 0:00	p7324	Ra◆◆l D◆◆az Arce	Eastern Conference	t9	21-25	23	2	2351	2	2
4/1/1996 0:00	p1640	Eduardo Hurtado	Western Conference	t17	26-30	21	6	2323	1	3
…	…	…	…	…	…	…	…	…	…	…

4/1/2022 0:00	p1596	Eddie Segura	Western Conference	t18	21-25	0	0	828	151	352
4/1/2022 0:00	p3943	Jonathan Perez	Western Conference	t17	16-20	0	0	8	151	352
4/1/2022 0:00	p6862	Pablo Bonilla	Western Conference	t29	21-25	0	0	252	151	352

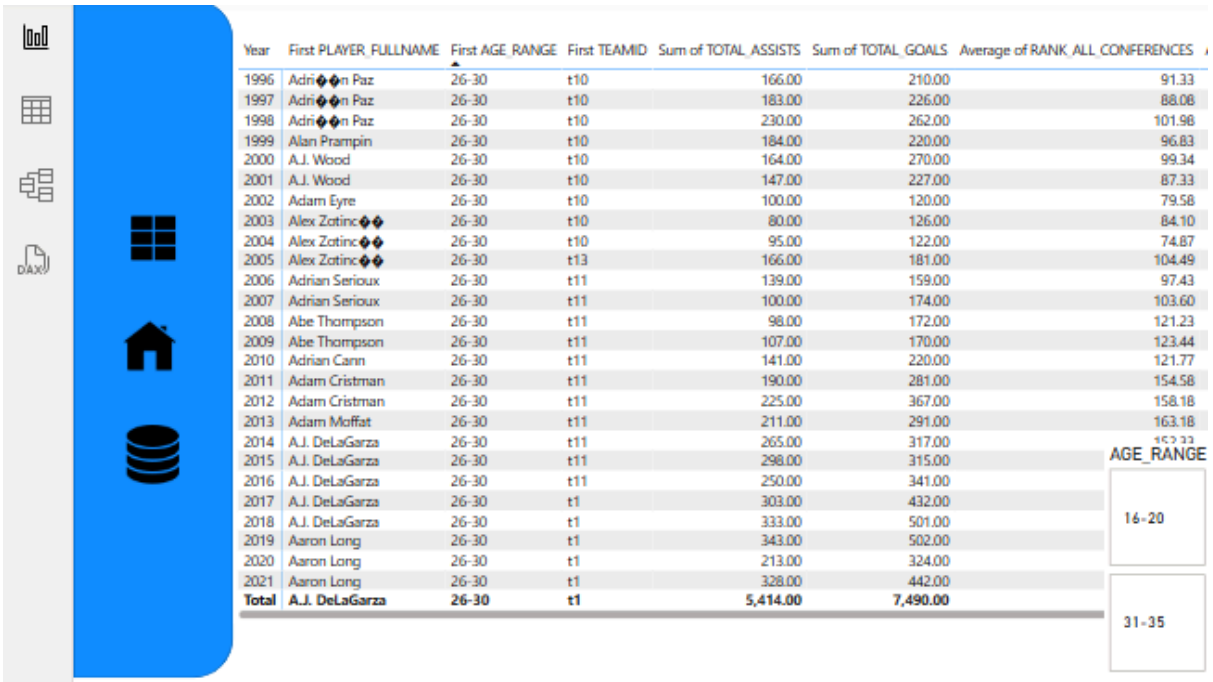
7.7.3. Pivot UI redevelopment



Years (SEASC	PLAYER	PLAYER_FULLNAME	CONFERENCE	TEAMID	AGE_RANGE	Values Average of RANK_ALL_CONFERENCES	Average of RANK_WITHIN_CONFERENCE	Sum of TOTA
1996						89.56326531	45.25306122	
1997						93.92156863	47.45098039	
1998	p10002	A.J. Wood	Eastern Conferen	t9	21-25	67	37	
	p10025	Bill Andracki	Eastern Conferen	t20	26-30	145	86	
	p10032	Billy Thompson	Eastern Conferen	t8	26-30	110	62	
	p10033	Billy Walsh	Eastern Conferen	t19	21-25	145	86	
	p10039	Brian Loftin	Eastern Conferen	t34	21-25	145	86	
	p10079	Vicente Figueroa	Western Confere	t31	16-20	145	60	
	p1019	David Kramer	Western Confere	t31	21-25	145	60	
	p1036	David Nakhid	Eastern Conferen	t24	31-35	145	86	
	p1048	David Quesada	Western Confere	t17	26-30	145	60	
	p1065	David Vaudreuil	Eastern Conferen	t20	31-35	67	37	
			Western Confere	t7	31-35	145	60	
	p1081	David Winner	Eastern Conferen	t20	26-30	145	86	
				t8	26-30	145	86	
	p116	Alen Kozi	Eastern Conferen	t20	21-25	145	86	

Q3.5 Q3.6 Q3.7 Q3.8 Q3.9 Q3.11 Q3.12 Team_Perf_Analysis vw_GameResults Q4.1 Q4.2 Q4.3 Q4.4 Q4.5 Q4.6 Q4.7

7.7.4. Power BI redevelopment



Year	First PLAYER_FULLNAME	First AGE_RANGE	First TEAMID	Sum of TOTAL_ASSISTS	Sum of TOTAL_GOALS	Average of RANK_ALL_CONFERENCES
1996	Adri□□n Paz	26-30	t10	166.00	210.00	91.33
1997	Adri□□n Paz	26-30	t10	183.00	226.00	88.08
1998	Adri□□n Paz	26-30	t10	230.00	262.00	101.98
1999	Alan Prampin	26-30	t10	184.00	220.00	96.83
2000	A.J. Wood	26-30	t10	164.00	270.00	99.34
2001	A.J. Wood	26-30	t10	147.00	227.00	87.33
2002	Adam Eyre	26-30	t10	100.00	120.00	79.58
2003	Alex Zotinc□□	26-30	t10	80.00	126.00	84.10
2004	Alex Zotinc□□	26-30	t10	95.00	122.00	74.87
2005	Alex Zotinc□□	26-30	t13	166.00	181.00	104.49
2006	Adrian Serioux	26-30	t11	139.00	159.00	97.43
2007	Adrian Serioux	26-30	t11	100.00	174.00	103.60
2008	Abe Thompson	26-30	t11	98.00	172.00	121.23
2009	Abe Thompson	26-30	t11	107.00	170.00	123.44
2010	Adrian Cann	26-30	t11	141.00	220.00	121.77
2011	Adam Cristman	26-30	t11	190.00	281.00	154.58
2012	Adam Cristman	26-30	t11	225.00	367.00	158.18
2013	Adam Moffat	26-30	t11	211.00	291.00	163.18
2014	A.J. DeLaGarza	26-30	t11	265.00	317.00	152.33
2015	A.J. DeLaGarza	26-30	t11	298.00	315.00	
2016	A.J. DeLaGarza	26-30	t11	250.00	341.00	
2017	A.J. DeLaGarza	26-30	t1	303.00	432.00	
2018	A.J. DeLaGarza	26-30	t1	333.00	501.00	
2019	Aaron Long	26-30	t1	343.00	502.00	
2020	Aaron Long	26-30	t1	213.00	324.00	
2021	Aaron Long	26-30	t1	328.00	442.00	
Total	**A.J. DeLaGarza**	**26-30**	**t1**	**5,414.00**	**7,490.00**	

AGE_RANGE

16-20

31-35

7.7.5. Comparison

Both queries analyze player performance by age group with identical categorization logic and ranking methodology. The Sprint 6 implementation accesses player data through linked server references ([CIS444.CAMPUS-QUEST.COM,21002].[oracle1].[MLS2]), while Sprint 4 queries normalized tables directly. Both queries use identical age range binning (16-20, 21-25, 26-30, 31-35, 36+) and employ the same RANK() functions for conference and

overall rankings based on goal scoring. The key difference is architectural: Sprint 6 operates in a distributed database environment, supporting cross-server analysis, while Sprint 4 processes data within a single database context. Despite differences in execution environment, both deliver consistent analytical insights about performance distribution across age groups, conferences, and seasons. Neither implementation includes wage budget analysis due to unavailable

8. Qlist3 - MLS2 Referee Analysis

8.1. Develop a report to show performance of referee for every year. Performance, for every referee, in terms of home team, away team, home team win, home team loss, away team win, away team loss, PK given, CRDY, CRDR and with their rank in all games they refereed. Rank should be based on PK, CRDY, and CRDR.

8.1.1. Query

```
WITH RefPerformance AS (
    SELECT
        REFID,
        GAME_YEAR,
        COUNT(*) AS Total_Games,

        SUM(CASE WHEN match_result = 'Home Win' THEN 1 ELSE 0 END) AS Home_Wins,
        SUM(CASE WHEN match_result = 'Away Win' THEN 1 ELSE 0 END) AS Away_Wins,
        SUM(CASE WHEN match_result = 'Home Loss' THEN 1 ELSE 0 END) AS Home_Losses,
        SUM(CASE WHEN match_result = 'Away Loss' THEN 1 ELSE 0 END) AS Away_Losses,

        COUNT(DISTINCT HOME_TID) AS Unique_Home_Teams,
        COUNT(DISTINCT AWAY_TID) AS Unique_Away_Teams,

        -- Safely extract PK
        SUM(CASE
                WHEN CHARINDEX('PK:', NOTES) > 0
                THEN TRY_CAST(
                    LEFT(
                        LTRIM(
                            SUBSTRING(NOTES, CHARINDEX('PK:', NOTES) + 3, 5)
                        ),
                        CHARINDEX(' ', LTRIM(SUBSTRING(NOTES, CHARINDEX('PK:', NOTES) + 3,
5) + ' ')) - 1
                    ) AS INT
                )
                ELSE 0
            END
        ) AS Penalties_Given,

        -- Safely extract CRDY
        SUM(CASE
                WHEN CHARINDEX('CRDY:', NOTES) > 0
                THEN TRY_CAST(
                    LEFT(
                        LTRIM(
                            SUBSTRING(NOTES, CHARINDEX('CRDY:', NOTES) + 5, 5)
                        ),
                        CHARINDEX(' ', LTRIM(SUBSTRING(NOTES, CHARINDEX('CRDY:', NOTES) +
5, 5) + ' ')) - 1
                    ) AS INT
                )
                ELSE 0
            END
        ) AS Yellow_Cards,

        -- Safely extract CRDR
        SUM(CASE
```

```
                WHEN CHARINDEX('CRDR:', NOTES) > 0
                THEN TRY_CAST(
                    LEFT(
                        LTRIM(
                            SUBSTRING(NOTES, CHARINDEX('CRDR:', NOTES) + 5, 5)
                        ),
                        CHARINDEX(' ', LTRIM(SUBSTRING(NOTES, CHARINDEX('CRDR:', NOTES) +
5, 5) + ' ')) - 1
                    ) AS INT
                )
                ELSE 0
            END
        ) AS Red_Cards

    FROM
        [oracle1].[dbo].[vw_GameResults]
    WHERE
        REFID IS NOT NULL
    GROUP BY
        REFID, GAME_YEAR
),

RankedRefPerformance AS (
    SELECT
        *,
        RANK() OVER (
            PARTITION BY GAME_YEAR
            ORDER BY
                Penalties_Given DESC,
                Yellow_Cards DESC,
                Red_Cards DESC
        ) AS Ref_Rank
    FROM
        RefPerformance
)

SELECT
    REFID,
    GAME_YEAR,
    Total_Games,
    Unique_Home_Teams,
    Unique_Away_Teams,
    Home_Wins,
    Home_Losses,
    Away_Wins,
    Away_Losses,
    Penalties_Given,
    Yellow_Cards,
    Red_Cards,
    Ref_Rank
FROM
    RankedRefPerformance
ORDER BY
    GAME_YEAR, Ref_Rank;
```

8.1.2. Result

REFID	GAME_YEAR	Total_Games	Unique_Home_Teams	Unique_Away_Teams	Home_Wins	Home_Losses	Away_Wins	Away_Losses	Penalties_Given	Yellow_Cards	Red_Cards	Ref_Rank

r030	1996-04-01 00:00:00	17	7	8	0	0	0	0	0	0	0	1
r096	1996-04-01 00:00:00	6	3	6	0	0	0	0	0	0	0	1
r101	1996-04-01 00:00:00	11	6	7	0	0	0	0	0	0	0	1
r115	1996-04-01 00:00:00	3	2	3	0	0	0	0	0	0	0	1
r008	1996-04-01 00:00:00	5	4	5	0	0	0	0	0	0	0	1
…												
r069	2022-04-01 00:00:00	5	5	5	0	0	0	0	0	0	0	1
r094	2022-04-01 00:00:00	12	8	11	0	0	0	0	0	0	0	1
r110	2022-04-01 00:00:00	18	12	15	0	0	0	0	0	0	0	
r043	2022-04-01 00:00:00	17	13	13	0	0	0	0	0	0	0	1
r033	2022-04-01 00:00:00	13	11	12	0	0	0	0	0	0	0	1a

8.1.3. Pivot UI redevelopment

Years (GAME_YEAR) 1996

Row Labels	Count of MATCH_RESULT	Sum of HOME_PENALTY	Sum of AWAY_PENALTY
Ali Saheli	6		
Arturo Angeles	14	5	6
Brian Hall	12	16	17
Byungil Noh	1		
Edwin Resendes	2	1	0
Esse Baharmast	22	15	11
Frank Gorog	8	3	6
John Hepburn	1		
Joshua Patlak	12	6	5
Kevin Stott	14	3	7
Kevin Terry	10	2	1
Marcel Yonan	5	2	6
Paul Tamberino	8	2	1

Home

Q3.11 Q3.12 Team_Perf_Analysis vw_GameResults Q4.1 Q4.2 Q4.3 Q4.4 Q4.5 Q4.6 Q4.7 Referee_Analysis Q5.1 Q5.2 GAM

Accessibility: Investigate

8.1.4. Power BI redevelopment

FIRST_LAST_NAME	First MATCH_RESULT	Sum of HOME_PENALTY	Sum of AWAY_PENALTY
	Away Win	3.00	1.00
Abbey Okulaja	Away Win		
Alan Kelly	Away Win	4.00	5.00
Alberto Giordano	Away Win	7.00	6.00
Alejandro Mariscal	Away Win		
Alex Chilowicz	Away Win		
Alex Prus	Away Win	2.00	6.00
Ali Mohamed Bujsaim	Away Win	1.00	4.00
Ali Saheli	Away Win	14.00	15.00
Allen Chapman	Away Win	14.00	16.00
Andres Pfefferkorn	Away Win		
Andrew Barnes	Away Win	9.00	8.00
Armando Villarreal	Away Win	20.00	20.00
Arturo Angeles	Away Win	14.00	13.00
Baldomero Toledo	Away Win	4.00	2.00
Ben Chouaf	Home Win		
Ben Jones	Away Win		
Brett Hugo	Away Win		
Brian Hall	Away Win	60.00	65.00
Byungil Noh	Away Win	0.00	2.00
Chris Penso	Away Win		
Colin Tait	Draw		
Daniel Fitzgerald	Draw		
Dave Gantar	Away Win		
Drew Fischer	Away Win	3.00	5.00
Edvin Jurisevic	Away Win		
Total	**Away Win**	**550.00**	**569.00**

8.1.5. Comparison

Both queries provide the same result and answer, however, the Oracle query from sprint 5 builds that analysis from normalized, multi-table sources, offering greater control and customization. The MSSQL query from sprint 6, seen above, relies on a streamlined dataset and uses a more comprehensive performance metric, making it ideal for quick insights.

Scrum Report

Scrum_Matrix – Sprint 6

Scrum		David Onyango			Eunsoo Park			Kiara Boyer			Pramudya Wicaksono			Rei Onishi		
1	Prepare Linked Server	Assign	Verify	Done	Assign	Verify	Done	Assign	Verify	Done	Assign	Verify	Done	Assign	Verify	Done
	Task1.1 Link servers	A	EP	A							A	RO	A			
	Task1.2 Develop Views in the linked server	A	EP	A							A	RO	A			
2	Redevelop Queries															
	Task2.1 Qlist1							A	PW	A						
	Task2.2 Qlist2													A	DO	A
	Task2.3 Qlist3				A	KB	A	A	PW	A				A	DO	A
3	Redevelop Pivot-UI and Power BI															
	Task3.1 Pivot UI										A	RO	A			
	Task3.2 Power BI	A	EP	A												
4	Compare differences between Sprint 2, 4, 5 and 6															
	Task4.1 Qlist1	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task4.2 Qlist2	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task4.2 Qlist3	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
5	Presentation															
	Task5.1 Presentation file				A	KB	A									
	Task5.2 Report – Cover Page				A	KB	A									
	Task5.3 Report – Table of Contents				A	KB	A									
	Task5.4 Report – Introduction							A	PW	A						
	Task5.5 Report – Deliverable	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task5.6 Report – Scrum Report				A	KB	A									
	Task5.7 Report – Final Analysis										A	RO	A			
	Task5.8 Report – Appendix				A	KB	A									
	Task5.9 Pert Chart							A	PW	A						
	Task5.10 Gantt Chart				A	KB	A									
	Task5.11 Burndown Chart				A	KB	A									
	Task5.12 Script for video recording	A	EP	A	A	KB	A	A	PW	A	A	RO	A	A	DO	A
	Task5.13 Presentation Recording	A	EP	A	A	KB	A	A	PW	A	A	RO	A			

Sprint6_deliverables: DDBS

Redo Sprint 2, 4 and 5 using DDBS Strategy

A	B	C	D	E	F	G	H	I
	Script Name	Databases	Oracle1	Oracle2	MSSQL1	MSSQL2	MSSQL3	Mongo
1	Zeota	Zeota	Zeota_Oracle1:	Zeota_Oracle2	Zeota_MSSQL1:			Zeota_Mongo:
1	Zeota Data Marts (Views)					All Data Marts		
2	IT4	IT4	IT4 (All Tables)		IT4 (All Tables)			IT4
	IT4 Data Marts (Views)					All data marts		
3	MLS	MLS	MLS(ALL Tables)		MLS(ALL Tables)			
3	MLS Data Marts (Views)					All data marts		
4	MLS2	MLS2	MLS2_Oracle1	MLS2_Oracle2	MLS2(ALL Tables)			
4	MLS2 Data Marts (Views)					All data marts		
5	PubCo	PubCo	PubCo(ALL Tables)		PubCo(ALL Tables)			
	PubCo Data Marts (Views)					All data marts		
6	PROP2	PROP2	PROP2(ALL Tables)		PROP2(ALL Tables)			
	PROP2 Data Marts (Views)					All data marts		
7	DGDB	DGDB				DGDB(ALL Tables)		
7	DGDB Data Marts (Views)					ALL Data Marts		
8	FAOES	FAOES		FAOES(All TABLES)				
8	FAOES Data Marts (Views)					ALL Data Marts		
9	Company	Company	Company(ALL Tables)		Company(ALL Tables)			
10	SP	SP	SP(ALL Tables)		SP(ALL Tables)			
	SP Data Marts (Views)					ALL Data Marts		
11	Camp	Camp	ALL Tables		ALL Tables			
12	Furn	Furn	ALL Tables		ALL Tables			
13	SPJ	SPJ	ALL Tables		ALL Tables			
	SPJ Data Marts (Views)					ALL Data Marts		
14	Book	Book	ALL Tables		ALL Tables			
	Book Data Marts (Views)					ALL Data Marts		
17	OMLS	OMLS		All objects				
18	OMLS2	OMLS2		All objects				
19	OPubCo	OPubCo		All objects				
20	OPROP2	OPROP2		All objects				
21	OOES	OOES		All objects				
22	Ozeota	OZeota		All objects				

1*(All tables without Branch_Region and Product_Price)
Zeota_Oracle1: All tables except Product Vertical fragment "**Product_Price**"
Zeota_Oracle2: Includes "**Product_Price**"
Zeota_MSSQL1:
Zeota_MSSQL2:
Zeota_Mongo:

MLS2_Oracle1: Every table except "**US_League**", "**MLS_History**",Team Vertical Fragment "**Team**"
MLS2_Oracle2: includes "**US_League**", "**MLS_History**",Team Vertical Fragment "**Team_Info**""
MLS2_MSSQL1:
MLS2_MSSQL2:

1. As the DDBS matrix shows, databases are divided in different servers.
2. Using Docker, the following architecture is considered (DDBS Strategy).
    a. Oracle1: Zeota_Oracle1, IT4, MLS, MLS2_Oracle1, …, and Book
    b. Oracle2: Zeota_Oracle2, MLS2_Oracle2, FAOES, all ORDB databases
    c. MSSQL1: Zeota_MSSQL1, IT4, MLS, MLS2_Oracle1, …, and Book
    d. MSSQL2: All data marts, DGDB,
    e. MSSQL3: contains NO DATABASES
    f. MONGO: Mongo_Zeota, IT4, SP

Note: All Data marts should be stored as views in MSSQL2 regardless where data is coming from.

Note: Data marts organized as a view SHOULD ONLY be MSSQL2

3. Implementation: All queries should be initiated from MSSQL3
    a. Redo SQL implementation of qlist1 done in Sprint2 using Linked services
        i. Format: For each query,
            1. list the query in English,
            2. indented SQL program, and
            3. result to be included in Sprint report.
            4. All program and results to be included in the sprint report
            5. Compare result with implantation done in Sprint2 and 5
                a. Verify (screenshot proof) that you are getting the same result as you did in SQL implementation of Sprint2 (screenshot proof)
        ii. Pivot Implementation
            1. Compare result with implantation done in Sprint2 and 5
            2. Verify (screenshot proof) that you are getting the same
        iii. PBI_UI sing the new DDBS Strategy (includingQList1, QList2, and QList3)
            1. Compare result with implantation done in Sprint5
        iv. Redevelop Pivot_UI using the DDBS Strategy (includingQList1, QList2, and QList3)
            1. Compare result with implantation done in Sprint5
    b. Data Marts
        i. Redo all View implementations of sprint4 using Linked services

ii. Redevelop Pivot_UI using Linked services connecting to views to provide solution to QList1, QList2, and QList3

iii. Compare the result with Sprint4

Note: Your Pivot excel should ONLY connect to views stored in MSSQL3

c. PBI

i. Redo PBI implementation using link services

ii. Compare the result with Sprint5

d. More detail will be provided later…

Final Analysis

Our team successfully completed Sprint 6 on time, meeting all objectives outlined at the start of the sprint. We focused on centralizing and validating data queries across multiple linked servers hosted via Docker, with MSSQL3 serving as the entry point. By redeveloping both the Pivot and Power BI interfaces and refactoring our SQL queries for scalability and modularity, we achieved greater consistency and maintainability across the system. All results were verified through comparative analysis with earlier sprints and confirmed for accuracy using screenshots and testing documentation.

One of our key strengths in this sprint was team coordination. Despite the complexity of the task—which involved server linking, query restructuring, and front-end redesign—we managed to stay aligned through clear communication and task tracking. Our burndown chart reflects steady progress, and the Pert chart shows logical task sequencing that helped prevent bottlenecks.

That said, we also identified areas for improvement moving forward. For example, during the Pivot implementation, Excel's limitations in counting distinct records led to inflated results when columns with multiple roles were duplicated in views. This is an inherent constraint of pivot tables, and future sprints could consider leveraging tools like Power BI for more accurate and flexible data summarization.

Additionally, while the Oracle and SQL Server queries returned comparable results, their structures differed significantly. To avoid redundant effort in cross-platform migrations, we recommend developing a standardized query template or wrapper function that can be adapted across database systems. This would streamline our development process and reduce the chances of inconsistencies.

Overall, Sprint 6 was a strong finish to our data rearchitecture efforts. The team worked collaboratively, stayed on schedule, and delivered a scalable, modular system that sets the stage for even more advanced analytics in future sprints.

Appendix

1. Title

This is team number 1 and this is our sprint 6 presentation.

2. Introduction

In this presentation, we will be going through the introduction, deliverables, scrum report, final analysis, and our overview of the sprint.
This sprint started on April 9th and then lasted until May 3rd 2025. Our Scrum master for this sprint is Kiara.

Our goal was to reorganize SQL queries and analytics tools across multiple Docker-hosted servers, ensuring everything was initiated from MSSQL3.
Some of the key tasks included setting up linked services, centralizing data marts as views in MSSQL2, and redeveloping both the Pivot and Power BI user interfaces.
We accomplished this by validating all outputs against previous sprint results, confirming accuracy through screenshots, and improving system scalability under the new DDBS setup.

3. Gantt Chart

In this Gantt Chart, you can see our updated timeline for sprint 6. We have regular meetings on Monday and Friday, but we also constantly communicated through Microsoft Teams. Specifically, for scrum meeting 4, just like last sprint, we had to meet in person to work on one local Power BI file.
As we split the work during the Sprint Planning Time, we assigned the task and set the due dates. Everyone got different work equally based on the difficulty, so we can all have a similar amount of time to work on. Basically our work was taking care of the queries that each person worked on in the last sprints.

4. Burndown Chart

In this Burndown chart, you can see the track of our project progress. The red line represents the estimated remaining effort, while the blue line shows the actual remaining effort. Ideally, these lines should track closely, but in this case, there’s a noticeable gap early in the sprint.
This occurred because I made a few adjustments to the sprint scope partway through — for example, adding one more in-person scrum meeting, refining some tasks, and updating priorities. Also, I marked several tasks as 'done' later than when they were actually completed. This delay in updating the board caused the chart to reflect a flat progress line for several days, even though the work was being done on schedule.

5. Pert Chart

In this Pert chart, you will see that we started with gathering all of our requirements. Then we moved onto connecting to all linked servers for us to then redo all 3 query lists. Then we compared our results. Next we redid both the PUI and PBI. After that, we updated the charts while working on the report. After that, we made a slideshow. Finally, we created a script which is the last step before submission.

6. Connecting Linked Server

We connected linked servers using Polybase with Azure Data Studio to virtualize databases from oracle sqldeveloper. As seen here as oracle1 and oracle2. Furthermore, we also connected the linked server to SSMS port 21000 and 21001, server 21001 is empty but

21000 is where all the SSMS database is stored based on the DDBS strategy. So, we identified oracle1, oracle2 and 21000 as the main databases that we query from.

7. Deliverables

7.1. Qlist implementation using Linked Server

In Sprint 6, we successfully reimplemented our three query lists through a distributed database strategy using linked servers. For Qlist1, we developed dynamic stored procedures in MSSQL3 that query metadata across all connected servers, consolidating system catalog information into centralized views in MSSQL2. This approach provides enterprise-wide visibility into database structures while maintaining the same analytical insights as our original implementation.
Our Qlist2 implementation accesses employee performance data through linked Oracle servers, leveraging pre-aggregated views that simplify the complex fuzzy logic processing. Despite architectural changes, we preserved consistent performance evaluations and decision categorization, enabling cross-branch comparisons through a unified analytical framework.

For Qlist3, we transformed our player performance analysis to operate through linked Oracle servers containing MLS2 data. By consolidating player statistics in central views and optimizing SQL Server window functions, we maintained consistent player rankings and performance comparisons across platforms. The implementation seamlessly handles cross-platform data type differences while delivering standardized team and conference analyses.

7.2. Pivot

During this pivot implementation, we connected excel to the views that we made from linked servers that were connected to SSMS port 21002. Furthermore, these views are then queried in the form of pivot tables that represent the queries. During the implementation, the results were comparable to the results of sprint 2 and sprint 4. Particularly showing nullable, check constraint tables, the results are accurate. However, since the view needs to also capture the multiple roles of each column, the columns are duplicated.
Therefore the queries that count records of a column, will be skewed, and presented at a higher number. This is expected, since pivot tables limitations include the inability to count distinct records.

7.3. PowerBI

For our Sprint 6 PowerBI implementation, we connected to multiple data sources through MSSQL3 as our central access point, establishing connections to Oracle1, Oracle2, MSSQL1, and MSSQL2 for comprehensive data access.
Rather than connecting directly to tables, we now leverage centralized views in MSSQL2, creating an abstraction layer that shields PowerBI from the underlying complexity while

providing consistent access patterns regardless of the source database. This approach enables standardized transformations before visualization and establishes a maintainable interface for future enhancements.
For each query list, we developed specialized dashboards that maintain visual and analytical consistency with our previous implementations. Our new dashboards provide comprehensive metadata analysis, employee performance visualization, and multi-dimensional player performance analytics across all systems. The true power of our implementation lies in its cross-server analysis capabilities, allowing us to compare structures across Oracle and SQL Server, analyze employee performance alongside player statistics, and create unified visualizations from multiple source systems.
Through rigorous testing, we confirmed that our Sprint 6 PowerBI outputs maintain exact numerical consistency with Sprint 5, preserve the same visual insights, and demonstrate comparable query performance despite the additional linked server architecture. This implementation successfully bridges the gap between distributed storage and unified analytics, creating a scalable framework that maintains consistent business insights across all query lists.

7.4. QList1

When comparing Qlist 1 queries from sprint 2 to sprint 6, and after running them, we observed that while the results were consistent, the query formats differed due to the systems used. This allowed us to explore different approaches to retrieving metadata from Oracle and SQL Server databases.

On the left, in Sprint 2, we focused on Oracle and used system views like ALL_TABLES and ALL_INDEXES to collect metadata. These queries were very straightforward and gave us quick access to things like table counts and index structures within a single schema. They were especially useful for schema-level audits and performance checks.

The linked servers in Sprint 6, on the other hand, used SQL Server and took a broader, more dynamic approach. By using cursors and dynamic SQL, we were able to query across multiple databases at once. This made Sprint 6's queries more scalable and better suited for enterprise-wide audits. They were especially good for identifying integrity issues and enforcing consistency across environments.

7.5. QList2

When querying for the answers, we encountered a significant problem with FAOES that EMP_Fuzzy Decision is empty, and when we tried to repopulate the table, the problem persisted, even though it said that the insert query went through.
EMP_Fuzzy_Decision is a key table that affects all of QList2. So, after discussing the problem with Dr. Azarbod, keeping in mind the timing constraints that persist, we concluded that, for

the purpose of this report, we can put the query that we think might work and explain the difference.

Let’s take a closer look at the difference between our Local Oracle Queries and the Linked Server Queries.
On the left, the Oracle approach joins multiple tables locally — EMPLOYEE, ORDERS, DECISIONS — applying complex business rules and fuzzy logic filters. It performs live aggregation and dynamic ranking, ensuring the results are up-to-date and validated, even excluding conflicting decisions like recommending a raise and firing the same employee.
This method is more resource-intensive but gives us precise, customizable insights — making it ideal for in-depth analysis, real-time dashboards, and complex reporting where business logic needs to be strictly applied.

On the right, we have the Linked Server Queries. These are simpler: they pull data from a pre-aggregated remote table like QList2. Filters are basic, using pattern matching like LIKE or IN. It skips the heavy lifting by depending on preprocessed data — which makes it much faster and efficient, but only as accurate as the data it's pulling from.
This approach is great for broad overviews, dashboards, and quick reporting across systems — where speed matters more than fine-tuned control.

7.6. QList3

QList 3 is still fresh in our minds, as it’s a QList we've worked with frequently in the recent sprints leading up to this one. In the query, we analyse player performance analysis. Through our distributed architecture, most queries maintained identical analytical results across platforms. Minor variations appeared in player home/away performance comparisons and nationality rankings, primarily due to SQL Server's date formatting and linked server data type handling rather than analytical differences. Team analysis queries for attendance metrics and conference rankings showed similar timestamp formatting variations while maintaining computational consistency. Referee analysis delivered completely consistent results, validating our approach. The salary-based analyses remain unimplemented due to missing data, consistent with previous sprints. Overall, our distributed implementation successfully maintains analytical integrity while enabling more scalable, enterprise-wide sports analytics capabilities.

8. Scrum Report

On this slide, I will explain our Scrum Report for the project. Here, we have outlined the various tasks assigned to each team member, including their roles and current statuses.
For example, David and Pram are tasked with preparing the linked server and developing views in the linked server, showcasing their crucial roles in setting up the foundational elements of our project. Meanwhile, Kiara, myself, and Rei are focused on redeveloping the pivot UI and making enhancements to Power BI.

Each of us has specific responsibilities that ensure our workload is balanced and that we are all contributing towards the project's success. Regular check-ins help us stay aligned and address any potential roadblocks.

9. Final Analysis

Our team successfully completed Sprint 6 on time, meeting all objectives outlined at the start of the sprint. We focused on centralizing and validating data queries across multiple linked servers hosted via Docker, with MSSQL3 serving as the entry point. By redeveloping both the Pivot and Power BI interfaces and refactoring our SQL queries for scalability and modularity, we achieved greater consistency and maintainability across the system. All results were verified through comparative analysis with earlier sprints and confirmed for accuracy using screenshots and testing documentation.

One of our key strengths in this sprint was team coordination. Despite the complexity of the task—which involved server linking, query restructuring, and front-end redesign—we managed to stay aligned through clear communication and task tracking. Our burndown chart reflects steady progress, and the Pert chart shows logical task sequencing that helped prevent bottlenecks.

That said, we also identified areas for improvement moving forward. For example, during the Pivot implementation, Excel's limitations in counting distinct records led to inflated results when columns with multiple roles were duplicated in views. This is an inherent constraint of pivot tables, and future sprints could consider leveraging tools like Power BI for more accurate and flexible data summarization.

Additionally, while the Oracle and SQL Server queries returned comparable results, their structures differed significantly. To avoid redundant effort in cross-platform migrations, we recommend developing a standardized query template or wrapper function that can be adapted across database systems. This would streamline our development process and reduce the chances of inconsistencies.

Overall, Sprint 6 was a strong finish to our data rearchitecture efforts. The team worked collaboratively, stayed on schedule, and delivered a scalable, modular system that sets the stage for even more advanced analytics in future sprints.